EXPLICIT AND ASYMPTOTIC ESTIMATES FOR THE ZERO-COUNTING FUNCTION OF THE RIEMANN ZETA-FUNCTION

VÍCTOR CASAÑA-GINER

ABSTRACT. Let $N(T)$ be the number of non-trivial zeros of the Riemann zeta-function with imaginary part in $(0,T]$, counted with multiplicity, and let $S(T)$ be the argument of ζ on the critical line, normalised so that $N(T)=\frac{1}{\pi}\vartheta(T)+1+S(T)$. Two estimates are proved by the method of Turing type that Amberger introduced for Dedekind zeta-functions and that Bellotti and Wong extended to Artin L-functions. The first is explicit: for every $T\geq 1$,

$$\left|N(T)-\frac{T}{2\pi}\log\frac{T}{2\pi e}\right|\leq 0.08382\log T+5.326,\qquad |S(T)|\leq 0.08382\log T+4.444,$$

and a piecewise linear bound, formed by six lines, lies below the explicit bounds of Amberger and of Bellotti and Wong for every $T\geq 1$. The kernels of the method are found by convex optimisation instead of by hand, the constant coming from the Euler product is computed exactly up to 10^8 and bounded beyond by a prime-number estimate of Rosser and Schoenfeld, and the elementary part is evaluated as one function of T. The second estimate is asymptotic: $\limsup_{T\to\infty}|S(T)|/\log T<0.0748883899703$. It comes from a hybrid of the kernel majorant with Jensen's formula on discs to the right of the critical line, where the growth of ζ is controlled by the subconvexity bounds of Bourgain, of Trudgian and Yang, of Tao, Trudgian and Yang, and of Heath-Brown. The constant is smaller than the value $0.0786252\ldots$ that the method of Backlund gives for discs centred near $s=1$ even under the Lindelöf hypothesis. Every inequality between elementary functions that enters a proof is certified in ball arithmetic, and no numerical information on the zeros is used.

CONTENTS

1. Introduction 2
2. Results 4
3. The identity of Turing type 5
4. The explicit estimate 8
4.1. The decomposition 8
4.2. The Euler product 9
4.3. The elementary part 11
4.4. The kernel inequality 13
4.5. Proof of Theorem 2.1 and of Corollary 2.2 14
5. Jensen discs and admissible profiles 15
6. The asymptotic estimate 17
7. Methods and remarks 18
7.1. Where the kernel inequality binds 18
7.2. Optimisation 19
7.3. Certification 19
7.4. Comparison of the two methods 20
Data availability 20
AI declaration 20
Mathematical software 20

Date: 1 November 2026.
ORCID 0000-0002-8252-2608.

References 20
Appendix A. Data tables 21
Appendix B. Programs 24
Appendix C. Outputs of the certification programs 40
Appendix D. Exact data 44

1. Introduction

Let ζ be the Riemann zeta-function and

$$\xi(s) = \tfrac{1}{2}s(s-1)\pi^{-s/2}\Gamma(\tfrac{s}{2})\zeta(s) \tag{1.1}$$

its completed form, an entire function that satisfies $\xi(s) = \xi(1-s)$ and is real and positive on the real axis [26, §2.1, (2.1.12)–(2.1.13)]. By the Euler product, ζ has no zeros in $\operatorname{Re} s > 1$, by the theorem of Hadamard and de la Vallée Poussin none on $\operatorname{Re} s = 1$ [8, Ch. 13], and by the functional equation none in $\operatorname{Re} s \le 0$ other than the trivial zeros $-2, -4, \ldots$, which are cancelled in ξ by the poles of $\Gamma(s/2)$. The zeros of ξ are therefore the non-trivial zeros of ζ, all in the strip $0 < \operatorname{Re} s < 1$. Throughout, $\rho = \beta + i\gamma$ runs over these zeros, counted with multiplicity, and

$$N(T) = \#\{\rho :\ 0 < \gamma \le T\} \qquad (T > 0), \tag{1.2}$$

so that $N(T)$ is right-continuous and $N(T) = N(T^+)$ at an ordinate T. Since $\xi(\bar{s}) = \overline{\xi(s)}$, the zeros are symmetric with respect to the real axis, and $N(T)$ counts half of the zeros with $|\gamma| \le T$. Riemann stated and von Mangoldt [31] proved that

$$N(T) = \mathcal{M}(T) + O(\log T), \qquad \mathcal{M}(T) = \frac{T}{2\pi}\log\frac{T}{2\pi e}. \tag{1.3}$$

The function

$$\vartheta(T) = \operatorname{Im}\operatorname{Log}\Gamma(\tfrac{1}{4} + \tfrac{iT}{2}) - \frac{T}{2}\log\pi \tag{1.4}$$

is the Riemann–Siegel theta function, with the branch of $\operatorname{Log}\Gamma$ that is real on $(0,\infty)$, and the argument of ζ on the critical line is the function S defined for $T > 0$ by

$$N(T) = \frac{1}{\pi}\vartheta(T) + 1 + S(T) \tag{1.5}$$

[26, §9.3]; with the convention (1.2), $S(T) = S(T^+)$ at the ordinates. Since $\vartheta(T)/\pi + 1 = \mathcal{M}(T) + \frac{7}{8} + O(1/T)$ (Lemma 4.8 below), the estimate (1.3) is equivalent to $S(T) = O(\log T)$.

The problem of making (1.3) explicit, in the form

$$\bigl|N(T) - \mathcal{M}(T)\bigr| \le C_1\log T + C_2\log\log T + C_3 \qquad (T \ge T_0), \tag{1.6}$$

goes back to Backlund [2], after von Mangoldt [31] and Grossmann [12], and was taken up by Rosser [22], Trudgian [27, 28], Platt and Trudgian [19], Hasanalizade, Shen and Wong [13] and Bellotti and Wong [4], all by the method of Backlund: the variation of $\arg\zeta$ on the horizontal segment from $\frac{1}{2} + iT$ to the right is controlled by Jensen's formula on a disc centred at a point $c + iT$ with $c > 1$, and the growth of ζ on that disc by convexity or subconvexity bounds. Table 1 lists the constants obtained in this way, in the form (1.6) and with the first height T_0 from which each bound is proved. The leading constant obtainable by this method is limited from below: for the family of discs used in [7], centred near $s = 1$ with radius near 1, the leading constant cannot be smaller than

$$\Lambda(0) = \frac{3\sqrt{3} - \pi}{12\pi\log 2} = 0.0786252\ldots \tag{1.7}$$

for any admissible growth profile of ζ, even under the Lindelöf hypothesis [7, Remark 5.2]; whether other discs could do better is not settled there.

In 2025 Amberger [1] introduced a different method, inspired by Turing's [30, Lemma 1], for Dedekind zeta-functions, and Bellotti and Wong [5] extended it to Artin L-functions and sharpened it. Instead of a disc, it uses an exact identity: for $d > \frac{1}{2}$, $N(T)$ equals a combination of the arguments of ξ at the two points $\frac{1}{2}+d+iT$ and $\frac{1}{2}+2d+iT$, which lie in the zero-free half-plane $\operatorname{Re} s > 1$, plus a sum over all the zeros of an explicit function $f(\frac{1}{2}-\beta, T-\gamma)$ (Lemma 3.1). The sum over the zeros is then estimated from above and from below by sums of functions of the form $\operatorname{Re}(x+it)^{-j}$ and $\operatorname{Im}(x+it)^{-j}$, whose sums over the zeros are the derivatives of ξ'/ξ at $\frac{1}{2}+d+iT$, again in the zero-free half-plane. For the Riemann zeta-function the method gives $|N(T) - \mathcal{M}(T)| \le 0.097 \log T + 4.807$ for all $T \ge 1$ in [1, Cor. 2], and $\min(0.0946 \log T + 5.746,\ 0.097 \log T + 4.954)$ in [5, Cor. 1.3]. Both articles choose the coefficients of the majorant by trial, with three or four of them, and Bellotti and Wong [5, §3] ask what the method can give when more coefficients are allowed and chosen by convex optimisation, and whether the infimum of the leading constant over all such majorants is zero.

This article contains two contributions to that method. The first is quantitative. The majorant is taken from a family with nineteen coefficients and the coefficients are chosen by a second-order cone programme, with the inequality that defines the majorant imposed on a grid and refined by cutting planes. The constant that comes from the Euler product is computed exactly in ball arithmetic over all prime powers up to 10^8, taking, for each prime, the maximum over the common phase of all its powers (an observation of Amberger [1, Lemma 9]), and bounded beyond 10^8 by partial summation with the inequality $\psi(x) < 1.03883x$ of Rosser and Schoenfeld [23, Thm. 12]. The remaining part, which comes from Γ, from $s(s-1)$ and from the main term, is evaluated as a single function of T whose supremum over $T \ge 1$ is certified. The result (Theorem 2.1) is a piecewise linear bound, valid for all $T \ge 1$, that lies below the bounds of [1] and [5] at every height; its last line has leading constant 0.08382.

The second contribution is methodological. By the maximum principle, the constraints that determine the optimal majorant are those at hypothetical zeros with β near 0 or near 1, and the method of Turing type, which uses no information on ζ inside the critical strip, cannot exclude them. In the hybrid of Section 5 the majorant is allowed to fail near $\beta \in \{0, 1\}$, and the excess is paid by weights of the form $\log^+(R/|\rho^* - s_0|)$, where $s_0 = c + i(T+v)$ with $c > 1$ and ρ^* is the one of ρ, $1-\bar{\rho}$ with real part $\ge \frac{1}{2}$. The sum of such a weight over the zeros is bounded by Jensen's formula on the disc $|s - s_0| < R$, and the integral of $\log|\zeta|$ on its boundary by the subconvexity bounds for ζ in the strip. The coefficients of the majorant and the weights of the discs are chosen together by linear programming. This gives (Theorem 2.3)

$$\limsup_{T\to\infty} \frac{|S(T)|}{\log T} < 0.0748883899703,$$

below the value (1.7). The bound is asymptotic: the subconvexity estimates used have no explicit constants, and no height from which the inequality holds is computed.

The identity of Lemma 3.1, the family of kernel functions and the way their sums over the zeros are evaluated are due to Amberger [1] and to Bellotti and Wong [5], as is the maximum over the phase in the Euler product; they are proved here again, with the normalisation used in this article, so that the article is self-contained. The convex optimisation, the certification of every inequality in ball arithmetic [17], and the hybrid with Jensen's formula are, to the author's knowledge, new. Backlund's method and the method of Turing type are compared in Section 7, where the computations are also described. All numerical statements in the proofs are certified in Arb ball arithmetic [17] through `python-flint`; the programs and their outputs are reproduced after the bibliography (Appendices B–D), and no computation of zeros of ζ, in particular no verification of the Riemann hypothesis up to some height, is used anywhere.

2. Results

Write $L = \log T$ and, for a finite set of lines $\ell_i(L) = A_{1,i}L + A_{3,i}$,

$$\min_i \ell_i(L) = \min_i (A_{1,i}L + A_{3,i}). \tag{2.1}$$

Theorem 2.1 (Explicit estimate)**.** *For every $T \geq 1$, with $L = \log T$,*

$$\begin{aligned} |N(T) - \mathcal{M}(T)| \leq \min\{&0.20176L + 1.369,\ 0.10823L + 2.171,\ 0.09473L + 2.834,\\ &0.09349L + 2.987,\ 0.08399L + 5.202,\ 0.08382L + 5.326\}, \end{aligned} \tag{2.2}$$

$$\begin{aligned} |S(T)| \leq \min\{&0.20176L + 0.844,\ 0.10823L + 1.286,\ 0.09473L + 1.946,\\ &0.09349L + 2.099,\ 0.08399L + 4.318,\ 0.08382L + 4.444\}. \end{aligned} \tag{2.3}$$

The six lines are called $\ell_5, \ell_{20}, \ell_{60}, \ell_{150}, \ell_{400}, \ell_{6000}$, after the value of L for which each was optimised. In (2.2) they are the smallest, respectively, on $[0, L_1]$, $[L_1, L_2]$, $[L_2, L_3]$, $[L_3, L_4]$, $[L_4, L_5]$ and $[L_5, \infty)$, where the breakpoints, at which two adjacent lines agree, are

$$L_1 = \tfrac{80200}{9353} = 8.574788\ldots, \qquad L_2 = \tfrac{442}{9} = 49.111111\ldots, \qquad L_3 = \tfrac{3825}{31} = 123.387096\ldots,$$
$$L_4 = \tfrac{4430}{19} = 233.157894\ldots, \qquad L_5 = \tfrac{12400}{17} = 729.411764\ldots$$

Each line is proved on its own for all $T \geq 1$; in particular

$$|N(T) - \mathcal{M}(T)| \leq 0.08382 \log T + 5.326, \qquad |S(T)| \leq 0.08382 \log T + 4.444 \qquad (T \geq 1). \tag{2.4}$$

Corollary 2.2 (Comparison)**.** *For every $T \geq 1$ the right side of* (2.2) *is smaller than* $\min(0.0946L + 5.746,\ 0.097L + 4.954)$ [5, Cor. 1.3] *by at least* 2.231*, and smaller than the minimum of the seven bounds* $0.097L + 4.807$*,* $0.1L + 4.140$*,* $0.12L + 2.830$*,* $0.14L + 2.306$*,* $0.16L + 2.027$*,* $0.18L + 1.847$*,* $0.2L + 1.722$ *of* [1, Cor. 2 and Table 6] *by at least* 0.291*. The single line* $0.08399L + 5.202$ *is smaller than* $0.0946L + 5.746$ *for every $T \geq 1$, and smaller than* $0.097L + 4.954$ *for $L > 0.248/0.01301 = 19.062\ldots$; the single line* $0.08382L + 5.326$ *is smaller than* $0.0946L + 5.746$ *for every $T \geq 1$, and smaller than* $0.097L + 4.954$ *for $L > 0.372/0.01318 = 28.224\ldots$.*

In the form (1.6) with $C_2 = 0$ and $T_0 = 1$, the last line of Theorem 2.1 has $(C_1, C_3) = (0.08382, 5.326)$; Table 1 puts it next to the earlier results. For $S(T)$, Bellotti and Wong [4, Cor. 1.5] prove $|S(T)| \leq 0.10076L + 0.24460 \log L + 7.20844$ for $T \geq e$, and [5, Cor. 1.3] gives, through $|S| \leq |N - \mathcal{M}| + \frac{7}{8} + |\varpi|/\pi$ (Lemma 4.8), $|S(T)| \leq 0.0946L + 6.691$ for $T \geq e$.

	C_1	C_2	C_3	T_0
von Mangoldt [31] (1905)	0.4320	1.9167	13.0788	28.5580
Grossmann [12] (1913)	0.2907	1.7862	7.0120	50
Backlund [2] (1918)	0.1370	0.4430	5.2250	200
Rosser [22] (1941)	0.1370	0.4430	2.4630	2
Trudgian [27] (2012)	0.1700	0	2.8730	e
Trudgian [28] (2014)	0.1120	0.2780	3.3850	e
Platt and Trudgian [19] (2015)	0.1100	0.2900	3.165	e
Hasanalizade, Shen and Wong [13] (2022)	0.1038	0.2573	9.4925	e
Bellotti and Wong [4] (2025)	0.10076	0.24460	8.08344	e
Amberger [1, Cor. 2] (2025, v5 of 2026)	0.097	0	4.807	1
Bellotti and Wong [5, Cor. 1.3] (2026)	0.0946	0	5.746	1
Theorem 2.1, line ℓ_{400}	0.08399	0	5.202	1
Theorem 2.1, line ℓ_{6000}	0.08382	0	5.326	1

Table 1. Explicit bounds $|N(T) - \mathcal{M}(T)| \leq C_1 \log T + C_2 \log\log T + C_3$ for $T \geq T_0$. The rows up to 2025 are taken from [5, Table 3].

For the asymptotic coefficient

$$\mathfrak{a} = \limsup_{T\to\infty} \frac{|S(T)|}{\log T}, \tag{2.5}$$

the explicit bounds give $\mathfrak{a} \le 0.0946$ [5] and $\mathfrak{a} \le 0.08382$ (Theorem 2.1), and Backlund's method with the exponent pair of Bourgain gives $\mathfrak{a} \le 0.0968160\ldots$ [7, Cor. 1.2].

Theorem 2.3 (Hybrid estimate)**.** $\mathfrak{a} \le C_{\mathrm{H}}$*, where* C_{H} *is the number* (6.1) *defined by the data of Section 6, and*

$$C_{\mathrm{H}} = 0.0748883899702606\ldots < 0.0748883899703. \tag{2.6}$$

In particular $\mathfrak{a} < \Lambda(0)$*, with* $\Lambda(0)$ *as in* (1.7)*.*

Theorem 2.3 is asymptotic: it states that for every $\varepsilon > 0$ there is $T_0(\varepsilon)$ with $|S(T)| \le (C_{\mathrm{H}} + \varepsilon)\log T$ for $T \ge T_0(\varepsilon)$, and $T_0(\varepsilon)$ is not computed, because the subconvexity bounds that enter the proof (Lemma 5.3) have no explicit constants. The explicit bounds of Theorem 2.1 are not improved by it at any finite height.

Remark 2.4 (Conventions). Amberger [1] and Bellotti and Wong [5] count a zero with imaginary part exactly T with weight $\frac{1}{2}$, that is, they use $\frac{1}{2}(N(T^+)+N(T^-))$. Since the bounds of Theorem 2.1 are proved for every T that is not an ordinate and $\mathcal{M}$ is continuous, they hold for $N(T^+)$ and for $N(T^-)$ at an ordinate, hence also for their mean. The comparisons of Corollary 2.2 are therefore between bounds for the same quantity.

Remark 2.5 (On the infimum of the leading constant). Bellotti and Wong [5, §3] ask whether the infimum of C_1 over all majorants of the type used in the method, with finitely many kernel functions, is 0. The computations of Section 7 give, for the family (3.13) optimised also over $d \in \{0.7, 0.8, 0.9\}$, the leading constants 0.08831, 0.08071, 0.0800 and 0.0795 for $m = 4, 10, 20, 40$ (grid optima, not certified), which suggest a positive limit near 0.079. The same computations with the admissibility constraint imposed only at $\beta = \frac{1}{2}$, as would suffice under the Riemann hypothesis, give 0.050, 0.020 and 0.0126 for $m = 4, 10, 20$. The obstruction thus comes from hypothetical zeros near $\beta \in \{0, 1\}$, and it is exactly what the hybrid of Theorem 2.3 removes, at the price of using subconvexity bounds for ζ inside the strip.

3. The identity of Turing type

On the closed half-plane $\operatorname{Re} s \ge 1$ the function ξ has no zeros (ζ has none, Γ has none, and $\xi(1) = \frac{1}{2}$), and the half-plane is simply connected, so $\arg\xi$ has a continuous branch there; it is normalised by $\arg\xi(\sigma) = 0$ for real $\sigma \ge 1$, where $\xi(\sigma) > 0$. For $d > \frac{1}{2}$ put

$$\sigma_1 = \tfrac{1}{2} + d, \qquad \sigma_2 = \tfrac{1}{2} + 2d, \qquad s_1 = \sigma_1 + iT, \qquad s_2 = \sigma_2 + iT. \tag{3.1}$$

For $b \in \mathbb{R}$ with $|b| < d$ and $t \ne 0$ put

$$f(b,t) = 2\arctan\frac{d+b}{t} + 2\arctan\frac{d-b}{t} - \arctan\frac{2d+b}{t} - \arctan\frac{2d-b}{t}, \tag{3.2}$$

and for a zero $\rho = \beta + i\gamma$ and a height T write

$$b_\rho = \tfrac{1}{2} - \beta, \qquad t_\rho = T - \gamma. \tag{3.3}$$

Since $0 < \beta < 1$, one has $|b_\rho| < \frac{1}{2} < d$. The function f is even in b and odd in t.

Lemma 3.1 (Identity of Turing type [30, 1, 5])**.** *Let* $d > \frac{1}{2}$ *and let* $T > 0$ *not be an ordinate. Then*

$$N(T) = \frac{1}{\pi}\Big(2\arg\xi(s_1) - \arg\xi(s_2)\Big) + \frac{1}{2\pi}\sum_{\rho} f(b_\rho, t_\rho), \tag{3.4}$$

where the series converges absolutely.

Proof. The argument principle. Let $\mathcal{R}$ be the rectangle with vertices σ_2, σ_2+iT, $1-\sigma_2+iT$, $1-\sigma_2$. The function ξ has no zeros on $\partial\mathcal{R}$: none on the vertical sides, which lie outside the critical strip; none on the top side, because T is not an ordinate; and none on the real segment, because $\xi(\sigma)>0$ for all real σ (for $\sigma\ge 1$ this was noted above; for $0<\sigma<1$ one has $\sigma(\sigma-1)<0$ and $\zeta(\sigma)<0$, the latter because $(1-2^{1-\sigma})\zeta(\sigma)=\sum_{n\ge1}(-1)^{n-1}n^{-\sigma}>0$ and $1-2^{1-\sigma}<0$; the case $\sigma\le 0$ follows from $\xi(\sigma)=\xi(1-\sigma)$). Since T is not an ordinate, the zeros of ξ inside $\mathcal{R}$, namely the non-trivial zeros with $0<\gamma<T$, are exactly the $N(T)$ zeros counted in (1.2), so by the argument principle the variation of $\arg\xi$ along $\partial\mathcal{R}$, counterclockwise, is $2\pi N(T)$.

Let Δ_1 be the variation of $\arg\xi$ along the path π_1 from σ_2 to σ_2+iT and then to $\frac12+iT$. The reflection $s\mapsto 1-\bar s$ in the critical line maps π_1 onto the path π_1' from $1-\sigma_2$ to $1-\sigma_2+iT$ to $\frac12+iT$, and $\xi(1-\bar s)=\xi(\bar s)=\overline{\xi(s)}$, so the variation of $\arg\xi$ along π_1' is $-\Delta_1$. The boundary $\partial\mathcal{R}$, counterclockwise, is π_1 followed by the reverse of π_1' followed by the real segment from $1-\sigma_2$ to σ_2, on which $\arg\xi$ is constant. Hence $2\pi N(T)=\Delta_1+\Delta_1$, that is, $\Delta_1=\pi N(T)$.

Along the vertical part of π_1 the variation is $\arg\xi(s_2)$, the branch defined above. Along the horizontal part, ξ has no zeros, and with

$$F(\sigma)=\operatorname{Im}\frac{\xi'}{\xi}(\sigma+iT)=\frac{\partial}{\partial\sigma}\arg\xi(\sigma+iT)\qquad(\tfrac12\le\sigma\le\sigma_2),\tag{3.5}$$

a continuous function, the variation from σ_2+iT to $\frac12+iT$ is $-\int_{1/2}^{\sigma_2}F(\sigma)\,d\sigma$. Therefore

$$\pi N(T)=\arg\xi(s_2)-\int_{1/2}^{\sigma_2}F(\sigma)\,d\sigma.\tag{3.6}$$

Since $\int_{\sigma_1}^{\sigma_2}F(\sigma)\,d\sigma=\arg\xi(s_2)-\arg\xi(s_1)$ (the segment lies in $\operatorname{Re}s\ge1$, where the branch is the one fixed above) and $\int_{1/2}^{\sigma_1}F(\sigma+d)\,d\sigma=\int_{\sigma_1}^{\sigma_2}F$,

$$\pi N(T)=2\arg\xi(s_1)-\arg\xi(s_2)-\int_{1/2}^{\sigma_1}\bigl(F(\sigma)-F(\sigma+d)\bigr)\,d\sigma.\tag{3.7}$$

The sum over the zeros. By the Hadamard product [8, Ch. 12],

$$\frac{\xi'}{\xi}(s)=B+\sum_\rho\Bigl(\frac{1}{s-\rho}+\frac{1}{\rho}\Bigr),\tag{3.8}$$

with a real constant B and absolute convergence for every s that is not a zero. Taking imaginary parts at $\sigma+iT$ and at $\sigma+d+iT$ and subtracting, the constants cancel and

$$F(\sigma)-F(\sigma+d)=\sum_\rho\operatorname{Im}\Bigl(\frac{1}{\sigma+iT-\rho}-\frac{1}{\sigma+d+iT-\rho}\Bigr),\tag{3.9}$$

an absolutely convergent series of differences (the two series (3.8), with their regularising terms $1/\rho$, are each absolutely convergent; nothing is asserted about $\sum_\rho(s-\rho)^{-1}$ alone). For $\frac12\le\sigma\le\sigma_1$ the ρ-th term has modulus at most $d/|t_\rho|^2$, because $|\sigma+iT-\rho|\ge|t_\rho|$ and $|\sigma+d+iT-\rho|\ge|t_\rho|$; since T is not an ordinate, $t_\rho\ne0$ for every ρ, only finitely many zeros have $|t_\rho|<1$, and $\sum_{|t_\rho|\ge1}|t_\rho|^{-2}<\infty$ [26, §9.4], the series (3.9) is dominated by a summable sequence independent of σ and may be integrated term by term over $[\frac12,\sigma_1]$. With $\operatorname{Im}\frac{1}{x+it}=-\frac{t}{x^2+t^2}$ and $\int\frac{-t\,dx}{x^2+t^2}=-\arctan\frac{x}{t}$,

$$\int_{1/2}^{\sigma_1}\operatorname{Im}\Bigl(\frac{1}{\sigma+iT-\rho}-\frac{1}{\sigma+d+iT-\rho}\Bigr)d\sigma=g(b_\rho,t_\rho),\tag{3.10}$$

where

$$g(b,t)=-2\arctan\frac{b+d}{t}+\arctan\frac{b}{t}+\arctan\frac{b+2d}{t},\tag{3.11}$$

since $\sigma-\beta$ runs from b_ρ to $b_\rho+d$ in the first term and $\sigma+d-\beta$ from $b_\rho+d$ to $b_\rho+2d$ in the second. For $|b|\le\frac12$ and $|t|\ge1$, the expansion $\arctan(y/t)=y/t-y^3/(3t^3)+O(|t|^{-5})$ gives $g(b,t)=O(|t|^{-3})$, because $-2(b+d)+b+(b+2d)=0$; hence $\sum_\rho g(b_\rho,t_\rho)$ converges absolutely.

The map $\rho \mapsto 1-\bar\rho$ is a bijection of the set of zeros that preserves multiplicities and maps (b_ρ, t_ρ) to $(-b_\rho, t_\rho)$. Hence

$$\sum_\rho g(b_\rho,t_\rho) = \frac{1}{2}\sum_\rho \big(g(b_\rho,t_\rho)+g(-b_\rho,t_\rho)\big) = -\frac{1}{2}\sum_\rho f(b_\rho,t_\rho),$$

because $g(b,t)+g(-b,t) = -f(b,t)$: the terms $\arctan\frac{b}{t}$ and $\arctan\frac{-b}{t}$ cancel, and the others give (3.2) with the opposite sign. Inserting this in (3.7) proves (3.4). □

For an index $\iota = (\mathrm{X}, j)$ with $\mathrm{X} \in \{\mathrm{R},\mathrm{I}\}$ and $j \ge 1$, and for $x > 0$, $t \in \mathbb{R}$, put

$$F_{\mathrm{R},j}(x,t) = \operatorname{Re}(x+it)^{-j}, \quad F_{\mathrm{I},j}(x,t) = -\operatorname{Im}(x+it)^{-j}, \quad G_\iota(b,t) = F_\iota(d+b,t)+F_\iota(d-b,t) \ \ (|b|<d). \tag{3.12}$$

A *kernel* is a real linear combination

$$U = \sum_{\iota\in\mathcal{B}} c_\iota G_\iota, \qquad \mathcal{B} = \mathcal{B}_m = \{(\mathrm{R},j) : 1 \le j \le m\} \cup \{(\mathrm{I},j) : 2 \le j \le m\}. \tag{3.13}$$

Each G_ι is even in b; $G_{\mathrm{R},j}$ is even in t and $G_{\mathrm{I},j}$ is odd in t, because $(x-it)^{-j} = \overline{(x+it)^{-j}}$. Write $\Phi_\mathrm{R}(z) = \operatorname{Re} z$, $\Phi_\mathrm{I}(z) = -\operatorname{Im} z$, and

$$D_1(s) = \frac{1}{s} + \frac{1}{s-1} - \frac{1}{2}\log\pi + \frac{1}{2}\psi_\Gamma\Big(\frac{s}{2}\Big), \qquad D_j(s) = \frac{1}{s^j} + \frac{1}{(s-1)^j} + \frac{(-1)^{j-1}}{(j-1)!\,2^j}\psi_\Gamma^{(j-1)}\Big(\frac{s}{2}\Big) \quad (j \ge 2), \tag{3.14}$$

where $\psi_\Gamma = \Gamma'/\Gamma$ is the digamma function (the letter ψ alone denotes the Chebyshev function $\psi(x) = \sum_{n\le x}\Lambda(n)$ in Section 4.2).

Lemma 3.2 (Sums of the kernel functions over the zeros)**.** *Let $d > \frac12$, $T > 0$ and $s_1 = \sigma_1 + iT$. For every $\iota = (\mathrm{X}, j) \in \mathcal{B}$ the series $\sum_\rho G_\iota(b_\rho, t_\rho)$ converges absolutely and*

$$\sum_\rho G_\iota(b_\rho,t_\rho) = 2\,\Phi_\mathrm{X}\big(D_j(s_1)\big) - \frac{2}{(j-1)!}\sum_{n\ge2}\Lambda(n)(\log n)^{j-1}\,\Phi_\mathrm{X}\big(n^{-s_1}\big), \tag{3.15}$$

where $\Phi_\mathrm{R}(n^{-s_1}) = n^{-\sigma_1}\cos(T\log n)$ and $\Phi_\mathrm{I}(n^{-s_1}) = n^{-\sigma_1}\sin(T\log n)$.

Proof. Since $s_1 - \rho = (d+b_\rho) + it_\rho$ and $s_1 - (1-\bar\rho) = (d-b_\rho)+it_\rho$, and $\rho \mapsto 1-\bar\rho$ permutes the zeros, $\sum_\rho G_\iota(b_\rho,t_\rho) = 2\sum_\rho F_\iota(s_1-\rho)$, where $F_\iota(z)$ stands for $F_\iota(\operatorname{Re} z, \operatorname{Im} z)$, provided the series converge absolutely. For $j \ge 2$: only finitely many zeros have $|s_1-\rho| < 1$ (the zeros are isolated and s_1 is not a zero), for the others $|s_1-\rho|^{-j} \le |s_1-\rho|^{-2}$, and $\sum_\rho |s_1-\rho|^{-2} < \infty$ [26, §9.4]. For $j = 1$, $\operatorname{Re}\frac{1}{s_1-\rho} = \frac{\sigma_1-\beta}{|s_1-\rho|^2} > 0$, and by (3.8) together with $B = -\sum_\rho \operatorname{Re}\frac{1}{\rho}$ [8, Ch. 12], the series of positive terms $\sum_\rho \operatorname{Re}\frac{1}{s_1-\rho}$ converges to $\operatorname{Re}\frac{\xi'}{\xi}(s_1)$.

From (1.1), for $\operatorname{Re} s > 1$,

$$\frac{\xi'}{\xi}(s) = \frac{1}{s} + \frac{1}{s-1} - \frac{1}{2}\log\pi + \frac{1}{2}\psi_\Gamma\Big(\frac{s}{2}\Big) + \frac{\zeta'}{\zeta}(s) = D_1(s) - \sum_{n\ge2}\Lambda(n)n^{-s}, \tag{3.16}$$

which gives (3.15) for $j = 1$. For $j \ge 2$, differentiating (3.8) $j-1$ times term by term (the differentiated series converge absolutely and locally uniformly away from the zeros) gives $\sum_\rho (s-\rho)^{-j} = \frac{(-1)^{j-1}}{(j-1)!}\big(\frac{\xi'}{\xi}\big)^{(j-1)}(s)$, and differentiating (3.16) $j-1$ times gives

$$\frac{(-1)^{j-1}}{(j-1)!}\Big(\frac{\xi'}{\xi}\Big)^{(j-1)}(s) = D_j(s) - \frac{1}{(j-1)!}\sum_{n\ge2}\Lambda(n)(\log n)^{j-1}n^{-s},$$

because $\frac{d^{j-1}}{ds^{j-1}}s^{-1} = (-1)^{j-1}(j-1)!\,s^{-j}$ and $\frac{d^{j-1}}{ds^{j-1}}\frac12\psi_\Gamma(\frac{s}{2}) = 2^{-j}\psi_\Gamma^{(j-1)}(\frac{s}{2})$. Applying Φ_X and multiplying by 2 proves (3.15). □

4. The explicit estimate

4.1. **The decomposition.** Fix $d > \frac{1}{2}$, $m \geq 2$ and a kernel $U = \sum_{\iota \in \mathcal{B}_m} c_\iota G_\iota$ such that

$$f(b,t) \leq U(b,t) \qquad \text{for } 0 \leq b \leq \tfrac{1}{2},\ t \neq 0. \tag{K}$$

Since f and U are even in b, (K) holds for $|b| \leq \frac{1}{2}$. Put $\varepsilon_{\mathrm{R}} = 1$, $\varepsilon_{\mathrm{I}} = -1$ and, for $L > 0$,

$$a(L) = -\frac{1}{\pi} \sum_{(\mathrm{R},j)\in\mathcal{B}} c_{\mathrm{R},j} \frac{L^{j-1}}{(j-1)!}, \qquad b(L) = \frac{1}{\pi}\Big(\frac{e^{-dL}-2}{L} - \sum_{(\mathrm{I},j)\in\mathcal{B}} c_{\mathrm{I},j} \frac{L^{j-1}}{(j-1)!}\Big). \tag{4.1}$$

For $s = \sigma + iT$ with $\sigma > 1$ and $T > 0$ put

$$\mathcal{A}(s) = \arg s + \arg(s-1) - \frac{T}{2}\log\pi + \operatorname{Im}\operatorname{Log}\Gamma\Big(\frac{s}{2}\Big), \qquad \arg s = \arctan\frac{T}{\sigma}, \quad \arg(s-1) = \arctan\frac{T}{\sigma-1}, \tag{4.2}$$

and

$$\begin{aligned} \mathcal{E}(T) &= \frac{1}{\pi}\big(2\mathcal{A}(s_1) - \mathcal{A}(s_2)\big) - \mathcal{M}(T), \\ \mathcal{E}_u(T) = \mathcal{E}(T) + \frac{1}{\pi}\sum_{\iota\in\mathcal{B}} c_\iota \Phi_{\mathrm{X}}\big(D_j(s_1)\big), &\qquad \mathcal{E}_\ell(T) = \mathcal{E}(T) - \frac{1}{\pi}\sum_{\iota\in\mathcal{B}} \varepsilon_{\mathrm{X}} c_\iota \Phi_{\mathrm{X}}\big(D_j(s_1)\big). \end{aligned} \tag{4.3}$$

Proposition 4.1 (Two-sided estimate)**.** *Under* (K)*, for every $T > 0$ that is not an ordinate,*

$$\begin{aligned} N(T) - \mathcal{M}(T) &\leq \mathcal{E}_u(T) + \sum_{n\geq 2} \Lambda(n) n^{-\sigma_1}\big(a(\log n)\cos(T\log n) + b(\log n)\sin(T\log n)\big), \\ N(T) - \mathcal{M}(T) &\geq \mathcal{E}_\ell(T) + \sum_{n\geq 2} \Lambda(n) n^{-\sigma_1}\big(-a(\log n)\cos(T\log n) + b(\log n)\sin(T\log n)\big), \end{aligned} \tag{4.4}$$

and both series converge absolutely.

Proof. For $\operatorname{Re} s > 1$, $\operatorname{Log}\zeta(s) = \sum_{n\geq 2} \frac{\Lambda(n)}{\log n} n^{-s}$ is the branch of $\log\zeta$ that is real on $(1,\infty)$ and continuous on the half-plane, and $\xi(s) = \frac{1}{2}\cdot s\cdot(s-1)\cdot\pi^{-s/2}\cdot\Gamma(\frac{s}{2})\cdot\zeta(s)$ is a product of non-vanishing factors each of which has a continuous argument on $\operatorname{Re} s \geq 1$, $T > 0$, vanishing on the real axis: $\arg s$ and $\arg(s-1)$ as in (4.2), $-\frac{T}{2}\log\pi$, $\operatorname{Im}\operatorname{Log}\Gamma(\frac{s}{2})$ and $\operatorname{Im}\operatorname{Log}\zeta(s)$. Their sum is a continuous argument of ξ vanishing on the real axis, hence it is the branch of Section 3:

$$\arg\xi(s) = \mathcal{A}(s) - \sum_{n\geq 2} \frac{\Lambda(n)}{\log n} n^{-\sigma}\sin(T\log n) \qquad (s = \sigma + iT,\ \sigma > 1,\ T > 0). \tag{4.5}$$

Since $n^{-\sigma_2} = n^{-\sigma_1} e^{-d\log n}$,

$$\frac{1}{\pi}\big(2\arg\xi(s_1) - \arg\xi(s_2)\big) = \frac{1}{\pi}\big(2\mathcal{A}(s_1) - \mathcal{A}(s_2)\big) + \frac{1}{\pi}\sum_{n\geq 2}\Lambda(n) n^{-\sigma_1}\frac{e^{-d\log n}-2}{\log n}\sin(T\log n). \tag{4.6}$$

Upper bound. By (K) at (b_ρ, t_ρ), $f(b_\rho,t_\rho) \leq U(b_\rho,t_\rho)$ for every zero, and the series $\sum_\rho U(b_\rho,t_\rho)$ converges absolutely by Lemma 3.2. Hence, by Lemma 3.1 and (3.15),

$$\begin{aligned} \frac{1}{2\pi}\sum_\rho f(b_\rho,t_\rho) &\leq \frac{1}{2\pi}\sum_\iota c_\iota \sum_\rho G_\iota(b_\rho,t_\rho) \\ &= \frac{1}{\pi}\sum_\iota c_\iota \Phi_{\mathrm{X}}\big(D_j(s_1)\big) - \frac{1}{\pi}\sum_{n\geq 2}\Lambda(n) n^{-\sigma_1}\sum_\iota c_\iota \frac{(\log n)^{j-1}}{(j-1)!}\Phi_{\mathrm{X}}\big(e^{-iT\log n}\big), \end{aligned}$$

and $\Phi_{\mathrm{R}}(e^{-iT\log n}) = \cos(T\log n)$, $\Phi_{\mathrm{I}}(e^{-iT\log n}) = \sin(T\log n)$, so, with $Q(L) = \sum_{(\mathrm{I},j)} c_{\mathrm{I},j} L^{j-1}/(j-1)! = \frac{e^{-dL}-2}{L} - \pi b(L)$, the inner sum over ι is $-\pi a(\log n)\cos(T\log n) + Q(\log n)\sin(T\log n)$ and the

right side equals

$$\frac{1}{\pi}\sum_{\iota} c_\iota \Phi_{\mathrm{X}}(D_j(s_1)) + \sum_n \Lambda(n) n^{-\sigma_1}\big(a(\log n)\cos(T\log n) + b(\log n)\sin(T\log n)\big) \\ - \frac{1}{\pi}\sum_n \Lambda(n) n^{-\sigma_1}\frac{e^{-d\log n}-2}{\log n}\sin(T\log n).$$

Adding (4.6) cancels the last sum and gives the first line of (4.4).

Lower bound. Since $f(b,-t) = -f(b,t)$, (K) at $(b_\rho, -t_\rho)$ gives $f(b_\rho,t_\rho) \ge -U(b_\rho,-t_\rho)$, and $U(b,-t) = \sum_\iota \varepsilon_{\mathrm{X}} c_\iota G_\iota(b,t)$ by the parity of the G_ι in t. Hence

$$\frac{1}{2\pi}\sum_\rho f(b_\rho,t_\rho) \ge -\frac{1}{\pi}\sum_\iota \varepsilon_{\mathrm{X}} c_\iota \Phi_{\mathrm{X}}\big(D_j(s_1)\big) \\ + \frac{1}{\pi}\sum_{n\ge 2}\Lambda(n)n^{-\sigma_1}\sum_\iota \varepsilon_{\mathrm{X}} c_\iota \frac{(\log n)^{j-1}}{(j-1)!}\Phi_{\mathrm{X}}\big(e^{-iT\log n}\big).$$

Here the inner sum over ι is $-\pi a(\log n)\cos(T\log n) - Q(\log n)\sin(T\log n)$, by (4.1) and $\varepsilon_{\mathrm{I}} = -1$, so the right side equals

$$-\frac{1}{\pi}\sum_\iota \varepsilon_{\mathrm{X}} c_\iota \Phi_{\mathrm{X}}(D_j(s_1)) + \sum_n \Lambda(n) n^{-\sigma_1}\big(-a(\log n)\cos(T\log n) + b(\log n)\sin(T\log n)\big) \\ - \frac{1}{\pi}\sum_n \Lambda(n) n^{-\sigma_1}\frac{e^{-d\log n}-2}{\log n}\sin(T\log n).$$

Adding (4.6) cancels the last sum and gives the second line of (4.4). Absolute convergence follows from $\sigma_1 > 1$ and the polynomial growth of a and b. □

For a prime p write $\lambda_k = (\log p)\,p^{-k\sigma_1}$, $a_k = a(k\log p)$, $b_k = b(k\log p)$ and

$$f_p(x) = \sum_{k\ge 1}\lambda_k\big(a_k\cos kx + b_k\sin kx\big) \qquad (x\in\mathbb{R}), \qquad M_p = \max_{x\in\mathbb{R}} f_p(x). \tag{4.7}$$

The series converges absolutely and uniformly, so f_p is continuous and 2π-periodic, and M_p exists. Grouping the prime powers $n = p^k$ in (4.4) according to p, with the common phase $x = T\log p$, the series in the first line is $\sum_p f_p(T\log p) \le \sum_p M_p$, and the series in the second line is $\sum_p(-f_p(-T\log p)) \ge -\sum_p M_p$, because $\sum_k \lambda_k(-a_k\cos kx + b_k\sin kx) = -f_p(-x)$. This is the observation of Amberger [1, Lemma 9] that the lower bound is obtained from the upper one by changing the sign of the odd part. With

$$F_u(T) = \mathcal{E}_u(T) - A_1\log T, \qquad F_\ell(T) = -\mathcal{E}_\ell(T) - A_1\log T, \tag{4.8}$$

Proposition 4.1 gives, for every $T>0$ not an ordinate and every A_1 and $C_\zeta \ge \sum_p M_p$,

$$\big|N(T) - \mathcal{M}(T)\big| \le A_1\log T + C_\zeta + \max\big(F_u(T), F_\ell(T)\big). \tag{4.9}$$

Sections 4.2–4.4 bound the three ingredients: C_ζ, $\sup_{T\ge1}\max(F_u,F_\ell)$, and the verification of (K).

4.2. **The Euler product.**

Lemma 4.2 (Bounds for M_p)**.** *Let $h(L) = \sqrt{a(L)^2 + b(L)^2}$ and $H_{p,k} = \lambda_k h(k\log p)$. Then*

(a) $M_p \le \sum_{k\ge1} H_{p,k}$; more precisely, for every $K\ge1$, $M_p \le \max_x \sum_{k\le K}\lambda_k(a_k\cos kx + b_k\sin kx) + \sum_{k>K} H_{p,k}$;

(b) for every integer $\mathsf{M}\ge 1$ and $x_i = 2\pi i/\mathsf{M}$ $(0\le i<\mathsf{M})$, and every $K\ge1$,

$$\max_x \sum_{k\le K}\lambda_k(a_k\cos kx + b_k\sin kx) \le \max_{0\le i<\mathsf{M}}\sum_{k\le K}\lambda_k(a_k\cos kx_i + b_k\sin kx_i) + \frac{\pi}{\mathsf{M}}\sum_{k\le K} kH_{p,k}.$$

Proof. (a) $|a\cos\theta + b\sin\theta| \le \sqrt{a^2+b^2}$ for real a, b, θ, and the maximum of a sum is at most the sum of the maxima. (b) The finite sum $\phi(x) = \sum_{k\le K} \lambda_k(a_k\cos kx + b_k \sin kx)$ is differentiable with $|\phi'(x)| \le \sum_{k\le K} k\lambda_k\sqrt{a_k^2+b_k^2}$, every real x is within π/M of some x_i modulo 2π, and ϕ is 2π-periodic; the mean value theorem gives the claim. □

Lemma 4.3 (The tail)**.** *Let $N_0 \ge 2$, $g(x) = h(\log x)\, x^{-\sigma_1}$ for $x \ge N_0$, and let $G\colon [N_0,\infty) \to [0,\infty)$ be non-increasing, integrable, with $G \ge g$. Then*

$$\sum_{n>N_0} \Lambda(n) g(n) \le G(N_0)\bigl(1.03883\, N_0 - \psi(N_0)\bigr) + 1.03883 \int_{N_0}^{\infty} G(x)\, dx, \tag{4.10}$$

where $\psi(x) = \sum_{n\le x} \Lambda(n)$.

Proof. For $0 < y < G(N_0)$ put $X_y = \sup\{x \ge N_0 : G(x) > y\}$, which is finite because G is integrable and non-increasing. If $n > N_0$ and $G(n) > y$ then $n \le X_y$, so $\sum_{n>N_0,\, G(n)>y} \Lambda(n) \le \psi(X_y) - \psi(N_0) < 1.03883 X_y - \psi(N_0)$ by $\psi(x) < 1.03883x$ for all $x > 0$ [23, Thm. 12]; for $y \ge G(N_0)$ the set is empty. By Tonelli's theorem,

$$\sum_{n>N_0} \Lambda(n) g(n) \le \sum_{n>N_0} \Lambda(n) G(n) = \int_0^{\infty} \sum_{\substack{n>N_0 \\ G(n)>y}} \Lambda(n)\, dy \le \int_0^{G(N_0)} \bigl(1.03883 X_y - \psi(N_0)\bigr) dy.$$

Finally $\int_0^{G(N_0)} X_y\, dy = N_0 G(N_0) + \int_0^{G(N_0)} (X_y - N_0)\, dy$, and for $x > N_0$ one has $X_y \ge x$ if $G(x) > y$, while $X_y \ge x$ implies $G(x) \ge y$ (there are $x' \ge x$ with $G(x') > y$, and $G(x) \ge G(x')$); hence $\{y : X_y \ge x\}$ and $\{y : y < G(x)\}$ differ by a set of measure zero, and $\int_0^{G(N_0)} (X_y - N_0)\, dy = \int_{N_0}^{\infty} \int_0^{G(N_0)} \mathbf{1}_{X_y \ge x}\, dy\, dx = \int_{N_0}^{\infty} G(x)\, dx$. This is (4.10). □

The majorant G is built as follows. Let $L_0 = \log N_0$ and let $L_{\mathrm{B}} > L_0$ be such that g is non-increasing on $[e^{L_{\mathrm{B}}}, \infty)$; then on $[e^{L_{\mathrm{B}}}, \infty)$ take $G = g$. On $[L_0, L_{\mathrm{B}}]$ choose points $L_0 = u_0 < u_1 < \cdots < u_r = L_{\mathrm{B}}$, let $\bar g_i$ be an upper bound of g on $[e^{u_i}, e^{u_{i+1}}]$ computed in ball arithmetic, and put $G = \max(\bar g_i, \bar g_{i+1}, \dots, \bar g_{r-1}, g(e^{L_{\mathrm{B}}}))$ on $[e^{u_i}, e^{u_{i+1}})$; this G is non-increasing and $G \ge g$, and $\int_{N_0}^{e^{L_{\mathrm{B}}}} G\, dx$ is a finite sum. The condition on L_{B} is verified by the next lemma. Write $a(L) = \frac{1}{\pi} P_a(L)$ and $b(L) = \frac{1}{\pi}\bigl(P_b(L) + \frac{e^{-dL}-2}{L}\bigr)$ with polynomials $P_a = \sum_i \mathsf{a}_i L^i$, $P_b = \sum_i \mathsf{b}_i L^i$ read off from (4.1), let D be the larger of their degrees, P the one of P_a, P_b of degree D (if both have degree D, take $P = P_a$), and $q_i = |\mathsf{a}_i| + |\mathsf{b}_i|$.

Lemma 4.4 (Monotonicity of g)**.** *Let $u = 1/L$ and*

$$\mathrm{num}(u) = u \sum_{i\ge 1} i q_i u^{D-i} + d\, u^{D+1} + 2u^{D+2}, \qquad \mathrm{low}(u) = |\mathsf{p}_D| - \sum_{i<D} |\mathsf{p}_i| u^{D-i} - \varepsilon_P u^{D+1},$$

where p_i are the coefficients of P and $\varepsilon_P = 2$ if $P = P_b$, $\varepsilon_P = 0$ if $P = P_a$. If $\mathrm{low}(u) > 0$ and $\mathrm{num}(u)/\mathrm{low}(u) < \sigma_1$ for all $u \in [0, 1/L_{\mathrm{B}}]$, then g is strictly decreasing on $[e^{L_{\mathrm{B}}}, \infty)$.

Proof. On $[e^{L_{\mathrm{B}}}, \infty)$ one has $h > 0$ (because $\mathrm{low} > 0$ gives $P \ne 0$ there), so $\log g$ is differentiable in $L = \log x$ with $\frac{d}{dL}\log g = \frac{aa' + bb'}{a^2+b^2} - \sigma_1 \le \frac{|a'|+|b'|}{h} - \sigma_1$, using $|a|, |b| \le h$. Further $h \ge \frac{1}{\pi}|P(L)| - \frac{\varepsilon_P}{\pi L} \ge \frac{L^D}{\pi}\mathrm{low}(1/L)$, since $|\frac{e^{-dL}-2}{L}| \le \frac{2}{L}$, and $|a'| + |b'| \le \frac{1}{\pi}\bigl(\sum_{i\ge1} i q_i L^{i-1} + \frac{d}{L} + \frac{2}{L^2}\bigr) = \frac{L^D}{\pi}\mathrm{num}(1/L)$, because $|\frac{d}{dL}\frac{e^{-dL}-2}{L}| \le \frac{de^{-dL}}{L} + \frac{|e^{-dL}-2|}{L^2} \le \frac{d}{L} + \frac{2}{L^2}$. Hence $\frac{d}{dL}\log g \le \mathrm{num}/\mathrm{low} - \sigma_1 < 0$. □

Proposition 4.5 (Certified value of C_ζ)**.** *Let $N_0 = 10^8$, $P_0 = 1000$, $\mathsf{M} = 4096$. Then $\sum_p M_p \le C_\zeta$ with*

$$C_\zeta = \sum_{p\le P_0} \min\Big(\max_{i<\mathsf{M}} \sum_{k:\,p^k\le N_0} \lambda_k(a_k\cos kx_i + b_k\sin kx_i) + \frac{\pi}{\mathsf{M}}\sum_{k:\,p^k\le N_0} kH_{p,k},\ \sum_{k:\,p^k\le N_0} H_{p,k}\Big) + \sum_{\substack{P_0<p\le N_0\\ p^k\le N_0}} H_{p,k} + \mathrm{Tail}, \tag{4.11}$$

where Tail *is the right side of* (4.10) *with L_{B} verified by Lemma 4.4. The program* `cz_gen_pp.py` *(Appendix B) evaluates every term of* (4.11) *in ball arithmetic and reports an upper bound of the resulting ball. For the six kernels of Theorem 2.1 the values are in Table 2.*

Proof. For $p \le P_0$, Lemma 4.2(a) with K the largest k with $p^k \le N_0$, then (b) for the finite sum, and the powers $p^k > N_0$ go to the tail; for $P_0 < p \le N_0$, Lemma 4.2(a) with the same split; for $p > N_0$, Lemma 4.2(a) with $K = 0$. All prime powers $n > N_0$ are covered by $\sum_{n>N_0}\Lambda(n)g(n)$, bounded by Lemma 4.3 with the majorant described before Lemma 4.4. The last integral $\int_{e^{L_{\mathrm{B}}}}^\infty g\,dx$ is bounded as follows. Let $L_1 = 800$ and let $\tilde g \ge g$ be the majorant of g on $[e^{L_1},\infty)$ given below; then

$$\int_{e^{L_{\mathrm{B}}}}^\infty g\,dx \le \int_{e^{L_{\mathrm{B}}}}^{e^{L_1}} g\,dx + \int_{e^{L_1}}^\infty \tilde g\,dx,$$

because the right side minus the left side equals $\int_{e^{L_1}}^{e^{L_{\mathrm{B}}}}(\tilde g - g)\,dx + \int_{e^{L_{\mathrm{B}}}}^\infty(\tilde g - g)\,dx \ge 0$ when $L_{\mathrm{B}} \ge L_1$ (in the six runs $L_{\mathrm{B}} = 2000$ or 5000, so the first integral on the right is negative), and equals $\int_{e^{L_1}}^\infty(\tilde g - g)\,dx \ge 0$ when $L_{\mathrm{B}} < L_1$. The integral $\int_{e^{L_{\mathrm{B}}}}^{e^{L_1}} g\,dx$ is computed by the rigorous quadrature of Arb [16] in the variable L, which evaluates the integrand $\sqrt{a(L)^2 + b(L)^2}\,e^{(1-\sigma_1)L}$ on complex balls and uses Gauss–Legendre rules only on sub-intervals where the implementation certifies that the integrand is holomorphic (the principal square root returns an indeterminate ball when its argument ball meets $(-\infty,0]$, and elsewhere it is holomorphic and coincides with h on the real segment, where $a^2 + b^2 \ge 0$), falling back to the enclosure of the integrand on the sub-interval otherwise, and the majorant is $\tilde g(x) = \frac{1}{\pi}\big(\sum_i q_i L^i + 3\big)e^{-\sigma_1 L}$ with $L = \log x$, which satisfies $\tilde g \ge g$ on $[e^{L_1},\infty)$ because $h \le |a| + |b| \le \frac{1}{\pi}\big(\sum_i q_i L^i + \frac{2}{L}\big)$ and $\frac{2}{L} \le 3$; its integral $\int_{e^{L_1}}^\infty \tilde g\,dx = \frac{1}{\pi}\int_{L_1}^\infty\big(\sum_i q_i L^i + 3\big)e^{(1-\sigma_1)L}\,dL$ is a finite combination of incomplete gamma functions evaluated in closed form. The partial sums over $n \le N_0$ are exact ball sums over the 5 761 455 primes below 10^8 and their powers. □

4.3. **The elementary part.**

Lemma 4.6 (Stirling's formula with remainder)**.** *For $|\operatorname{ph} z| < \pi$, $\operatorname{Log}\Gamma(z) = (z - \frac12)\operatorname{Log} z - z + \frac12\log(2\pi) + \mathcal{R}(z)$ with $|\mathcal{R}(z)| \le \sec^2(\frac12\operatorname{ph} z)/(12|z|)$ [9, §5.11(ii)], [18, (8.4.06)]. Consequently, for $\operatorname{Re} z > 0$ and every integer $k \ge 0$,*

$$|\mathcal{R}^{(k+1)}(z)| \le (k+1)!\,\frac{2}{3|z|}\Big(\frac{2}{|z|}\Big)^{k+1}. \tag{4.12}$$

Proof. On the circle $|w - z| = \frac12|z|$ one has $|w| \ge \frac12|z|$ and $|\operatorname{ph} w - \operatorname{ph} z| \le \arcsin\frac12 = \frac{\pi}{6}$, so $|\operatorname{ph} w| < \frac{\pi}{2} + \frac{\pi}{6} = \frac{2\pi}{3}$ and $\sec^2(\frac12\operatorname{ph} w) < \sec^2\frac{\pi}{3} = 4$; hence $|\mathcal{R}(w)| \le \frac{4}{12\cdot|z|/2} = \frac{2}{3|z|}$ there, and Cauchy's estimate for the $(k+1)$-st derivative on this circle gives (4.12). □

Since $\psi_\Gamma = (\operatorname{Log}\Gamma)'$, Lemma 4.6 gives $\psi_\Gamma(z) = \operatorname{Log} z - \frac{1}{2z} + \mathcal{R}'(z)$ and, for $k \ge 1$,

$$\psi_\Gamma^{(k)}(z) = \frac{(-1)^{k-1}(k-1)!}{z^k} + \frac{(-1)^k k!}{2z^{k+1}} + \mathcal{R}^{(k+1)}(z). \tag{4.13}$$

Lemma 4.7 (The elementary part for large T)**.** *Let $T \ge T_0 \ge 2$, $z_1 = s_1/2$, $C_1 = c_{\mathrm{R},1}/(2\pi)$ and suppose $A_1 \ge C_1$. Then*

$$F_u(T) \le \frac{7}{8} - C_1 \log(2\pi) + \frac{\mathsf{K}}{T}, \qquad F_\ell(T) \le -\frac{7}{8} - C_1 \log(2\pi) + \frac{\mathsf{K}}{T}, \tag{4.14}$$

where

$$\begin{aligned}\mathsf{K} = \frac{1}{\pi}\Big(4d + \frac{\sigma_1^2}{2} + \frac{\sigma_2(\sigma_2-1)}{2} + 1 + \frac{\sigma_2^2}{2} + \sigma_1(\sigma_1-1)\Big) + \frac{|c_{\mathrm{R},1}|}{\pi}\Big((2\sigma_1-1) + \frac{\sigma_1^2}{4} + \frac{8}{3}\Big) \\ + \frac{T_0}{\pi} \sum_{\substack{\iota=(\mathrm{X},j)\in\mathcal{B}\\ j\ge 2}} |c_\iota| \Big(\frac{2}{T_0^j} + \frac{1}{(j-1)!\,2^j}\Big[(j-2)!\Big(\frac{2}{T_0}\Big)^{j-1} + \frac{(j-1)!}{2}\Big(\frac{2}{T_0}\Big)^j + j!\,\frac{4}{3T_0}\Big(\frac{4}{T_0}\Big)^j\Big]\Big).\end{aligned} \tag{4.15}$$

Proof. The term $\mathcal{E}(T)$. For $s = \sigma + iT$ with $\sigma > 1$, $\arg s = \frac{\pi}{2} - \arctan\frac{\sigma}{T}$ and $\arg(s-1) = \frac{\pi}{2} - \arctan\frac{\sigma-1}{T}$, so $\arg s + \arg(s-1) = \pi - \theta_\sigma$ with $0 \le \theta_\sigma \le \frac{2\sigma-1}{T}$. With $z = s/2$, Lemma 4.6 gives

$$\operatorname{Im}\operatorname{Log}\Gamma(z) = \Big(\frac{\sigma}{2} - \frac{1}{2}\Big)\arg z + \frac{T}{2}\log|z| - \frac{T}{2} + \operatorname{Im}\mathcal{R}(z),$$

with $\arg z = \frac{\pi}{2} - \arctan\frac{\sigma}{T}$ and $\log|z| = \log\frac{T}{2} + \frac{1}{2}\log(1 + \frac{\sigma^2}{T^2})$, and $|\mathcal{R}(z)| \le \frac{1}{3T}$ since $|\operatorname{ph} z| < \frac{\pi}{2}$ and $|z| \ge \frac{T}{2}$. Hence

$$\mathcal{A}(s) = \pi + \frac{T}{2}\log\frac{T}{2\pi} - \frac{T}{2} + \frac{(\sigma-1)\pi}{4} + \delta_\sigma, \quad \delta_\sigma = -\theta_\sigma - \frac{\sigma-1}{2}\arctan\frac{\sigma}{T} + \frac{T}{4}\log\Big(1 + \frac{\sigma^2}{T^2}\Big) + \operatorname{Im}\mathcal{R}(z),$$

with $0 \le \frac{\sigma-1}{2}\arctan\frac{\sigma}{T} \le \frac{\sigma(\sigma-1)}{2T}$ and $0 \le \frac{T}{4}\log(1+\frac{\sigma^2}{T^2}) \le \frac{\sigma^2}{4T}$. Since $2(\sigma_1-1) - (\sigma_2-1) = -\frac{1}{2}$ and $2(2\sigma_1-1) = 2\sigma_2 - 1 = 4d$,

$$\frac{1}{\pi}\big(2\mathcal{A}(s_1) - \mathcal{A}(s_2)\big) = 1 + \mathcal{M}(T) - \frac{1}{8} + \frac{2\delta_{\sigma_1} - \delta_{\sigma_2}}{\pi},$$

and, collecting the bounds (the arguments contribute at most $\frac{4d}{T}$ in absolute value, the three remainders at most $\frac{1}{T}$),

$$|\mathcal{E}(T) - \tfrac{7}{8}| \le \frac{1}{\pi T}\Big(4d + \sigma_1(\sigma_1-1) + \frac{\sigma_1^2}{2} + \frac{\sigma_2(\sigma_2-1)}{2} + \frac{\sigma_2^2}{4} + 1\Big) \le \frac{\mathsf{K}_\Gamma}{T},$$

where K_Γ is the first bracket of (4.15) (which is larger, having $\frac{\sigma_2^2}{2}$ in place of $\frac{\sigma_2^2}{4}$).

The term $j = 1$. By $\psi_\Gamma(z) = \operatorname{Log} z - \frac{1}{2z} + \mathcal{R}'(z)$ (the formula preceding (4.13)), $\operatorname{Re} D_1(s_1) = \operatorname{Re}\frac{1}{s_1} + \operatorname{Re}\frac{1}{s_1-1} - \frac{1}{2}\log\pi + \frac{1}{2}\log|z_1| - \frac{1}{2}\operatorname{Re}\frac{1}{s_1} + \frac{1}{2}\operatorname{Re}\mathcal{R}'(z_1)$, and $\log|z_1| = \log\frac{T}{2} + \frac{1}{2}\log(1+\frac{\sigma_1^2}{T^2})$, so

$$\operatorname{Re} D_1(s_1) = \frac{1}{2}\log\frac{T}{2\pi} + \delta_1, \qquad |\delta_1| \le \frac{1}{T^2}\Big((2\sigma_1-1) + \frac{\sigma_1^2}{4} + \frac{\sigma_1}{2} + \frac{8}{3}\Big) \le \frac{1}{2T}\Big((2\sigma_1-1) + \frac{\sigma_1^2}{4} + \frac{\sigma_1}{2} + \frac{8}{3}\Big),$$

using $0 \le \operatorname{Re}\frac{1}{s} \le \frac{\sigma}{T^2}$, $0 \le \operatorname{Re}\frac{1}{s-1} \le \frac{\sigma-1}{T^2}$, $|\mathcal{R}'(z_1)| \le \frac{4}{3|z_1|^2} \le \frac{16}{3T^2}$ from (4.12), and $T \ge 2$. Since $\frac{\sigma_1}{2} \le (2\sigma_1-1) + \frac{\sigma_1^2}{4} + \frac{8}{3}$ for $\sigma_1 > 1$, the last bracket halved is at most $(2\sigma_1-1) + \frac{\sigma_1^2}{4} + \frac{8}{3}$, so $|c_{\mathrm{R},1}|\,|\delta_1|/\pi$ is at most the second term of (4.15) divided by T.

The terms $j \ge 2$. $|D_j(s_1)| \le \frac{2}{T^j} + \frac{|\psi_\Gamma^{(j-1)}(z_1)|}{(j-1)!\,2^j}$, and by (4.13) with $k = j-1 \ge 1$, (4.12) and $|z_1| \ge \frac{T}{2}$,

$$|\psi_\Gamma^{(j-1)}(z_1)| \le (j-2)!\Big(\frac{2}{T}\Big)^{j-1} + \frac{(j-1)!}{2}\Big(\frac{2}{T}\Big)^j + j!\,\frac{4}{3T}\Big(\frac{4}{T}\Big)^j.$$

The resulting bound for $|D_j(s_1)|$ is of the form $\sum_{\nu\ge1}\alpha_\nu T^{-\nu}$ with $\alpha_\nu \ge 0$ (for $j = 2$ the first term is $\frac{1}{2T}$), so T times it is non-increasing in T and $|D_j(s_1)| \le T_0\,\mathrm{bound}(T_0)/T$ for $T \ge T_0$; this gives the third term of (4.15).

Conclusion. $\mathcal{E}_u(T) = \mathcal{E}(T) + \frac{c_{\mathrm{R},1}}{\pi}\operatorname{Re} D_1(s_1) + \frac{1}{\pi}\sum_{j\ge 2} c_\iota \Phi_{\mathrm{X}}(D_j(s_1)) = \frac{7}{8} + C_1\log\frac{T}{2\pi} + \Delta(T)$ with $|\Delta(T)| \le \mathsf{K}/T$, so $F_u(T) = \frac{7}{8} - C_1\log(2\pi) + (C_1 - A_1)\log T + \Delta(T) \le \frac{7}{8} - C_1\log(2\pi) + \mathsf{K}/T$, because $A_1 \ge C_1$ and $\log T \ge 0$. Likewise $\mathcal{E}_\ell(T) = \frac{7}{8} - C_1\log\frac{T}{2\pi} + \Delta_\ell(T)$ with $|\Delta_\ell| \le \mathsf{K}/T$, and $F_\ell(T) = -\frac{7}{8} - C_1\log(2\pi) + (C_1 - A_1)\log T - \Delta_\ell(T) \le -\frac{7}{8} - C_1\log(2\pi) + \mathsf{K}/T$. □

Lemma 4.8 (The theta function)**.** *For $T > 0$, $\frac{1}{\pi}\vartheta(T) + 1 = \mathcal{M}(T) + \frac{7}{8} + \frac{\varpi(T)}{\pi}$ with $|\varpi(T)| \le (\frac{1}{16} + \frac{1}{8} + \frac{1}{3})\frac{1}{T}$.*

Proof. With $z = \frac{1}{4} + \frac{iT}{2}$, Lemma 4.6 gives $\vartheta(T) = -\frac{1}{4}\arg z + \frac{T}{2}\log|z| - \frac{T}{2} - \frac{T}{2}\log\pi + \operatorname{Im}\mathcal{R}(z)$, and $\arg z = \frac{\pi}{2} - \arctan\frac{1}{2T}$, $\log|z| = \log\frac{T}{2} + \frac{1}{2}\log(1 + \frac{1}{4T^2})$. Hence $\vartheta(T) = \frac{T}{2}\log\frac{T}{2\pi} - \frac{T}{2} - \frac{\pi}{8} + \varpi(T)$ with $\varpi = \frac{1}{4}\arctan\frac{1}{2T} + \frac{T}{4}\log(1 + \frac{1}{4T^2}) + \operatorname{Im}\mathcal{R}(z)$, and the three terms are at most $\frac{1}{8T}$, $\frac{1}{16T}$ and $\frac{1}{3T}$ ($|z| \ge \frac{T}{2}$, $|\operatorname{ph} z| < \frac{\pi}{2}$). Dividing by π and using $\pi\mathcal{M}(T) = \frac{T}{2}\log\frac{T}{2\pi} - \frac{T}{2}$ gives the claim. □

For the argument, write $\mathcal{E}_u^S = \mathcal{E}_u - \frac{7}{8} - \frac{\varpi}{\pi}$, $\mathcal{E}_\ell^S = \mathcal{E}_\ell - \frac{7}{8} - \frac{\varpi}{\pi}$ and $F_u^S = \mathcal{E}_u^S - A_1\log T$, $F_\ell^S = -\mathcal{E}_\ell^S - A_1\log T$. Since $S(T) = N(T) - \mathcal{M}(T) - \frac{7}{8} - \frac{\varpi(T)}{\pi}$ by (1.5) and Lemma 4.8, Proposition 4.1 gives

$$|S(T)| \le A_1\log T + C_\zeta + \max\bigl(F_u^S(T), F_\ell^S(T)\bigr), \tag{4.16}$$

and Lemma 4.7 gives, for $T \ge T_0$, $\max(F_u^S, F_\ell^S) \le -C_1\log(2\pi) + (\mathsf{K} + \frac{1}{\pi}(\frac{1}{16} + \frac{1}{8} + \frac{1}{3}))/T$.

Proposition 4.9 (Certified suprema of the elementary part)**.** *Let $T_0 = 10^4$. The program* `elem_gen.py` *(Appendix B) covers $[1, T_0]$ by 9215 intervals $[\alpha, \beta]$ with $\beta/\alpha \le 1.001$ and, on each, encloses F_u and F_ℓ (or F_u^S, F_ℓ^S) by the centred form $F(\frac{\alpha+\beta}{2}) + F'([\alpha,\beta]) \cdot [-\frac{\beta-\alpha}{2}, \frac{\beta-\alpha}{2}]$, where $F'([\alpha,\beta])$ is a ball enclosing the derivative on the interval; for $T \ge T_0$ it uses Lemma 4.7. The resulting upper bounds for $\sup_{T\ge1}\max(F_u, F_\ell)$ and $\sup_{T\ge1}\max(F_u^S, F_\ell^S)$ for the six kernels are in Table 2.*

Proof. The functions F_u, F_ℓ are real-analytic in $T > 0$; the centred form is the mean value theorem $F(T) = F(T_{\mathrm{m}}) + F'(T_*)(T - T_{\mathrm{m}})$ for some T_* between T and T_{m}. The derivative is computed from $\frac{d}{dT}\arg s = \operatorname{Re}\frac{1}{s}$, $\frac{d}{dT}\operatorname{Im}\operatorname{Log}\Gamma(\frac{s}{2}) = \frac{1}{2}\operatorname{Re}\psi_\Gamma(\frac{s}{2})$, $\frac{d}{dT}\mathcal{M}(T) = \frac{1}{2\pi}\log\frac{T}{2\pi}$, $\frac{d}{dT}D_j(s) = iD_j'(s)$ with $D_1'(s) = -s^{-2} - (s-1)^{-2} + \frac{1}{4}\psi_\Gamma'(\frac{s}{2})$ and $D_j'(s) = -js^{-j-1} - j(s-1)^{-j-1} + \frac{(-1)^{j-1}}{(j-1)!\,2^{j+1}}\psi_\Gamma^{(j)}(\frac{s}{2})$ for $j \ge 2$, and, in the S-mode, $\frac{d}{dT}\vartheta(T) = \frac{1}{2}\operatorname{Re}\psi_\Gamma(\frac{1}{4} + \frac{iT}{2}) - \frac{1}{2}\log\pi$. The polygamma values on an interval are enclosed by $\psi_\Gamma^{(k)}(z_{\mathrm{m}}) + \psi_\Gamma^{(k+1)}(Z)\cdot i[-\frac{r}{2}, \frac{r}{2}]$, where Z is the ball of the segment and r its length in T, which is valid because $\psi_\Gamma^{(k)}(z+\eta) - \psi_\Gamma^{(k)}(z) = \int_0^1 \psi_\Gamma^{(k+1)}(z+u\eta)\eta\,du$ lies in η times the convex hull of the enclosure. All other operations are ball operations, so each interval bound is rigorous, and the supremum over $[1, T_0]$ is the maximum of the 9215 upper bounds. For $T \ge T_0$ the bound (4.14) is monotone in T, so it is evaluated at T_0. □

4.4. **The kernel inequality.** Condition (K) is an inequality between explicit elementary functions of two real variables on a half-strip. It is certified by bisection, as follows. For $t \ne 0$, by $\arctan y + \arctan\frac{1}{y} = \operatorname{sgn}(y)\frac{\pi}{2}$ and since $d \pm b > 0$ and $2d \pm b > 0$ for $0 \le b \le \frac{1}{2}$,

$$f(b,t) = \operatorname{sgn}(t)\,\pi - 2\arctan\frac{t}{d+b} - 2\arctan\frac{t}{d-b} + \arctan\frac{t}{2d+b} + \arctan\frac{t}{2d-b}, \tag{4.17}$$

so on each closed half-box $[0, \frac{1}{2}] \times [0, T_1]$ (with $\operatorname{sgn} = +1$) and $[0, \frac{1}{2}] \times [-T_1, 0]$ (with $\operatorname{sgn} = -1$) the right side of (4.17) is real-analytic, including at $t = 0$, and so is U, because $d \pm b \ge d - \frac{1}{2} > 0$.

Lemma 4.10 (Tail of the kernel inequality)**.** *Let $T_1 \ge 2d + \frac{1}{2}$, $0 \le b \le \frac{1}{2}$, $|t| \ge T_1$ and $u = 1/t$. Then*

$$t^2 f(b,t) \le 4d^3u + \mathsf{K}_5|u|^3, \qquad \mathsf{K}_5 = \frac{1}{5}\Bigl(2\bigl(d + \tfrac{1}{2}\bigr)^5 + 2d^5 + \bigl(2d + \tfrac{1}{2}\bigr)^5 + (2d)^5\Bigr), \tag{4.18}$$

and

$$t^2U(b,t) = c_{\mathrm{R},1}\Big(\frac{d+b}{1+(d+b)^2u^2} + \frac{d-b}{1+(d-b)^2u^2}\Big) + \sum_{\substack{\iota\in\mathcal{B}\\ j\geq 2}} c_\iota \sum_{x=d\pm b} \Phi_{\mathrm{X}}\Big(u^{j-2}(xu+i)^{-j}\Big), \qquad (4.19)$$

a real-analytic function of (b,u) *on* $[0,\frac{1}{2}]\times[-1/T_1,1/T_1]$ *with value* $2dc_{\mathrm{R},1} - 2c_{\mathrm{R},2}$ *at* $u=0$.

Proof. Write $f=\sum_{i=1}^4 \alpha_i \arctan(y_i u)$ with $(\alpha_i)=(2,2,-1,-1)$ and $(y_i)=(d+b,d-b,2d+b,2d-b)$; then $|y_i u| \leq (2d+\frac{1}{2})/T_1 \leq 1$. For $|y|\leq 1$, $\arctan y = y - \frac{y^3}{3} + r(y)$ with $|r(y)| \leq \frac{|y|^5}{5}$ (alternating series). Since $\sum \alpha_i y_i = 0$ and $\sum \alpha_i y_i^3 = 2(d+b)^3 + 2(d-b)^3 - (2d+b)^3 - (2d-b)^3 = -12d^3$,

$$f = 4d^3u^3 + \sum_i \alpha_i r(y_i u), \qquad \Big|\sum_i \alpha_i r(y_i u)\Big| \leq \frac{|u|^5}{5}\sum_i |\alpha_i||y_i|^5 \leq \mathsf{K}_5|u|^5,$$

using $|y_1|\leq d+\frac{1}{2}$, $|y_2|\leq d$, $|y_3|\leq 2d+\frac{1}{2}$, $|y_4|\leq 2d$. Multiplying by $t^2=u^{-2}$ gives (4.18). For (4.19): $t^2\operatorname{Re}(x+it)^{-1} = t^2\frac{x}{x^2+t^2} = \frac{x}{1+x^2u^2}$, and for $j\geq 2$, $x+it=(xu+i)/u$, so $t^2(x+it)^{-j} = u^{j-2}(xu+i)^{-j}$. At $u=0$ the terms with $j=1$ give $2dc_{\mathrm{R},1}$, the terms with $j=2$ give $c_{\mathrm{R},2}\cdot 2\operatorname{Re}(i^{-2}) + c_{\mathrm{I},2}\cdot 2(-\operatorname{Im} i^{-2}) = -2c_{\mathrm{R},2}$, and the terms with $j\geq 3$ vanish. □

Proposition 4.11 (Certification of (K))**.** *The program* `certify_hybrid.py` *(Appendix B), run with an empty list of discs, certifies* (K) *for a kernel as follows. (a) On* $[0,\frac{1}{2}]\times[0,T_1]$ *and on* $[0,\frac{1}{2}]\times[-T_1,0]$ *it evaluates* $H=U-f$*, with* f *in the form* (4.17)*, on boxes by the centred form* $H(\text{centre}) + \nabla H(\text{box})\cdot(\text{half-widths})$*, and bisects every box on which the ball of* H *does not lie in* $(0,\infty)$*; it stops when all boxes are certified. (b) For* $|t|\geq T_1$ *it certifies* $t^2U - 4d^3u - \mathsf{K}_5|u|^3 > 0$ *on* $[0,\frac{1}{2}]\times(0,1/T_1]$ *and on* $[0,\frac{1}{2}]\times[-1/T_1,0)$ *in the same way, with the expressions of Lemma 4.10; by* (4.18) *this implies* $U>f$ *there. The driver tries* $T_1=100$ *first and* $T_1=2000$ *if the first run fails; both satisfy* $T_1\geq 2d+\frac{1}{2}$*. For each of the six kernels of Theorem 2.1 the run terminated with every box certified (Appendix C).*

Proof. The centred form is the two-variable mean value theorem for the real-analytic function H on a convex box, with the gradient enclosed by balls; a positive lower bound of the enclosure on a box proves $U>f$ on the box. The union of the certified boxes is the whole half-strip $0\leq b\leq\frac{1}{2}$, $0<|t|\leq T_1$ (the boxes are closed and include $t=0$, where (4.17) is the continuous extension), together with $|t|\geq T_1$ by (b). □

line	d	m	A_1	$C_\zeta <$	$\sup\max(F_u,F_\ell)\leq$	A_3^N	$\sup\max(F_u^S,F_\ell^S)\leq$	A_3^S
ℓ_5	1	5	0.20176	0.70360002	0.66536630	1.369	0.13950212	0.844
ℓ_{20}	1.2	5	0.10823	1.46807534	0.70207096	2.171	−0.18278630	1.286
ℓ_{60}	1	5	0.09473	2.10426949	0.72894181	2.834	−0.15847558	1.946
ℓ_{150}	0.85	5	0.09349	2.25578346	0.73031710	2.987	−0.15757622	2.099
ℓ_{400}	0.85	10	0.08399	4.46374163	0.73756572	5.202	−0.14582663	4.318
ℓ_{6000}	0.85	10	0.08382	4.58951035	0.73573766	5.326	−0.14641029	4.444

TABLE 2. The six kernels of Theorem 2.1: parameters, certified upper bounds for C_ζ and for the suprema of the elementary part, and the resulting constants $A_3^N = \lceil C_\zeta + \sup\max(F_u,F_\ell)\rceil_3$, $A_3^S = \lceil C_\zeta + \sup\max(F_u^S,F_\ell^S)\rceil_3$ (rounded up to three decimals); $A_1 = \lceil c_{\mathrm{R},1}/(2\pi)\rceil_5$. The coefficients c_ι are in Appendix A.

4.5. **Proof of Theorem 2.1 and of Corollary 2.2.**

Proof of Theorem 2.1. Each line $A_1L + A_3$ of (2.2) corresponds to a kernel listed in Table 2 and Appendix A, with $A_1 = \lceil C_1 \rceil_5 \geq C_1 = c_{\mathrm{R},1}/(2\pi)$. For it, (K) holds by Proposition 4.11; $\sum_p M_p \leq C_\zeta$ by Proposition 4.5; and $\sup_{T\geq 1} \max(F_u, F_\ell)$ is bounded by Proposition 4.9, with $A_1 \geq C_1$ as required in Lemma 4.7. By (4.9), for every $T \geq 1$ that is not an ordinate, $|N(T) - \mathcal{M}(T)| \leq A_1 \log T + C_\zeta + \sup \max(F_u, F_\ell) \leq A_1 \log T + A_3^N$. At an ordinate T, $N(T) = \lim_{t\to T^+} N(t)$ and $\mathcal{M}$ is continuous, so the inequality persists. The same argument with (4.16) gives (2.3), with $S(T) = S(T^+)$ at the ordinates. □

Proof of Corollary 2.2. The difference between the minimum of finitely many lines and the minimum of finitely many other lines is a piecewise linear function of $L \geq 0$, whose minimum over $[0, \infty)$ is attained at $L = 0$, at a breakpoint of one of the two envelopes, or, if the slopes allow, approached as $L \to \infty$. Here the smallest slope of the bounds of [5] and of [1] is 0.0946 and 0.097 respectively, larger than 0.08382, so the differences tend to $+\infty$. Evaluating at $L = 0$ and at the finitely many breakpoints, in exact rational arithmetic, gives the minimal differences 2.231 (at $L = 49.11$) and 0.291 (at $L = 8.57$), both positive. The statements about single lines are the comparisons of two lines each. When both the slope and the constant of the new line are smaller, the inequality holds for all $L \geq 0$: this is the case for $0.08399L + 5.202$ and for $0.08382L + 5.326$ against $0.0946L + 5.746$. When the new line has the smaller slope but the larger constant, the inequality holds exactly for L larger than the quotient of the difference of the constants by the difference of the slopes: $(5.202 - 4.954)/(0.097 - 0.08399)$ and $(5.326 - 4.954)/(0.097 - 0.08382)$ against $0.097L + 4.954$. □

5. Jensen discs and admissible profiles

Definition 5.1 (Admissible profile)**.** An *admissible profile* is a continuous function $\mathfrak{e}\colon [\frac{1}{2}, 1] \to [0, \infty)$ with $\mathfrak{e}(1) = 0$ such that, for every $\varepsilon > 0$,

$$|\zeta(\sigma + it)| \leq K_\varepsilon |t|^{\mathfrak{e}(\sigma)+\varepsilon} \qquad \text{for } \tfrac{1}{2} \leq \sigma \leq 1,\ |t| \geq 2, \tag{5.1}$$

with a constant K_ε independent of σ and t. It is extended by $\mathfrak{e}(\sigma) = 0$ for $\sigma > 1$.

For $\frac{1}{2} \leq \sigma \leq 1$ let $\mu(\sigma)$ be the infimum of the $\xi \geq 0$ such that $\zeta(\sigma + it) \ll |t|^\xi$ as $|t| \to \infty$ [26, §5.1]; μ is convex, hence continuous, on $[\frac{1}{2}, 1]$, and $\mu(1) = 0$. By definition, for every $\varepsilon > 0$ and each fixed σ, $\zeta(\sigma + it) \ll_{\sigma,\varepsilon} |t|^{\mu(\sigma)+\varepsilon}$ for $|t| \geq 2$.

Lemma 5.2 (Interpolation of pointwise bounds)**.** *Let $\frac{1}{2} = \varsigma_0 < \varsigma_1 < \cdots < \varsigma_n = 1$ and $e_0 \geq e_1 \geq \cdots \geq e_n = 0$ be such that, for each i and each $\varepsilon > 0$, $|\zeta(\varsigma_i + it)| \leq K_{i,\varepsilon}|t|^{e_i+\varepsilon}$ for $|t| \geq 2$. Then the piecewise linear function $\mathfrak{e}$ with $\mathfrak{e}(\varsigma_i) = e_i$ is an admissible profile.*

Proof. Fix $\varepsilon > 0$ and $i < n$, and put $\mathcal{Z}(s) = (s-1)\zeta(s)$, $Q = 10$. On the strip $\varsigma_i \leq \operatorname{Re} s \leq \varsigma_{i+1}$, $\mathcal{Z}$ is holomorphic and of polynomial growth: by [26, (2.1.4)], $\zeta(s) = \frac{s}{s-1} - s\int_1^\infty \{x\} x^{-s-1}\,dx$ for $\operatorname{Re} s > 0$, so $|\mathcal{Z}(s)| \leq |s| + |s||s-1|/\operatorname{Re} s \leq 3|s|(1 + |s|)$ there. On the line $\operatorname{Re} s = \varsigma_i$: for $|t| \geq 2$, $|s - 1| \leq |Q + s|$ (as $Q + \varsigma_i \geq |\varsigma_i - 1|$) and $|t| \leq |Q + s|$, so $|\mathcal{Z}(s)| \leq K_{i,\varepsilon}|Q + s|^{1+e_i+\varepsilon}$; for $|t| \leq 2$, $\mathcal{Z}$ is bounded on the compact segment and $|Q + s| \geq 10$, so the same bound holds with a larger constant A_i. Likewise $|\mathcal{Z}(s)| \leq B_i|Q+s|^{1+e_{i+1}+\varepsilon}$ on $\operatorname{Re} s = \varsigma_{i+1}$. Since $1+e_i+\varepsilon \geq 1+e_{i+1}+\varepsilon \geq 0$, the theorem of Phragmén–Lindelöf type of Rademacher [20] (also [10, Cor. 3] with $\alpha_2 = 0$) gives, on the strip,

$$|\mathcal{Z}(s)| \leq \big(A_i|Q+s|^{1+e_i+\varepsilon}\big)^{\frac{\varsigma_{i+1}-\sigma}{\varsigma_{i+1}-\varsigma_i}} \big(B_i|Q+s|^{1+e_{i+1}+\varepsilon}\big)^{\frac{\sigma-\varsigma_i}{\varsigma_{i+1}-\varsigma_i}} \leq \max(A_i, B_i)\,|Q+s|^{1+\mathfrak{e}(\sigma)+\varepsilon},$$

because the exponent interpolates linearly. For $|t| \geq 2$, $|Q + s|^2 = (Q + \sigma)^2 + t^2 \leq 121 + t^2 \leq 36t^2$ and $|s - 1| \geq |t|$, so $|\zeta(s)| \leq \max(A_i, B_i)6^{1+e_0+\varepsilon}|t|^{\mathfrak{e}(\sigma)+\varepsilon}$ on the strip. Taking the maximum over the n strips gives (5.1); $\mathfrak{e}$ is continuous, non-negative, and $\mathfrak{e}(1) = 0$. □

An *exponent pair* (k,ℓ) in the sense of [26, §5.20] satisfies $0 \le k \le \frac{1}{2} \le \ell \le 1$, and

$$\mu(\ell - k) \le k \tag{5.2}$$

[24, Lemma 23], [26, (5.20.1)].

Lemma 5.3 (The profile $\mathfrak{e}_{\mathrm{P}}$)**.** *Let $\mathcal{N}$ be the set of the following points (ς, e) with $\frac{1}{2} \le \varsigma \le 1$, for each of which $\mu(\varsigma) \le e$ is a theorem:*

- *(i)* $(\frac{1}{2}, \frac{13}{84})$ [6, Thm. 5]*;*
- *(ii) the breakpoints of the piecewise linear bound for μ of Trudgian and Yang* [29, Thm. 2.4]*, that is, $(\varsigma, \mu^{\mathrm{TY}}(\varsigma))$ for $\varsigma \in \{\frac{88225}{153852}, \frac{521}{796}, \frac{53141}{76066}, \frac{3620}{5119}, \frac{52209}{69128}, \frac{1389}{1736}, \frac{134765}{163248}, \frac{18193}{21906}, \frac{249}{280}, \frac{277}{300}\}$, where μ^{TY} is the bound of that theorem (its ten pieces agree at the common endpoints, and the last piece is stated there up to $\frac{277}{300}$);*
- *(iii) $(\ell - k, k)$ for the four exponent pairs (k,ℓ) of Tao, Trudgian and Yang* [24, Thm. 20]*: $(\frac{89}{1282}, \frac{997}{1282})$, $(\frac{652397}{9713986}, \frac{7599781}{9713986})$, $(\frac{10769}{351096}, \frac{609317}{702192})$, $(\frac{89}{3478}, \frac{15327}{17390})$, by* (5.2)*;*
- *(iv) (ς_n, μ_n) with $\varsigma_n = 1 - \frac{3n^2-3n+2}{n(n-1)^2(n+2)}$ and $\mu_n = \frac{2}{(n-1)^2(n+2)}$ for $6 \le n \le 399$: by Heath-Brown* [14, Thm. 2]*, $(\frac{2}{(n-1)^2(n+2)}, 1 - \frac{3n-2}{n(n-1)(n+2)} + \varepsilon')$ is an exponent pair for every $n \ge 3$ and $\varepsilon' > 0$, so* (5.2) *gives $\mu(\varsigma_n + \varepsilon') \le \mu_n$ for every $\varepsilon' > 0$, and $\mu(\varsigma_n) \le \mu_n$ by the continuity of μ;*
- *(v)* $(1, 0)$*, since $\zeta(1 + it) \ll \log|t|$* [26, Thm. 3.5].

Let $\mathfrak{e}_{\mathrm{P}}$ be the piecewise linear function through the vertices of the lower convex hull of $\mathcal{N}$ (Table 5 in Appendix A lists them; the four points of (ii) with $\varsigma \in \{\frac{3620}{5119}, \frac{134765}{163248}, \frac{18193}{21906}, \frac{249}{280}\}$ lie on or above chords of other points and are not vertices). Then $\mathfrak{e}_{\mathrm{P}}$ is an admissible profile.

Proof. At each vertex (ς_i, e_i) of the hull, $\mu(\varsigma_i) \le e_i$, so $|\zeta(\varsigma_i + it)| \le K_{i,\varepsilon}|t|^{e_i+\varepsilon}$ for $|t| \ge 2$ and every $\varepsilon > 0$, by the definition of μ. The vertices have $\varsigma_0 = \frac{1}{2}$, $\varsigma_{\mathrm{last}} = 1$, and non-increasing ordinates (the values in Table 5 decrease, and the last is 0). Lemma 5.2 applies. □

The points of $\mathcal{N}$ that are not vertices are not needed: the chord between two vertices is a valid bound by Lemma 5.2, and it is smaller. No use is made of the convexity of μ beyond its continuity in (iv).

A *disc datum* is a triple (c, R, v) of real numbers with $c > 1$, $R > 0$ and $c - R \ge \frac{1}{2}$. Its weight is

$$W(b,t) = \log^+ \frac{R}{\sqrt{(c - \frac{1}{2} - |b|)^2 + (t+v)^2}} \qquad (|b| < \tfrac{1}{2},\ t \in \mathbb{R}), \tag{5.3}$$

and, for an admissible profile $\mathfrak{e}$,

$$J(\mathfrak{e}; c, R) = \frac{1}{2\pi}\int_0^{2\pi} \mathfrak{e}(c + R\cos\theta)\, d\theta = \frac{1}{\pi}\int_0^{\pi} \mathfrak{e}(c + R\cos\theta)\, d\theta. \tag{5.4}$$

Since $\mathfrak{e} = 0$ on $[1, \infty)$ and $c - R \ge \frac{1}{2}$, only the values of $\mathfrak{e}$ on $[c - R, 1] \subset [\frac{1}{2}, 1]$ enter.

Lemma 5.4 (Jensen bound)**.** *Let $\mathfrak{e}$ be an admissible profile, (c, R, v) a disc datum and $\varepsilon > 0$. There is T_ε such that for all $T \ge T_\varepsilon$,*

$$\sum_\rho W(b_\rho, t_\rho) \le 2\big(J(\mathfrak{e}; c, R) + \varepsilon\big)\log T, \tag{5.5}$$

and the same bound holds for $\sum_\rho W(b_\rho, -t_\rho)$.

Proof. Let $s_0 = c + i(T + v)$ and, for a zero ρ, let ρ^* be the one of ρ, $1 - \bar\rho$ with real part $\ge \frac{1}{2}$, so $\rho^* = \frac{1}{2} + |b_\rho| + i\gamma$. Then $|\rho^* - s_0|^2 = (c - \frac{1}{2} - |b_\rho|)^2 + (t_\rho + v)^2$ and $W(b_\rho, t_\rho) = \log^+(R/|\rho^* - s_0|)$. The point ρ^* is a zero of ζ, and each zero w with $\operatorname{Re} w > \frac{1}{2}$ is ρ^* for exactly two zeros (w and

$1-\bar w$), which have the same multiplicity, while a zero w with $\operatorname{Re} w=\frac12$ is ρ^* only for $\rho=w$. Since the open disc $\mathbb{D}=\{|s-s_0|<R\}$ lies in $\operatorname{Re} s>c-R\ge\frac12$,

$$\sum_\rho W(b_\rho,t_\rho)\le 2\sum_{w\in\mathbb{D}}\log\frac{R}{|w-s_0|},\tag{5.6}$$

the sum over the zeros of ζ in $\mathbb{D}$ with multiplicity. For $T>R+|v|+1$ the closed disc $\bar{\mathbb{D}}$ does not contain $s=1$, so ζ is holomorphic on it, and $\zeta(s_0)\ne0$ because $\operatorname{Re} s_0=c>1$. Jensen's formula [25, §3.61], [21, Thm. 15.18] (which allows zeros on the circle) gives

$$\sum_{w\in\mathbb{D}}\log\frac{R}{|w-s_0|}=\frac{1}{2\pi}\int_0^{2\pi}\log|\zeta(s_0+Re^{i\theta})|\,d\theta-\log|\zeta(s_0)|.\tag{5.7}$$

By the Euler product, $|\zeta(s_0)|\ge\prod_p(1+p^{-c})^{-1}=\zeta(2c)/\zeta(c)$. On the circle, $s=\sigma+it$ with $\sigma=c+R\cos\theta\ge\frac12$ and $t=T+v+R\sin\theta\in[T-R-|v|,T+R+|v|]$, so $\log t=\log T+O(1/T)$ and $t\ge2$ for large T. Where $\sigma\le1$, (5.1) gives $\log|\zeta(s)|\le(\mathfrak{e}(\sigma)+\varepsilon/2)\log t+\log K_{\varepsilon/2}$. Where $1\le\sigma\le2$, $|\zeta(s)|\le K\log t$ [26, Thm. 3.5], and where $\sigma\ge2$, $|\zeta(s)|\le\zeta(2)$; in both cases $\log|\zeta(s)|\le\log\log t+O(1)=o(\log T)$. Inserting in (5.7) and (5.6),

$$\sum_\rho W(b_\rho,t_\rho)\le2\big(J(\mathfrak{e};c,R)+\tfrac{\varepsilon}{2}\big)\log T+O_\varepsilon(\log\log T)+2\log\frac{\zeta(c)}{\zeta(2c)},$$

and the last two terms are at most $\varepsilon\log T$ for T large. Replacing t_ρ by $-t_\rho$ in W amounts to replacing v by $-v$, which changes neither J nor the argument. □

6. The asymptotic estimate

Lemma 6.1 (Main term, asymptotic form)**.** *For fixed $d>\frac12$, as $T\to\infty$, $\frac1\pi(2\arg\xi(s_1)-\arg\xi(s_2))=\frac1\pi\vartheta(T)+1+O(1)$.*

Proof. By (4.5), $\arg\xi(s)=\mathcal{A}(s)+O(1)$ for $\sigma\in\{\sigma_1,\sigma_2\}$, since $|\sum_n\frac{\Lambda(n)}{\log n}n^{-\sigma}\sin(T\log n)|\le\log\zeta(\sigma)$. The proof of Lemma 4.7 gives $\frac1\pi(2\mathcal{A}(s_1)-\mathcal{A}(s_2))=\mathcal{M}(T)+\frac78+O(1/T)$, and Lemma 4.8 gives $\mathcal{M}(T)+\frac78=\frac1\pi\vartheta(T)+1+O(1/T)$. □

Lemma 6.2 (Sums of the kernel functions, asymptotic form)**.** *For fixed $d>\frac12$ and a kernel U, as $T\to\infty$,*

$$\sum_\rho U(b_\rho,t_\rho)=c_{\mathrm{R},1}\log T+O(1),\qquad\sum_\rho U(b_\rho,-t_\rho)=c_{\mathrm{R},1}\log T+O(1).$$

Proof. By Lemma 3.2, $\sum_\rho G_\iota(b_\rho,t_\rho)=2\Phi_{\mathrm X}(D_j(s_1))+O(1)$, the arithmetic sum being bounded by $\frac{2}{(j-1)!}\sum_n\Lambda(n)(\log n)^{j-1}n^{-\sigma_1}<\infty$. By the proof of Lemma 4.7, $\operatorname{Re}D_1(s_1)=\frac12\log\frac{T}{2\pi}+O(T^{-2})$, $\operatorname{Im}D_1(s_1)=O(1)$ and $D_j(s_1)=O(T^{-(j-1)})=o(1)$ for $j\ge2$, because $|\psi_\Gamma^{(j-1)}(z_1)|=O(T^{-(j-1)})$ by (4.13) and (4.12); for $j=2$ this is only $O(T^{-1})$, which suffices. Hence $\sum_\rho U(b_\rho,t_\rho)=c_{\mathrm{R},1}\log T+O(1)$. The second statement follows from $U(b,-t)=\sum_\iota\varepsilon_{\mathrm X}c_\iota G_\iota(b,t)$ and the same bounds. □

Fix $d=\frac{9}{10}$, $m=10$, the 19 coefficients c_ι ($\iota\in\mathcal{B}_{10}$) of Table 3, and the $K=85$ disc data (c_k,R_k,v_k) with weights $\kappa_k\ge0$ of Table 4 (Appendix A). The c_ι and κ_k are the exact binary numbers of the file `hyb_m10_P.json` (Appendix D); the disc data are the rational numbers of the table, obtained from the file by rounding to denominators $\le10^4$ and replacing R_k by $\min(R_k,c_k-\frac12)$, so that $c_k-R_k\ge\frac12$ exactly. Let W_k be the weight (5.3) of (c_k,R_k,v_k) and put

$$U=\sum_{\iota\in\mathcal{B}_{10}}c_\iota G_\iota,\qquad C_{\mathrm H}=\frac{c_{\mathrm{R},1}}{2\pi}+\frac1\pi\sum_{k=1}^K\kappa_kJ(\mathfrak{e}_{\mathrm P};c_k,R_k).\tag{6.1}$$

Proposition 6.3 (Certified hybrid inequality)**.** *For $0 \le b \le \frac{1}{2}$ and $t \neq 0$,*

$$f(b,t) \le U(b,t) + \sum_{k=1}^{K} \kappa_k W_k(b,t). \tag{6.2}$$

Moreover $C_{\mathrm{H}} \in [0.07488838997026060 \pm 2.75 \cdot 10^{-18}]$, so $C_{\mathrm{H}} < 0.0748883899703$; its two parts are $c_{\mathrm{R},1}/(2\pi) = 0.0656118899398\ldots$ and $\frac{1}{\pi}\sum_k \kappa_k J_k = 0.009276500030\ldots$.

Proof. The program `certify_hybrid.py` with the file `hyb_m10_P.json` and $T_1 = 40$ (Appendix C) proceeds as in Proposition 4.11, with these changes. (a) On the two half-boxes $[0,\frac{1}{2}] \times [0,T_1]$ and $[0,\frac{1}{2}] \times [-T_1,0]$ it bounds $H = U - f + \sum_k \kappa_k W_k^{\mathrm{low}}$ from below, where $U - f$ is enclosed by the centred form and W_k^{low} is the lower endpoint of the ball of $\log \frac{R_k}{\sqrt{(c_k-\frac{1}{2}-b)^2+(t+v_k)^2}}$ over the box when that endpoint is positive, and 0 otherwise; since $\kappa_k \ge 0$ and $W_k \ge W_k^{\mathrm{low}}$ on the box, a positive lower bound for H proves (6.2) on the box. It terminated with all boxes certified (1 648 951 boxes for $t > 0$, 2 054 381 for $t < 0$). (b) For $|t| \ge T_1 = 40$ every W_k vanishes, because $W_k(b,t) > 0$ requires $|t + v_k| < R_k$, hence $|t| < R_k + |v_k| \le 2.2 < T_1$ (Table 4), and $T_1 \ge 2d + \frac{1}{2} = 2.3$; the tail is certified as in Proposition 4.11(b) (155 116 boxes). (c) The profile $\mathfrak{e}_{\mathrm{P}}$ is piecewise linear with rational vertices and $c_k - R_k \ge \frac{1}{2}$, so by (5.4) each $J(\mathfrak{e}_{\mathrm{P}}; c_k, R_k)$ is a finite sum, over the pieces $[\varsigma_i, \varsigma_{i+1}] \cap [c_k - R_k, 1] = [p,q]$ with $\mathfrak{e}_{\mathrm{P}} = \alpha + \beta\sigma$ there, of

$$\frac{1}{\pi}\Big[(\alpha + \beta c_k)(\theta_p - \theta_q) + \beta R_k(\sin\theta_p - \sin\theta_q)\Big], \qquad \theta_\sigma = \arccos\frac{\sigma - c_k}{R_k} \quad (\theta_p \ge \theta_q),$$

since $\sigma = c_k + R_k\cos\theta$ and $d\sigma = -R_k\sin\theta\, d\theta$; the arguments of arccos lie in $[-1,1]$ because $p, q \in [c_k - R_k, c_k + R_k]$. The program evaluates these sums in ball arithmetic and reports the ball of C_{H} stated. □

Proof of Theorem 2.3. Let $\varepsilon > 0$ and let $T \to \infty$ through values that are not ordinates. All functions in (6.2) are even in b, and $|b_\rho| < \frac{1}{2}$, so (6.2) holds at (b_ρ, t_ρ) and at $(b_\rho, -t_\rho)$ for every zero; with $f(b,-t) = -f(b,t)$,

$$-U(b_\rho,-t_\rho) - \sum_k \kappa_k W_k(b_\rho,-t_\rho) \le f(b_\rho,t_\rho) \le U(b_\rho,t_\rho) + \sum_k \kappa_k W_k(b_\rho,t_\rho).$$

Summing over ρ (all series converge absolutely: those of U by Lemma 3.2, those of W_k because only finitely many zeros lie in each disc) and using Lemmas 6.2 and 5.4 with the admissible profile $\mathfrak{e}_{\mathrm{P}}$ of Lemma 5.3,

$$\Big|\frac{1}{2\pi}\sum_\rho f(b_\rho,t_\rho)\Big| \le \Big(\frac{c_{\mathrm{R},1}}{2\pi} + \frac{1}{\pi}\sum_k \kappa_k\big(J(\mathfrak{e}_{\mathrm{P}}; c_k, R_k) + \varepsilon\big)\Big)\log T + O(1)$$

for $T \ge \max_k T_\varepsilon(k)$. By Lemma 3.1 and Lemma 6.1, $S(T) = \frac{1}{2\pi}\sum_\rho f(b_\rho,t_\rho) + O(1)$, so $|S(T)| \le (C_{\mathrm{H}} + \varepsilon\sum_k \kappa_k/\pi)\log T + O(1)$. Dividing by $\log T$, letting $T \to \infty$ (at the ordinates $S(T) = S(T^+)$ is a limit of values at non-ordinates, so the limit superior over non-ordinates is $\mathfrak{a}$) and then $\varepsilon \to 0$ gives $\mathfrak{a} \le C_{\mathrm{H}}$, and Proposition 6.3 gives (2.6). Finally $0.0748883899703 < 0.0786 < \Lambda(0)$. □

7. Methods and remarks

7.1. **Where the kernel inequality binds.** For fixed coefficients, $U - f$ is harmonic in (b,t) on each of the half-strips $|b| < d$, $t > 0$ and $|b| < d$, $t < 0$: each $\arctan\frac{y}{t}$ is an argument of $t + iy$, hence harmonic in (y,t) and in (b,t), and each $F_\iota(d \pm b, t)$ is the real or imaginary part of a holomorphic function of $(d \pm b) + it$. By the minimum principle on the rectangles $[-\frac{1}{2},\frac{1}{2}] \times [\varepsilon, R]$ and $[-\frac{1}{2},\frac{1}{2}] \times [-R, -\varepsilon]$, followed by $\varepsilon \to 0$ and $R \to \infty$, $f \le U$ on $|b| \le \frac{1}{2}$, $t \neq 0$ holds as soon as $U - f \ge 0$ on the lines $b = \pm\frac{1}{2}$ and $\liminf(U - f) \ge 0$ as $t \to 0^{\pm}$ and as $|t| \to \infty$, uniformly in b. The last condition is automatic, because $U - f \to 0$ uniformly in b as $|t| \to \infty$; more precisely

$t^2(U-f) \to 2dc_{\mathrm{R},1} - 2c_{\mathrm{R},2}$ (Lemma 4.10), so the condition on the lines $b = \pm\frac{1}{2}$ forces this constant to be non-negative, and for every kernel certified here it is positive, since the boxes of part (b) of the certification contain $u = 0$. The validity of the kernel inequality is thus controlled by its boundary $b = \pm\frac{1}{2}$, that is, by hypothetical zeros with β arbitrarily close to 0 or 1; in the optimised kernels the active and nearly active constraints indeed lie near $b = \pm\frac{1}{2}$. The values of ζ on $\operatorname{Re} s > 1$, which are all the method uses, cannot exclude clusters of order $\log T$ of such zeros near height T, and the numerical optima of Remark 2.5 reflect this. The weights W_k relax exactly these constraints: in Table 4 every disc has $c_k - R_k \geq \frac{1}{2}$, and only the values of $\mathfrak{e}_{\mathrm{P}}$ on $[\frac{1}{2}, 1]$ enter C_{H}.

7.2. **Optimisation.** For fixed d the set of admissible (c_ι) in (K), and of admissible (c_ι, κ_k) in (6.2), is convex, being defined by linear inequalities indexed by (b, t). For Theorem 2.1, the quantity minimised at a target height L^* is $A_1 L^* + C_\zeta + \sup F$, where C_ζ is approximated by $\sum_n \Lambda(n) n^{-\sigma_1} h(\log n)$ over the prime powers $n \leq 2 \cdot 10^4$ plus a quadrature of the tail (for ℓ_{6000}, with the maximum over a grid of 96 phases in place of h for the primes $p \leq 300$), a convex function of the coefficients (a sum of Euclidean norms and of pointwise maxima of linear functions), and $\sup F$ by its maximum on a grid of 260 heights in $[1, 10^6]$ and its limit at $T = \infty$. The problem is a second-order cone programme, solved with `cvxpy` and Clarabel; the constraint (K) is imposed on a grid of 26×2200 points (b, t) with a margin $\eta/(1+t^2)$, $\eta = 2 \cdot 10^{-4}$, and the points where the current solution violates it on a finer grid (201×24000) are added as cutting planes, until no violation remains. The optimisation was run for $d \in \{0.65, 0.75, 0.8, 0.85, 0.9, 0.95, 1, 1.2\}$, $m \in \{2, 3, 4, 5, 7, 10\}$ and $L^* \in \{5, 20, 60, 150, 400, 1000, 2500, 6000, 20000\}$; the six kernels of Theorem 2.1 are the best found for $L^* = 5, 20, 60, 150, 400, 6000$. Longer kernels did not lower the leading constant below 0.0838: for $L^* = 20000$ the optimum is again $A_1 = 0.08381$.

For Theorem 2.3 the objective is (6.1), linear in (c_ι, κ_k), so the problem is a linear programme (HiGHS through `scipy`), with the same grid, cutting planes and a margin $\eta = 10^{-3}$. The candidate discs were all (c, R, v) with

$$c \in \{1.03, 1.06, 1.1, \log 3, 1.15, 1.2, 1.25, 1.3, 1.4, 1.5, \log 5, 2, e, 3\},$$
$$R = c - 1 + x, \quad x \in \{0.05, 0.1, \dots, 0.5\}, \quad R \leq c - \tfrac{1}{2},$$

and $v \in \{0, \pm 0.2, \pm 0.35, \pm 0.5, \pm 0.65, \pm 0.8, \pm 1, \pm 1.3, \pm 1.6, \pm 2\}$; the optimum uses 85 of the 2660 candidates. The first certification of the LP solution found a true violation of (6.2), of size $-8.9 \cdot 10^{-5}$ at $(b, t) \approx (0.467, -0.450)$, between two cutting planes; it was repaired by adding $2 \cdot 10^{-4}$ to $c_{\mathrm{R},1}$, which adds $2 \cdot 10^{-4} G_{\mathrm{R},1} > 0$ to U and $3.2 \cdot 10^{-5}$ to C_{H}. The repaired kernel is the one of Table 3. With the Bourgain profile (the piecewise linear function through the points $(\lambda_j - \kappa_j, \kappa_j)$, $(\kappa_j, \lambda_j) = A^j(\frac{13}{84}, \frac{55}{84})$, $0 \leq j \leq 7$, and $(1, 0)$, where A is the A-process) in place of $\mathfrak{e}_{\mathrm{P}}$ the same construction gives, on the grid, 0.07531; with $\mathfrak{e}_{\mathrm{P}}$ it gives 0.07477 before the margin and the repair; with $m = 20$ the grid value is 0.0740 (cutting planes not converged); none of these is certified. The grid values quoted in Remark 2.5 are from the same programs.

7.3. **Certification.** All certificates use `python-flint` 0.9.0 (Arb [17], FLINT [11]) at 256 bits of precision; a ball computed by Arb contains the exact value of the expression evaluated, and the programs only ever use the endpoints of balls in the direction that makes the final statement weaker. The kernel inequality for the hybrid ($T_1 = 40$) took 716 seconds; for the kernel of ℓ_{400} the three parts of Proposition 4.11 needed 11 835 193, 9 801 985 and 4 194 302 boxes and 2716 seconds, and for that of ℓ_{6000}, 3243 seconds. The constant C_ζ for each kernel takes between 80 and 165 seconds (the sum over the 5 761 455 primes below 10^8 and their powers, and the tail), and the elementary part about 120 seconds per mode. The coefficients are passed to the certifiers as the exact binary numbers written by the optimiser (the JSON files of Appendix D, whose decimal strings round-trip), so there is no rounding between search and certificate other than the rational rounding of the disc data described before (6.1), which is itself part of the certified data.

7.4. **Comparison of the two methods.** Backlund's method bounds the variation of $\arg\zeta$ on a horizontal segment ending at $\frac{1}{2}+iT$ by the number of zeros of $\operatorname{Re}\zeta$ on it, which Jensen's formula controls through the growth of ζ on a disc centred at $c+iT$, $c>1$, that reaches $\frac{1}{2}-\delta$; the growth to the left of the critical line is supplied by the functional equation, and the factor $\frac{1}{\log 2}$ in (1.7) comes from the geometry of a disc that reaches $\frac{1}{2}$ from a centre at distance about 1. The method of Turing type bounds a sum over all the zeros, each weighted by $f(b_\rho,t_\rho)$, by its exact evaluation at points of $\operatorname{Re}s>1$, and needs no information inside the strip; its limit, in the computations of Remark 2.5, is near 0.079. The hybrid keeps the exact evaluation for most of the sum and uses Jensen's formula only where the kernel fails, on small discs that lie entirely in $\operatorname{Re}s\ge\frac{1}{2}$, so that only the growth of ζ to the right of the critical line, where it is smallest, is paid for. Any improvement of the bounds for $\mu(\sigma)$ on $(\frac{1}{2},1)$ lowers C_{H} through $J(\mathfrak{e}_{\mathrm{P}};c_k,R_k)$ without any other change in the proof. Making Theorem 2.3 explicit requires versions of (5.1) with explicit constants, for which the available bounds are those of Hiary, Patel and Yang [15] on the critical line, of Yang [32] on the lines $\sigma=1-m/(2^m-2)$, and of Bellotti [3] in the strip; with them the method gives bounds of the form $A_1\log T+B\log\log T+C$ with A_1 near 0.076, valid from a computable height, and this is left for a later article.

Data availability. The programs, the coefficient files and the outputs of all certificates are reproduced in Appendices B–D and are available at `https://github.com/vcasana1/PaperMath/tree/explicit-S-turing/numerics/explicit_S_turing`.

AI declaration. The ideas of this article, the constructions and the strategy of the proofs are the author's. Generative AI tools were used during the work and the preparation of the manuscript: Claude (Anthropic), models Claude Fable 5.1, Claude Opus 5.5 and Claude Sonnet 5.5; ChatGPT (OpenAI), model Astra 6; and Microsoft 365 Copilot, based on the GPT-5 reasoning model. Under the author's direction, Claude and ChatGPT were used to draft and revise the LaTeX text, to check proofs, constants and citations against the sources, and to search the literature. The Python programs of Appendix B were written with Claude. Microsoft 365 Copilot was used only to review drafts of the manuscript. The author has checked the whole article and takes full responsibility for its content. No AI tool is an author.

Mathematical software. The computations of Appendices B–C use Python 3.13 with `numpy`, `scipy`, `cvxpy` (solver Clarabel), `mpmath` and `python-flint` 0.9.0 (Arb ball arithmetic [17], FLINT 3.6.0 [11]).

References

[1] V. S. G. Amberger, Estimating the number of zeros of Dedekind zeta-functions, arXiv:2510.27444v5, 12 October 2026 (v1: 31 October 2025), cited with the numbering of v5, doi:10.48550/arXiv.2510.27444. (Cited on pp. 3, 4, 5, 9, and 15.)

[2] R. J. Backlund, Über die Nullstellen der Riemannschen Zetafunktion, *Acta Math.* **41** (1916) 345–375, doi:10.1007/BF02422950. (Cited on pp. 2 and 4.)

[3] C. Bellotti, Explicit bounds for the Riemann zeta function and a new zero-free region, *J. Math. Anal. Appl.* **536** (2024) 128249, doi:10.1016/j.jmaa.2024.128249. (Cited on p. 20.)

[4] C. Bellotti and P.-J. Wong, Improved estimates for the argument and zero-counting function of the Riemann zeta-function, *Math. Comp.* (2025, published online 14 November 2025), statements numbered as in arXiv:2412.15470v2 (doi:10.48550/arXiv.2412.15470), doi:10.1090/mcom/4133. (Cited on pp. 2 and 4.)

[5] C. Bellotti and P.-J. Wong, Counting zeros of Artin L-functions, arXiv:2606.31441v1, 30 June 2026, doi:10.48550/arXiv.2606.31441. (Cited on pp. 3, 4, 5, and 15.)

[6] J. Bourgain, Decoupling, exponential sums and the Riemann zeta function, *J. Amer. Math. Soc.* **30** (2017) 205–224, theorems numbered as in arXiv:1408.5794v2 (doi:10.48550/arXiv.1408.5794), doi:10.1090/jams/860. (Cited on pp. 16 and 24.)

[7] V. Casaña-Giner, Bounds for the argument of the Riemann zeta function and a prime-corrected Riemann–von Mangoldt approximation of its zeros, manuscript, version 57 (31 October 2026), submitted; statements numbered as in that version. (Cited on pp. 2 and 5.)
[8] H. Davenport, *Multiplicative Number Theory*, 3rd edn., revised by H. L. Montgomery, Graduate Texts in Mathematics, Vol. 74 (Springer, New York, 2000), doi:10.1007/978-1-4757-5927-3. (Cited on pp. 2, 6, and 7.)
[9] NIST Digital Library of Mathematical Functions, §5.11(ii), Release 1.2.4 of 2025-03-15, F. W. J. Olver et al., eds., `https://dlmf.nist.gov/5.11.ii`. (Cited on p. 11.)
[10] A. Fiori, A note on the Phragmén–Lindelöf theorem, arXiv:2502.13282v3, 2026, doi:10.48550/arXiv.2502.13282. (Cited on p. 15.)
[11] The FLINT team, FLINT: Fast Library for Number Theory, version 3.6.0, `https://flintlib.org`. (Cited on pp. 19, 20, and 24.)
[12] J. Grossmann, Über die Nullstellen der Riemannschen Zeta-Funktion und der Dirichletschen L-Funktionen, Ph.D. thesis, Georg-August-Universität Göttingen (1913). (Cited on pp. 2 and 4.)
[13] E. Hasanalizade, Q. Shen and P.-J. Wong, Counting zeros of the Riemann zeta function, *J. Number Theory* **235** (2022) 219–241, doi:10.1016/j.jnt.2021.06.032. (Cited on pp. 2 and 4.)
[14] D. R. Heath-Brown, A new kth derivative estimate for exponential sums via Vinogradov's mean value, *Proc. Steklov Inst. Math.* **296** (2017) 88–103, theorems numbered as in arXiv:1601.04493v3 (doi:10.48550/arXiv.1601.04493), doi:10.1134/S0081543817010084. (Cited on pp. 16 and 24.)
[15] G. A. Hiary, D. Patel and A. Yang, An improved explicit estimate for $\zeta(1/2+it)$, *J. Number Theory* **256** (2024) 195–217, doi:10.1016/j.jnt.2023.09.003. (Cited on p. 20.)
[16] F. Johansson, Numerical integration in arbitrary-precision ball arithmetic, in: *Mathematical Software – ICMS 2018*, Lecture Notes in Computer Science, Vol. 10931 (Springer, Cham, 2018) 255–263, doi:10.1007/978-3-319-96418-8_30. (Cited on p. 11.)
[17] F. Johansson, Arb: efficient arbitrary-precision midpoint-radius interval arithmetic, *IEEE Trans. Comput.* **66** (2017) 1281–1292, doi:10.1109/TC.2017.2690633. (Cited on pp. 3, 19, 20, and 24.)
[18] F. W. J. Olver, *Asymptotics and Special Functions* (Academic Press, New York, 1974); reprinted (A K Peters, Wellesley, MA, 1997), doi:10.1201/9781439864548. (Cited on p. 11.)
[19] D. J. Platt and T. S. Trudgian, An improved explicit bound on $|\zeta(\frac{1}{2}+it)|$, *J. Number Theory* **147** (2015) 842–851, doi:10.1016/j.jnt.2014.08.019. (Cited on pp. 2 and 4.)
[20] H. Rademacher, On the Phragmén–Lindelöf theorem and some applications, *Math. Z.* **72** (1959) 192–204, doi:10.1007/BF01162949. (Cited on p. 15.)
[21] W. Rudin, *Real and Complex Analysis*, 3rd edn. (McGraw-Hill, New York, 1987). (Cited on p. 17.)
[22] J. B. Rosser, Explicit bounds for some functions of prime numbers, *Amer. J. Math.* **63** (1941) 211–232, doi:10.2307/2371291. (Cited on pp. 2 and 4.)
[23] J. B. Rosser and L. Schoenfeld, Approximate formulas for some functions of prime numbers, *Illinois J. Math.* **6** (1962) 64–94, doi:10.1215/ijm/1255631807. (Cited on pp. 3 and 10.)
[24] T. Tao, T. S. Trudgian and A. Yang, New exponent pairs, zero density estimates, and zero additive energy estimates: a systematic approach, arXiv:2501.16779v1, 28 January 2025, doi:10.48550/arXiv.2501.16779. (Cited on pp. 16 and 24.)
[25] E. C. Titchmarsh, *The Theory of Functions*, 2nd edn. (Oxford University Press, Oxford, 1939). (Cited on p. 17.)
[26] E. C. Titchmarsh, *The Theory of the Riemann Zeta-Function*, 2nd edn., revised by D. R. Heath-Brown (Clarendon Press, Oxford, 1986). (Cited on pp. 2, 6, 7, 15, 16, 17, and 24.)
[27] T. S. Trudgian, An improved upper bound for the argument of the Riemann zeta-function on the critical line, *Math. Comp.* **81** (2012) 1053–1061, doi:10.1090/S0025-5718-2011-02537-8. (Cited on pp. 2 and 4.)
[28] T. S. Trudgian, An improved upper bound for the argument of the Riemann zeta-function on the critical line II, *J. Number Theory* **134** (2014) 280–292, doi:10.1016/j.jnt.2013.07.017. (Cited on pp. 2 and 4.)
[29] T. S. Trudgian and A. Yang, Toward optimal exponent pairs, arXiv:2306.05599v3, 15 July 2024, doi:10.48550/arXiv.2306.05599. (Cited on pp. 16 and 24.)
[30] A. M. Turing, Some calculations of the Riemann zeta-function, *Proc. London Math. Soc. (3)* **3** (1953) 99–117, doi:10.1112/plms/s3-3.1.99. (Cited on pp. 3 and 5.)
[31] H. von Mangoldt, Zur Verteilung der Nullstellen der Riemannschen Funktion $\xi(t)$, *Math. Ann.* **60** (1905) 1–19, doi:10.1007/BF01447494. (Cited on pp. 2 and 4.)
[32] A. Yang, Explicit bounds on $\zeta(s)$ in the critical strip and a zero-free region, *J. Math. Anal. Appl.* **534** (2024) 128124, doi:10.1016/j.jmaa.2024.128124. (Cited on p. 20.)

Appendix A. Data tables

ι	c_ι	ι	c_ι
$(\mathrm{R},1)$	0.412251662846	$(\mathrm{I},2)$	0.733432854223
$(\mathrm{R},2)$	0.37014555218	$(\mathrm{I},3)$	−0.136213182184
$(\mathrm{R},3)$	0.284395447653	$(\mathrm{I},4)$	0.151451727439
$(\mathrm{R},4)$	0.158673925096	$(\mathrm{I},5)$	−0.117330434161
$(\mathrm{R},5)$	0.0696021832249	$(\mathrm{I},6)$	−0.0362095774481
$(\mathrm{R},6)$	−0.0614451777775	$(\mathrm{I},7)$	0.105985276636
$(\mathrm{R},7)$	−0.0411067019792	$(\mathrm{I},8)$	−0.0623450621431
$(\mathrm{R},8)$	0.0757521503477	$(\mathrm{I},9)$	0.0167253638541
$(\mathrm{R},9)$	−0.0325419425169	$(\mathrm{I},10)$	−0.00179758385541
$(\mathrm{R},10)$	0.00566316150651		

TABLE 3. Coefficients c_ι of the hybrid kernel ($d = \frac{9}{10}$, $m = 10$), to twelve significant digits; the exact binary values are in Appendix D.

Table 4: The 85 disc data (c_k, R_k, v_k) (rational, as used in the certificate), the weights κ_k (ten significant digits; exact binary values in Appendix D), $J_k = J(\mathfrak{e}_\mathrm{P}; c_k, R_k)$ and the contributions $\kappa_k J_k/\pi$ to C_H. The centres include $\log 3$ and $\log 5$ rounded to denominators $\le 10^4$.

k	c_k	R_k	v_k	κ_k	J_k	$\kappa_k J_k/\pi$
1	103/100	9/50	−1/2	0.04794121511	0.0069138770	0.000105507
2	103/100	9/50	1/2	0.0369050143	0.0069138770	$8.12189 \cdot 10^{-5}$
3	103/100	23/100	−13/20	0.04834170661	0.010786963	0.000165986
4	103/100	23/100	13/20	0.157399738	0.010786963	0.000540447
5	103/100	7/25	−13/20	0.02587833919	0.015048973	0.000123963
6	103/100	7/25	13/20	0.1061684833	0.015048973	0.000508572
7	103/100	33/100	−13/20	0.0236188805	0.019571517	0.000147141
8	103/100	33/100	13/20	0.1023900228	0.019571517	0.000637870
9	103/100	33/100	4/5	0.003545510823	0.019571517	$2.20878 \cdot 10^{-5}$
10	103/100	19/50	−13/20	0.0190191896	0.024737703	0.000149762
11	103/100	19/50	13/20	0.06465743477	0.024737703	0.000509129
12	103/100	43/100	1	0.003616521045	0.030311094	$3.48934 \cdot 10^{-5}$
13	103/100	12/25	−1	0.003351206341	0.036143828	$3.85554 \cdot 10^{-5}$
14	103/100	12/25	1	0.01526810593	0.036143828	0.000175659
15	103/100	53/100	−8/5	0.0001397445226	0.042238079	$1.87884 \cdot 10^{-6}$
16	103/100	53/100	−13/10	0.0002969055751	0.042238079	$3.99184 \cdot 10^{-6}$
17	103/100	53/100	−1	0.0009108669136	0.042238079	$1.22464 \cdot 10^{-5}$
18	103/100	53/100	1	0.004480174729	0.042238079	$6.02350 \cdot 10^{-5}$
19	103/100	53/100	13/10	0.001486648476	0.042238079	$1.99877 \cdot 10^{-5}$
20	53/50	4/25	−1/2	0.04885368436	0.0032593311	$5.06846 \cdot 10^{-5}$
21	53/50	4/25	1/2	0.161044244	0.0032593311	0.000167080
22	53/50	21/100	−1/2	0.02345342409	0.0063616476	$4.74926 \cdot 10^{-5}$
23	53/50	21/100	1/2	0.2275992324	0.0063616476	0.000460883
24	53/50	13/50	−1/2	0.08939597402	0.010091730	0.000287166
25	53/50	13/50	1/2	0.1519683713	0.010091730	0.000488168
26	53/50	41/100	−4/5	0.01373215771	0.023727069	0.000103713
27	53/50	41/100	−13/20	0.01864800559	0.023727069	0.000140840
28	53/50	41/100	13/20	0.0852639485	0.023727069	0.000643961
29	53/50	41/100	4/5	0.01741657093	0.023727069	0.000131540

k	c_k	R_k	v_k	κ_k	J_k	$\kappa_k J_k/\pi$
30	53/50	23/50	4/5	0.01374894904	0.029208025	0.000127827
31	53/50	14/25	−8/5	$4.075855253 \cdot 10^{-6}$	0.040975795	$5.31614 \cdot 10^{-8}$
32	53/50	14/25	−13/10	$4.409961547 \cdot 10^{-5}$	0.040975795	$5.75191 \cdot 10^{-7}$
33	53/50	14/25	13/10	0.0001588715206	0.040975795	$2.07216 \cdot 10^{-6}$
34	11/10	3/20	−7/20	0.0550560678	0.00078557774	$1.37672 \cdot 10^{-5}$
35	11/10	1/4	−1/2	0.06981663463	0.0057973918	0.000128837
36	11/10	1/4	1/2	0.1773566699	0.0057973918	0.000327288
37	11/10	2/5	4/5	0.07198100074	0.017623649	0.000403798
38	11/10	1/2	−4/5	0.0004564504973	0.027910450	$4.05518 \cdot 10^{-6}$
39	11/10	3/5	−1	0.000612420815	0.039458156	$7.69196 \cdot 10^{-6}$
40	11/10	3/5	1	0.002747211855	0.039458156	$3.45048 \cdot 10^{-5}$
41	11/10	3/5	13/10	0.000214610018	0.039458156	$2.69548 \cdot 10^{-6}$
42	5225/4756	229/1153	−1/2	0.0585032614	0.0029087233	$5.41667 \cdot 10^{-5}$
43	5225/4756	229/1153	1/2	0.2549235582	0.0029087233	0.000236027
44	5225/4756	2329/9368	1/2	0.08999758511	0.0058145056	0.000166569
45	5225/4756	1743/5837	−1/2	0.01018159396	0.0093687838	$3.03633 \cdot 10^{-5}$
46	5225/4756	1743/5837	1/2	0.190930449	0.0093687838	0.000569388
47	5225/4756	2667/5945	−4/5	0.002973806964	0.022593493	$2.13868 \cdot 10^{-5}$
48	5225/4756	2667/5945	4/5	0.01658999344	0.022593493	0.000119311
49	5225/4756	4132/8287	−4/5	0.002292772967	0.027952580	$2.04001 \cdot 10^{-5}$
50	5225/4756	4132/8287	4/5	0.0114306256	0.027952580	0.000101705
51	5225/4756	3222/5873	4/5	0.001994315479	0.033591760	$2.13244 \cdot 10^{-5}$
52	5225/4756	2847/4756	−13/10	$5.446961828 \cdot 10^{-5}$	0.039507983	$6.84998 \cdot 10^{-7}$
53	23/20	1/5	−1/2	0.08543526352	0.00067784859	$1.84340 \cdot 10^{-5}$
54	23/20	1/5	−7/20	0.04346495036	0.00067784859	$9.37825 \cdot 10^{-6}$
55	23/20	1/5	7/20	0.2263816956	0.00067784859	$4.88455 \cdot 10^{-5}$
56	23/20	9/20	−4/5	0.002794177723	0.016543682	$1.47142 \cdot 10^{-5}$
57	23/20	1/2	−4/5	0.003393622056	0.021311247	$2.30209 \cdot 10^{-5}$
58	23/20	11/20	−4/5	0.00693896347	0.026509943	$5.85536 \cdot 10^{-5}$
59	23/20	11/20	4/5	0.03251223251	0.026509943	0.000274350
60	23/20	13/20	−13/10	$5.267650803 \cdot 10^{-5}$	0.037780806	$6.33488 \cdot 10^{-7}$
61	23/20	13/20	−1	0.000433168236	0.037780806	$5.20928 \cdot 10^{-6}$
62	23/20	13/20	1	0.001776596527	0.037780806	$2.13654 \cdot 10^{-5}$
63	23/20	13/20	13/10	0.0002226772507	0.037780806	$2.67792 \cdot 10^{-6}$
64	6/5	1/4	1/2	0.334519418	0.00060498120	$6.44189 \cdot 10^{-5}$
65	6/5	7/10	−13/10	$4.572964533 \cdot 10^{-5}$	0.036302130	$5.28421 \cdot 10^{-7}$
66	6/5	7/10	−1	0.0003445239333	0.036302130	$3.98109 \cdot 10^{-6}$
67	6/5	7/10	1	0.001546601473	0.036302130	$1.78715 \cdot 10^{-5}$
68	6/5	7/10	13/10	0.0001463592826	0.036302130	$1.69123 \cdot 10^{-6}$
69	5/4	3/10	−7/20	0.08907592125	0.00055148537	$1.56367 \cdot 10^{-5}$
70	5/4	3/10	7/20	0.3795839671	0.00055148537	$6.66334 \cdot 10^{-5}$
71	5/4	3/4	−13/10	$3.644530876 \cdot 10^{-5}$	0.034985511	$4.05863 \cdot 10^{-7}$
72	5/4	3/4	−1	0.0003355290098	0.034985511	$3.73653 \cdot 10^{-6}$
73	5/4	3/4	1	0.001426333987	0.034985511	$1.58840 \cdot 10^{-5}$
74	5/4	3/4	13/10	0.0002927499897	0.034985511	$3.26013 \cdot 10^{-6}$
75	13/10	4/5	−13/10	$4.556491809 \cdot 10^{-5}$	0.033803191	$4.90274 \cdot 10^{-7}$
76	13/10	4/5	−1	0.0008241713773	0.033803191	$8.86799 \cdot 10^{-6}$
77	13/10	4/5	1	0.003939227432	0.033803191	$4.23857 \cdot 10^{-5}$
78	7/5	9/20	−7/20	0.3713075056	0.00044923233	$5.30952 \cdot 10^{-5}$
79	7/5	9/20	7/20	0.6314222324	0.00044923233	$9.02903 \cdot 10^{-5}$
80	7/5	9/10	−13/10	0.0001234809797	0.031760037	$1.24834 \cdot 10^{-6}$
81	7/5	9/10	13/10	0.001441638426	0.031760037	$1.45743 \cdot 10^{-5}$

k	c_k	R_k	v_k	κ_k	J_k	$\kappa_k J_k/\pi$
82	3/2	11/20	−7/20	0.09582582545	0.00040600316	$1.23840 \cdot 10^{-5}$
83	3/2	11/20	7/20	0.378111256	0.00040600316	$4.88651 \cdot 10^{-5}$
84	9993/6209	2980/4519	−7/20	0.1767418509	0.00037055405	$2.08469 \cdot 10^{-5}$
85	9993/6209	2980/4519	7/20	0.4838434428	0.00037055405	$5.70698 \cdot 10^{-5}$
					$\frac{1}{\pi}\sum_k \kappa_k J_k =$	0.009276500030

Table 5: The vertices (ς_i, e_i) of the profile $\mathfrak{e}_{\mathrm{P}}$ of Lemma 5.3, and the source of the bound $\mu(\varsigma_i) \le e_i$. The Heath-Brown vertices $n = 10, \dots, 398$ are omitted from the table; they are generated by `profile_published.py` (Appendix B).

i	ς_i	e_i	source
0	1/2	13/84	Bourgain [6, Thm. 5]
1	88225/153852	4742/38463	Trudgian–Yang [29, Thm. 2.4]
2	521/796	18/199	Trudgian–Yang [29, Thm. 2.4]
3	53141/76066	2779/38033	Trudgian–Yang [29, Thm. 2.4]
4	454/641	89/1282	Tao–Trudgian–Yang [24, Thm. 20, Lemma 23]
5	3473692/4856993	652397/9713986	Tao–Trudgian–Yang [24, Thm. 20, Lemma 23]
6	52209/69128	2371/43205	Trudgian–Yang [29, Thm. 2.4]
7	1389/1736	9/217	Trudgian–Yang [29, Thm. 2.4]
8	587779/702192	10769/351096	Tao–Trudgian–Yang [24, Thm. 20, Lemma 23]
9	7441/8695	89/3478	Tao–Trudgian–Yang [24, Thm. 20, Lemma 23]
10	277/300	1/100	Heath-Brown [14, Thm. 2], $n = 6$
11	535/567	1/162	Heath-Brown [14, Thm. 2], $n = 7$
12	375/392	1/245	Heath-Brown [14, Thm. 2], $n = 8$
13	3059/3168	1/352	Heath-Brown [14, Thm. 2], $n = 9$
	⋮	(Heath-Brown nodes $n = 10, \dots, 398$) ⋮	
403	6336001297/6336120399	1/31760002	Heath-Brown [14, Thm. 2], $n = 399$
404	1	0	$\zeta(1 + it) \ll \log t$ [26, Thm. 3.5]

ι	c_ι	ι	c_ι
(R, 1)	1.26767379327	(I, 2)	−0.37840733351
(R, 2)	0.700929451456	(I, 3)	0.134730628931
(R, 3)	−0.494633048662	(I, 4)	−0.017083827788
(R, 4)	0.097515550525	(I, 5)	0.000100343368477
(R, 5)	−0.000589396293163		

Table 6. Coefficients of the kernel of the line ℓ_5 ($d = 1$, $m = 5$), to twelve significant digits.

Appendix B. Programs

The programs are written in Python 3 and use `python-flint` (Arb ball arithmetic [17], FLINT [11]) at 256 bits, `numpy`, `scipy` and `cvxpy` with the solver Clarabel. The certification programs are those of Sections B–B; the search programs that produced the coefficients are reproduced

ι	c_ι	ι	c_ι
$(\mathrm{R},1)$	0.679978474949	$(\mathrm{I},2)$	1.18435020971
$(\mathrm{R},2)$	0.804110639165	$(\mathrm{I},3)$	−0.576537489831
$(\mathrm{R},3)$	0.777427090742	$(\mathrm{I},4)$	0.00610293149501
$(\mathrm{R},4)$	−0.0084486134969	$(\mathrm{I},5)$	$-2.10769863064 \cdot 10^{-5}$
$(\mathrm{R},5)$	$2.95597215845 \cdot 10^{-5}$		

TABLE 7. Coefficients of the kernel of the line ℓ_{20} ($d = 1.2$, $m = 5$), to twelve significant digits.

ι	c_ι	ι	c_ι
$(\mathrm{R},1)$	0.595153203515	$(\mathrm{I},2)$	0.80794661471
$(\mathrm{R},2)$	0.591131249545	$(\mathrm{I},3)$	−0.34004800791
$(\mathrm{R},3)$	0.388921642826	$(\mathrm{I},4)$	0.00364916717989
$(\mathrm{R},4)$	−0.00432516763692	$(\mathrm{I},5)$	$-1.27504674131 \cdot 10^{-5}$
$(\mathrm{R},5)$	$1.53789024752 \cdot 10^{-5}$		

TABLE 8. Coefficients of the kernel of the line ℓ_{60} ($d = 1$, $m = 5$), to twelve significant digits.

ι	c_ι	ι	c_ι
$(\mathrm{R},1)$	0.587356111095	$(\mathrm{I},2)$	0.529298607056
$(\mathrm{R},2)$	0.495527358923	$(\mathrm{I},3)$	−0.269742540483
$(\mathrm{R},3)$	0.171910239516	$(\mathrm{I},4)$	0.0395067096637
$(\mathrm{R},4)$	−0.0451542915632	$(\mathrm{I},5)$	−0.000234590737695
$(\mathrm{R},5)$	0.000271746755402		

TABLE 9. Coefficients of the kernel of the line ℓ_{150} ($d = 0.85$, $m = 5$), to twelve significant digits.

ι	c_ι	ι	c_ι
$(\mathrm{R},1)$	0.527699716861	$(\mathrm{I},2)$	0.579501653809
$(\mathrm{R},2)$	0.448444622334	$(\mathrm{I},3)$	−0.243041549546
$(\mathrm{R},3)$	0.2073491544	$(\mathrm{I},4)$	0.0959846058781
$(\mathrm{R},4)$	−0.0517088136271	$(\mathrm{I},5)$	−0.0223162904392
$(\mathrm{R},5)$	0.0427743920458	$(\mathrm{I},6)$	0.00101667452746
$(\mathrm{R},6)$	−0.00216867871949	$(\mathrm{I},7)$	$-2.32679097042 \cdot 10^{-5}$
$(\mathrm{R},7)$	$5.22536634852 \cdot 10^{-5}$	$(\mathrm{I},8)$	$3.09645235738 \cdot 10^{-7}$
$(\mathrm{R},8)$	$-7.19471178778 \cdot 10^{-7}$	$(\mathrm{I},9)$	$-2.32748949201 \cdot 10^{-9}$
$(\mathrm{R},9)$	$5.54823288096 \cdot 10^{-9}$	$(\mathrm{I},10)$	$7.769753796 \cdot 10^{-12}$
$(\mathrm{R},10)$	$-1.89056202487 \cdot 10^{-11}$		

TABLE 10. Coefficients of the kernel of the line ℓ_{400} ($d = 0.85$, $m = 10$), to twelve significant digits.

in Section B for completeness, but no statement of this article depends on them. All files are in the repository cited in Section 7, in the folders `hybrid_asymptotic/` and `family/` to which the relative paths in the programs refer; the deposited archive is flat, so a path such as

ι	c_ι	ι	c_ι
$(\mathrm{R},1)$	0.526619965261	$(\mathrm{I},2)$	0.583424601581
$(\mathrm{R},2)$	0.44752559746	$(\mathrm{I},3)$	-0.23667212703
$(\mathrm{R},3)$	0.217633359331	$(\mathrm{I},4)$	0.0956784075046
$(\mathrm{R},4)$	-0.0663261021081	$(\mathrm{I},5)$	-0.0185896622954
$(\mathrm{R},5)$	0.0492247519465	$(\mathrm{I},6)$	0.000831384847612
$(\mathrm{R},6)$	-0.0024545979738	$(\mathrm{I},7)$	$-1.89384002059\cdot10^{-5}$
$(\mathrm{R},7)$	$5.84151825308\cdot10^{-5}$	$(\mathrm{I},8)$	$2.51756719451\cdot10^{-7}$
$(\mathrm{R},8)$	$-7.9615067149\cdot10^{-7}$	$(\mathrm{I},9)$	$-1.8927048877\cdot10^{-9}$
$(\mathrm{R},9)$	$6.08645994128\cdot10^{-9}$	$(\mathrm{I},10)$	$6.32254880932\cdot10^{-12}$
$(\mathrm{R},10)$	$-2.0583832928\cdot10^{-11}$		

TABLE 11. Coefficients of the kernel of the line ℓ_{6000} ($d=0.85$, $m=10$), to twelve significant digits.

`../hybrid_asymptotic/certify_hybrid.py` in `certify_line_pp.py` must be read as the file of the same name in the archive.

`certify_hybrid.py`: certification of the kernel inequality (with or without discs) and of C_{H}

```
# Certification (Arb ball arithmetic, python-flint) of the kernel inequality
#   f(b,t,d) <= sum_j c_j [F_j(d+b,t)+F_j(d-b,t)] + sum_k kappa_k W_k(b,t)    for 0<=b<=1/2, t != 0,
# with f(b,t,d)=2atan((b+d)/t)+2atan((d-b)/t)-atan((b+2d)/t)-atan((2d-b)/t),
# F_Rj=Re (x+it)^(-j), F_Ij=-Im (x+it)^(-j), W_k=log^+(R_k/sqrt((c_k-1/2-b)^2+(t+v_k)^2)),
# and of the value C = c_R1/(2pi) + sum_k kappa_k J_k/pi, J_k=(1/pi) int_0^pi e(c_k+R_k cos th) dth.
import sys, json, time
from fractions import Fraction as Fr
from flint import arb, acb, ctx
ctx.prec = 256
def A(q): return arb(q.numerator)/q.denominator if isinstance(q,Fr) else arb(q)
def box(lo,hi): return A(lo).union(A(hi))          # lo,hi Fractions (dyadic) -> ball containing [lo,hi]

sol=json.load(open(sys.argv[1])); T1=Fr(sys.argv[2]) if len(sys.argv)>2 else Fr(40)
d=Fr(sol['d']).limit_denominator(10**6); B=[tuple(x) for x in sol['B']]
c=[Fr(x) for x in sol['c']]                                  # exact binary values of the LP coefficients
discs=[(Fr(cc).limit_denominator(10**4),Fr(R).limit_denominator(10**4),Fr(v).limit_denominator(10**4)) for cc
     ,R,v in sol['discs']]
discs=[(cc,min(R,cc-Fr(1,2)),v) for cc,R,v in discs]   # rational rounding must keep c-R >= 1/2 (shrinking R
     only lowers W)
kap=[Fr(x) for x in sol['kappa']]
assert all(k>=0 for k in kap) and all(cc-R>=Fr(1,2) and cc>1 for cc,R,v in discs)
dA=A(d); half=A(Fr(1,2)); PI=arb.pi()
cA=[A(x) for x in c]; kA=[A(x) for x in kap]; dsA=[(A(cc),A(R).log(),A(v)) for cc,R,v in discs]

def kernel(b,t):
    U=arb(0)
    for x in (dA+b, dA-b):
        w=acb(1)/acb(x,t); p=acb(1)
        for i,(k,j) in enumerate(B):
            pass
        pw=[acb(1)]
        for j in range(1,max(jj for _,jj in B)+1): pw.append(pw[-1]*w)
        for i,(k,j) in enumerate(B):
            U+= cA[i]*(pw[j].real if k=='R' else -pw[j].imag)
    return U
def fbox(b,t,sgn):
    # valid on the closure of t>0 (sgn=+1) or t<0 (sgn=-1)
    return sgn*PI-2*(t/(dA+b)).atan()-2*(t/(dA-b)).atan()+(t/(2*dA+b)).atan()+(t/(2*dA-b)).atan()
def Wlow(b,t):
```

```
    s=arb(0)
    for (cc,lR,v),k in zip(dsA,kA):
        if k==0: continue
        val=lR-((cc-half-b)**2+(t+v)**2).log()/2
        lo=val.lower()
        if lo>0: s+=k*lo
    return s

JMAX=max(jj for _,jj in B)
def grad(b,t):
    # partial derivatives of U-f in b and t, evaluated on a box (balls b,t)
    gb=arb(0); gt=arb(0)
    for x,sg in ((dA+b,1),(dA-b,-1)):
        w=acb(1)/acb(x,t); pw=[acb(1)]
        for j in range(1,JMAX+2): pw.append(pw[-1]*w)
        for i,(k,j) in enumerate(B):
            dx=-j*pw[j+1]; dt=acb(0,-j)*pw[j+1]
            if k=='R': gb+=cA[i]*sg*dx.real; gt+=cA[i]*dt.real
            else: gb-=cA[i]*sg*dx.imag; gt-=cA[i]*dt.imag
    y1=dA+b; y2=dA-b; y3=2*dA+b; y4=2*dA-b; t2=t*t
    ft=-2*y1/(y1*y1+t2)-2*y2/(y2*y2+t2)+y3/(y3*y3+t2)+y4/(y4*y4+t2)
    fb=2*t/(y1*y1+t2)-2*t/(y2*y2+t2)-t/(y3*y3+t2)+t/(y4*y4+t2)
    return gb-fb, gt-ft
def Hcentered(b0,b1,t0,t1,sgn):
    bm=(b0+b1)/2; tm=(t0+t1)/2; b=box(b0,b1); t=box(t0,t1)
    H0=kernel(A(bm),A(tm))-fbox(A(bm),A(tm),sgn)
    gb,gt=grad(b,t)
    H=H0+gb*box(-(b1-b0)/2,(b1-b0)/2)+gt*box(-(t1-t0)/2,(t1-t0)/2)
    return H+Wlow(b,t)
def cert_main(blo,bhi,tlo,thi,sgn,maxdepth=60):
    stack=[(blo,bhi,tlo,thi,0)]; n=0; worst=None
    while stack:
        b0,b1,t0,t1,dep=stack.pop(); n+=1
        b=box(b0,b1); t=box(t0,t1)
        H=Hcentered(b0,b1,t0,t1,sgn)
        if H>0: continue
        if dep>=maxdepth: raise RuntimeError('fail at b=[%s,%s] t=[%s,%s] H=%s'%(float(b0),float(b1),float(t0
    ),float(t1),H))
        wb=(b1-b0)/Fr(1,20); tm=max(abs(t0),abs(t1)); wt=(t1-t0)/max(Fr(1,20),tm*Fr(3,10))
        if wb>=wt: m_=(b0+b1)/2; stack+= [(b0,m_,t0,t1,dep+1),(m_,b1,t0,t1,dep+1)]
        else: m_=(t0+t1)/2; stack+= [(b0,b1,t0,m_,dep+1),(b0,b1,m_,t1,dep+1)]
    return n

# tail |t|>=T1, u=1/t: t^2 (U - f) >= t^2 U - 4d^3 u - K|u|^3, with t^2 U = sum c_j [Phi_j(d+b,u)+Phi_j(d-b,u
    )],
# Phi_R1 = x/(1+x^2u^2), Phi_j = Re/-Im of u^(j-2) (xu+i)^(-j) for j>=2;  K = (1/5) sum |alpha_i| y_i^5 with
    y_i<=2d+1/2.
def tailval(b,u):
    V=arb(0)
    for x in (dA+b, dA-b):
        for i,(k,j) in enumerate(B):
            if j==1:
                assert k=='R'; V+=cA[i]*x/(1+x*x*u*u)
            else:
                z=acb(u)**(j-2)*(acb(1)/acb(x*u,1))**j
                V+=cA[i]*(z.real if k=='R' else -z.imag)
    return V
ymax=2*dA+half; K=(2*(dA+half)**5+2*dA**5+ymax**5+(2*dA)**5)/5   # sum |alpha_i| y_i^5 /5, y_i in {b+d,d-b,b
    +2d,2d-b}, |b|<=1/2 (crude upper bound)
def cert_tail(u0,u1,maxdepth=40):
    assert u1*(2*d+Fr(1,2))<=1
    stack=[(Fr(0),Fr(1,2),u0,u1,0)]; n=0
    while stack:
        b0,b1,a0,a1,dep=stack.pop(); n+=1
        b=box(b0,b1); u=box(a0,a1)
```

```
        H=tailval(b,u)-4*dA**3*u-K*abs(u)**3
        if H>0: continue
        if dep>=maxdepth: raise RuntimeError('tail fail b=[%s,%s] u=[%s,%s] H=%s'%(float(b0),float(b1),float(
    a0),float(a1),H))
        if (b1-b0)*8>=(a1-a0)*T1: m_=(b0+b1)/2; stack+=[(b0,m_,a0,a1,dep+1),(m_,b1,a0,a1,dep+1)]
        else: m_=(a0+a1)/2; stack+=[(b0,b1,a0,m_,dep+1),(b0,b1,m_,a1,dep+1)]
    return n

def Jexact(nodes,cc,R):
    # (1/pi) int_0^pi e(cc+R cos th) dth for e piecewise linear through nodes, e=0 for sigma>=1, cc-R>=1/2
    tot=arb(0); ccA=A(cc); RA=A(R)
    lo,hi=cc-R,min(cc+R,Fr(1))
    for (p,ep),(q,eq) in zip(nodes,nodes[1:]):
        a,bq=max(p,lo),min(q,hi)
        if a>=bq: continue
        Bs=(eq-ep)/(q-p); As=ep-Bs*p
        tha=A((a-cc)/R).acos(); thb=A((bq-cc)/R).acos()      # tha>=thb
        tot+= (A(As)+A(Bs)*ccA)*(tha-thb)+A(Bs)*RA*(tha.sin()-thb.sin())
    return tot/PI
def nodes_of(p):
    k,l=Fr(13,84),Fr(55,84); nodes=[]
    if p=='T':   # ANTEDB Table 6.2 (Tao-Trudgian-Yang) + Heath-Brown nodes, see profile_tty.py
        from profile_tty import nodes_tty
        return nodes_tty()
    if p=='P':   # published-source nodes only (lower convex hull), see profile_published.py
        from profile_published import nodes_published
        return nodes_published()
    if p=='B':
        for _ in range(8):
            nodes.append((l-k,k)); k,l=k/(2*k+2),Fr(1,2)+l/(2*k+2)
        nodes.append((Fr(1),Fr(0)))
    else:
        nodes=[(Fr(1,2),Fr(27,164))]+[(1-Fr(m,2**m-2),Fr(1,2**m-2)) for m in range(4,10)]+[(Fr(1),Fr(0))]
    return nodes

if __name__=='__main__':
    t_=time.time()
    nodes=nodes_of(sol['prof'])
    J=[Jexact(nodes,cc,R) for cc,R,v in discs]
    C=cA[0]/(2*PI)+sum((k*j for k,j in zip(kA,J)),arb(0))/PI
    print('d =',d,' m =',max(j for _,j in B),' profile',sol['prof'],' discs used:',len(discs))
    print('C = c_R1/(2pi) + sum kappa_k J_k/pi =',C.str(20,radius=False),' (ball',C.str(5),')')
    print('kernel part c_R1/(2pi) =',(cA[0]/(2*PI)).str(12,radius=False))
    lim=2*dA*cA[B.index(('R',1))]-2*cA[B.index(('R',2))]; print('limit t^2 U at |t|->inf =',lim.str(10), lim
    >0)
    n1=cert_main(Fr(0),Fr(1,2),Fr(0),T1,+1); print('t in (0,T1]: certified, boxes',n1,flush=True)
    n2=cert_main(Fr(0),Fr(1,2),-T1,Fr(0),-1); print('t in [-T1,0): certified, boxes',n2,flush=True)
    assert all(abs(v)+R<=T1 for cc,R,v in discs)    # W_k = 0 for |t|>=T1
    n3=cert_tail(Fr(0),1/T1); n4=cert_tail(-1/T1,Fr(0)); print('|t|>=T1: certified, boxes',n3+n4)
    print('all certified;  C <',C.upper().str(12,radius=False),'  time %.0fs'%(time.time()-t_))
```

`profile_published.py`: the nodes of the profile $\mathfrak{e}_{\mathrm{P}}$

```
# Upper bound for mu(sigma) on [1/2,1] from published sources only, as a piecewise linear function through
    nodes
# (sigma_i, e_i) with mu(sigma_i) <= e_i a theorem in each case:
#  - 1/2: 13/84 (Bourgain 2017, Thm 5);
#  - breakpoints of Trudgian-Yang, Toward optimal exponent pairs, arXiv:2306.05599v3, Thm 2.4 (pieces are
    continuous);
#  - l-k for the exponent pairs (k,l) of Tao-Trudgian-Yang, arXiv:2501.16779v1, Thm 20, via Lemma 23 (mu(l-k)
     <= k);
#  - Heath-Brown 2017 (Proc. Steklov Inst. Math. 296), Thm 2: pairs (2/((n-1)^2(n+2)), 1-(3n-2)/(n(n-1)(n+2))
    +eps), n >= 3,
```

```
#    giving mu(sigma_n) <= mu_n, sigma_n = 1-(3n^2-3n+2)/(n(n-1)^2(n+2)), mu_n = 2/((n-1)^2(n+2)) (continuity
     of mu), 6 <= n <= 399;
#  - (1, 0) (zeta(1+it) << log t).
# The profile is the lower convex hull of these points (nodes above a chord of two others are dropped: the
    chord is a
# valid bound by the Phragmen-Lindelof interpolation lemma, and smaller).
from fractions import Fraction as Fr
TY = [(Fr(1,2),Fr(88225,153852),Fr(31,84),Fr(3,7)),(Fr(88225,153852),Fr(521,796),Fr(220633,620612),Fr
    (62831,155153)),
 (Fr(521,796),Fr(53141,76066),Fr(1333,3825),Fr(1508,3825)),(Fr(53141,76066),Fr(3620,5119),Fr(405,1202),Fr
    (227,601)),
 (Fr(3620,5119),Fr(52209,69128),Fr(49318855,170145110),Fr(52938216,170145110)),(Fr(52209,69128),Fr(1389,1736)
    ,Fr(471957,1682490),Fr(502648,1682490)),
 (Fr(1389,1736),Fr(134765,163248),Fr(2841,10316),Fr(3016,10316)),(Fr(134765,163248),Fr(18193,21906),Fr
    (859,3214),Fr(908,3214)),
 (Fr(18193,21906),Fr(249,280),Fr(5*8707,180277),Fr(5*9067,180277)),(Fr(249,280),Fr(277,300),Fr(29,130),Fr
    (30,130))]
PAIRS = [(Fr(89,1282),Fr(997,1282)),(Fr(652397,9713986),Fr(7599781,9713986)),(Fr(10769,351096),Fr
    (609317,702192)),(Fr(89,3478),Fr(15327,17390))]
def hb(n): return 1-Fr(3*n*n-3*n+2, n*(n-1)**2*(n+2)), Fr(2, (n-1)**2*(n+2))
def raw_nodes(nmax=400):
    for (l1,h1,a1,b1),(l2,h2,a2,b2) in zip(TY,TY[1:]): assert a1-b1*h1 == a2-b2*h1
    nodes = {}
    for lo,hi,a,b in TY: nodes[lo] = a-b*lo; nodes[hi] = a-b*hi
    for k,l in PAIRS: nodes[1-k] = min(nodes.get(1-k, Fr(1)), k)
    for n in range(6, nmax): s,m = hb(n); nodes[s] = min(nodes.get(s, Fr(1)), m)
    nodes[Fr(1)] = Fr(0)
    return sorted(nodes.items())
def nodes_published(nmax=400):
    P = raw_nodes(nmax); hull = []
    def cross(o,a,b): return (a[0]-o[0])*(b[1]-o[1])-(a[1]-o[1])*(b[0]-o[0])
    for p in P:
        while len(hull) >= 2 and cross(hull[-2], hull[-1], p) <= 0: hull.pop()
        hull.append(p)
    return hull
if __name__ == '__main__':
    H = nodes_published()
    print(len(H), 'nodes; first ones:')
    for s,e in H[:14]: print('  sigma = %-20s  e = %-18s  (%.6f, %.6f)' % (s, e, s, e))
```

cz_gen_pp.py: certification of C_ζ (per-prime phase maximum)

```
# Rigorous upper bound for C_zeta(c) = sum_n Lambda(n) n^(-sigma1) hypot(A0(log n), B0(log n)), any kernel
    basis
#   A0(L) = -(1/pi) sum_{(R,j)} c L^(j-1)/(j-1)!,   B0(L) = (1/pi)[ (e^(-dL)-2)/L - sum_{(I,j)} c L^(j-1)/(j
    -1)! ]
# Exact Arb sum over all prime powers n <= N0; tail n > N0 by partial summation with psi(x) < 1.03883 x
# (Rosser-Schoenfeld 1962, Thm 12) and certified monotonicity of g(x) = hypot(A0,B0)(log x) x^(-sigma1) on [
    N0, oo).
import sys, json, time, numpy as np
from fractions import Fraction as Fr
from math import factorial
from flint import arb, acb, ctx
ctx.prec = 256
def A(q): return arb(q.numerator)/arb(q.denominator) if isinstance(q, Fr) else arb(q)
sol = json.load(open(sys.argv[1])); N0 = int(float(sys.argv[2]))
d = Fr(sol['d']).limit_denominator(10**6); B = [tuple(b) for b in sol['B']]; c = [A(Fr(x)) for x in sol['c']]
PI = arb.pi(); dA = A(d); s1 = A(Fr(1, 2)+d); a = s1-1
# polynomial coefficients (in L) of pi*A0 and of the polynomial part of pi*B0
DEG = max(j for _, j in B)-1
pa = [arb(0)]*(DEG+1); pb = [arb(0)]*(DEG+1)
for (k, j), cc in zip(B, c):
    if k == 'R': pa[j-1] -= cc/factorial(j-1)
    else: pb[j-1] -= cc/factorial(j-1)
```

```
def poly(p, L): return sum((p[i]*L**i for i in range(len(p))), arb(0))
def h(L):
    A0 = poly(pa, L)/PI; B0 = (((-dA*L).exp()-2)/L+poly(pb, L))/PI
    return (A0*A0+B0*B0).sqrt()
t0 = time.time()
s = np.ones(N0+1, np.bool_); s[:2] = False
for i in range(2, int(N0**0.5)+1):
    if s[i]: s[i*i::i] = False
P = np.nonzero(s)[0]; del s
tot = arb(0); psi = arb(0)
P0 = int(sys.argv[3]) if len(sys.argv) > 3 else 1000   # per-prime phase maximum (Amberger 2025, Lemma 2.7)
     for p <= P0
M = 4096
xs = [2*arb.pi()*i/M for i in range(M)]; hx = arb.pi()/M   # grid points and half-spacing
gain = arb(0)
for p in P.tolist():
    lp = arb(p).log(); n = p; k = 1; coef = []; hyp = arb(0)
    while n <= N0:
        L = k*lp; w = lp*(-s1*L).exp(); A0 = poly(pa, L)/PI; B0 = (((-dA*L).exp()-2)/L+poly(pb, L))/PI
        coef.append((k, w*A0, w*B0)); hyp += w*(A0*A0+B0*B0).sqrt(); psi += lp; n *= p; k += 1
    if p <= P0:
        # f_p(x) = sum_k a_k cos(kx) + b_k sin(kx); max_x f_p <= max_i f_p(x_i) + hx * sum_k k |(a_k,b_k)|
        Lip = sum((kk*(a*a+b*b).sqrt() for kk, a, b in coef), arb(0))
        best = None
        for x in xs:
            v = sum((a*(kk*x).cos()+b*(kk*x).sin() for kk, a, b in coef), arb(0))
            best = v.upper() if best is None or v.upper() > best else best
        mb = arb(best)+hx*Lip
        mb = mb if mb < hyp else hyp
        gain += hyp-mb; tot += mb
    else:
        tot += hyp
print('per-prime phase maximum for p <=', P0, ': gain', gain.str(8), flush=True)
print('primes', len(P), ' partial sum (n<=%d) =' % N0, tot.str(15), ' psi(N0) =', psi.str(15), ' %.0fs' % (
     time.time()-t0), flush=True)
L0 = arb(N0).log()
# monotonicity: d/dL log g = (A0 A0' + B0 B0')/h^2 - sigma1 < 0 on [L0, LB] (boxes), and for L >= LB by a
     crude ratio bound
def cpoly(p, Lb):   # centred form: p(m) + p'(Lb)(Lb - m)
    m = arb(Lb.mid()); dp = [p[i]*i for i in range(1, len(p))]
    return poly(p, m)+poly(dp, Lb)*(Lb-m)
def dlogg_upper(Lb):
    dpa = [pa[i]*i for i in range(1, len(pa))]; dpb = [pb[i]*i for i in range(1, len(pb))]
    e = (-dA*Lb).exp()
    A0 = cpoly(pa, Lb)/PI; B0 = ((e-2)/Lb+cpoly(pb, Lb))/PI
    A1_ = cpoly(dpa, Lb)/PI; B1_ = (-(e-2)/(Lb*Lb)-dA*e/Lb+cpoly(dpb, Lb))/PI
    return (A0*A1_+B0*B1_)/(A0*A0+B0*B0)-s1
# L >= LB: (log h)' <= (|A0'|+|B0'|)/h and h >= |P| for the one of A0, B0 of larger degree D.
# In u = 1/L in [0, 1/LB]: num/L^D and low/L^D are polynomials in u, evaluated on the u-ball.
# LB is increased until the crude bound works (needed when the top kernel coefficients are tiny).
DA = max([i for i in range(DEG+1) if pa[i] != 0], default=-1); DB = max([i for i in range(DEG+1) if pb[i] !=
     0], default=-1)
q = [abs(pa[i])+abs(pb[i]) for i in range(DEG+1)]
if DA >= DB: D, pp, extra = DA, pa, 0
else: D, pp, extra = DB, pb, 2
if DEG > D: raise SystemExit('degree bookkeeping')
for LB in [2000.0, 5000.0, 2e4, 1e5, 1e6, 1e7]:
    u = arb(0).union(1/arb(LB))
    num = u*sum((i*q[i]*u**(D-i) for i in range(1, DEG+1)), arb(0))+2*u**(D+2)+dA*u**(D+1)
    low = abs(pp[D])-sum((abs(pp[i])*u**(D-i) for i in range(D)), arb(0))-extra*u**(D+1)
    if low > 0:
        ratio = num/low
        if ratio < s1: break
assert low > 0 and ratio < s1, ('crude ratio fails', ratio)
print('g monotone on [%g, oo): crude bound %s < sigma1' % (LB, ratio.str(6)))
```

```
# On [L0, LB] no monotonicity is assumed: G(L) = sup_{L' >= L} g(L') is majorized by a non-increasing step
    function,
# the running maximum (from the right) of upper bounds of g on small boxes, started at g(LB) (g decreasing
    beyond LB).
def h_up(Lb):
    A0 = cpoly(pa, Lb)/PI; B0 = (((-dA*Lb).exp()-2)/Lb+cpoly(pb, Lb))/PI
    au = abs(A0).upper(); bu = abs(B0).upper()
    return (au*au+bu*bu).sqrt().upper()
edges = [float(L0.lower())-1e-9]
while edges[-1] < LB: edges.append(min(edges[-1]+max(0.003, 0.0002*edges[-1]), LB))
gup = []
for i in range(len(edges)-1):
    Lb = arb(edges[i]).union(arb(edges[i+1])); gup.append(h_up(Lb)*(-s1*arb(edges[i])).exp())
run = h(arb(LB))*(-s1*arb(LB)).exp(); G = [None]*len(gup)
for i in range(len(gup)-1, -1, -1):
    run = run if run > gup[i] else (gup[i] if gup[i] > run else run.union(gup[i]).upper()); G[i] = run.upper
    ()
# int_{N0}^{oo} G(x) dx = sum_i G_i (e^{L_{i+1}} - e^{L_i}) + int_{LB}^{oo} g(x) dx
Istep = sum((G[i]*(arb(edges[i+1]).exp()-arb(edges[i]).exp()) for i in range(len(G))), arb(0))
L1 = arb(800)
def fz(z, an):
    A0 = sum((acb(pa[i])*z**i for i in range(len(pa))), acb(0))/PI
    B0 = (((-acb(dA)*z).exp()-2)/z+sum((acb(pb[i])*z**i for i in range(len(pb))), acb(0)))/PI
    return (A0*A0+B0*B0).sqrt(analytic=an)*((1-acb(s1))*z).exp()
I1 = Istep+acb.integral(fz, acb(LB), acb(L1)).real
def inc(k, X): return (-a*X).exp()*sum((arb(factorial(k))/factorial(i)*X**i/a**(k-i+1) for i in range(k+1)),
    arb(0))
I2 = (sum(((abs(pa[i])+abs(pb[i]))*inc(i, L1) for i in range(DEG+1)), arb(0))+3*inc(0, L1))/PI
gN0 = G[0]
tail = gN0*(arb('1.03883')*N0-psi)+arb('1.03883')*(I1+I2)
Cz = tot+tail
print('tail <=', tail.str(10), ' (integral', I1.str(10), ', far tail', I2.str(5), ')')
print('C_zeta <', Cz.upper().str(12, radius=False), '   time %.0fs' % (time.time()-t0))
json.dump({'Czeta_upper': Cz.upper().str(15, radius=False)}, open(sys.argv[1].replace('.json', '_czpp.json'),
     'w'))
```

`elem_gen.py`: certification of $\sup F_u$, $\sup F_\ell$ (modes N and S)

```
# Rigorous sup over T >= 1 of F_u(T) = Elem_u(T) - A1 log T and F_l(T) = -Elem_l(T) - A1 log T, any kernel
    basis.
# Elem_u = (1/pi)(2 Ael(s1) - Ael(s2)) - main(T) + (1/pi) sum_i c_i Phi_i(D_j(s1)),  Elem_l: same with -(sum
    c_R Phi_R - sum c_I Phi_I),
# Ael(s) = arg s + arg(s-1) - (T/2) log pi + Im log Gamma(s/2),  D_1 = 1/s+1/(s-1)-(1/2)log pi+(1/2)psi(s/2),
# D_j = s^-j + (s-1)^-j + ((-1)^(j-1)/(j-1)!) 2^-j psi^(j-1)(s/2)  (j >= 2);  Phi_R = Re, Phi_I = -Im.
# [1, T0]: centred form on geometric intervals; T >= T0: Stirling with Olver's remainder |R_1(z)| <= sec^2(ph
    z/2)/(12|z|)
# (DLMF 5.11(ii)) and Cauchy estimates |R_1^(k)(z)| <= k! (2/(3|z|)) (2/|z|)^k on the circle |w-z| = |z|/2.
import sys, json
from fractions import Fraction as Fr
from math import factorial
from flint import arb, acb, ctx
ctx.prec = 256
def A(q): return arb(q.numerator)/arb(q.denominator) if isinstance(q, Fr) else arb(q)
sol = json.load(open(sys.argv[1])); A1 = A(Fr(sys.argv[2])); T0 = int(sys.argv[3]) if len(sys.argv) > 3 else
    10**4
MODE = sys.argv[4] if len(sys.argv) > 4 else 'N'   # 'N': |N - main|;  'S': |S(T)| = |N - theta/pi - 1|
d = Fr(sol['d']).limit_denominator(10**6); B = [tuple(b) for b in sol['B']]; c = [A(Fr(x)) for x in sol['c']]
J = max(j for _, j in B)
PI = arb.pi(); s1r = A(Fr(1, 2)+d); s2r = A(Fr(1, 2)+2*d); LOGPI = PI.log()
cR1 = c[B.index(('R', 1))]; C1 = cR1/(2*PI)
assert A1 >= C1
def psi(k, z): return z.polygamma(k)
def Dj(j, s, ps):   # ps[k] = psi^(k)(s/2)
    if j == 1: return 1/s+1/(s-1)-LOGPI/2+ps[0]/2
```

```
    return s**(-j)+(s-1)**(-j)+arb((-1)**(j-1))/factorial(j-1)/2**j*ps[j-1]
def dDj(j, s, ps):  # d/ds D_j
    if j == 1: return -s**-2-(s-1)**-2+ps[1]/4
    return -j*s**(-j-1)-j*(s-1)**(-j-1)+arb((-1)**(j-1))/factorial(j-1)/2**(j+1)*ps[j]
def combine(vals):
    R = sum((cc*vals[i].real for i, ((k, j), cc) in enumerate(zip(B, c)) if k == 'R'), arb(0))
    I = sum((cc*(-vals[i].imag) for i, ((k, j), cc) in enumerate(zip(B, c)) if k == 'I'), arb(0))
    return R, I
def F_at(T):
    s1 = acb(s1r, T); s2 = acb(s2r, T)
    Ael = lambda s: (s.log().imag+(s-1).log().imag)-T/2*LOGPI+(s/2).lgamma().imag
    base = (2*Ael(s1)-Ael(s2))/PI-T/(2*PI)*(T/(2*PI*arb(1).exp())).log()
    if MODE == 'S':   # replace main(T) by theta(T)/pi + 1
        theta = acb(arb(1)/4, T/2).lgamma().imag-T/2*LOGPI
        base = base+T/(2*PI)*(T/(2*PI*arb(1).exp())).log()-theta/PI-1
    ps = [psi(k, s1/2) for k in range(J)]
    R, I = combine([Dj(j, s1, ps) for (_, j) in B])
    return base+(R+I)/PI-A1*T.log(), -(base-(R-I)/PI)-A1*T.log()
def dF_ball(Tb):
    Tm = arb(Tb.mid()); rad = Tb-Tm
    def P(k, sr):   # psi^(k) over the ball, mean value form
        zm = acb(sr, Tm)/2; zb = acb(sr, Tb)/2
        return psi(k, zm)+psi(k+1, zb)*acb(0, rad/2)
    s1 = acb(s1r, Tb); s2 = acb(s2r, Tb)
    dA = lambda s, sr: ((1/s).real+(1/(s-1)).real)-LOGPI/2+P(0, sr).real/2
    dbase = (2*dA(s1, s1r)-dA(s2, s2r))/PI-(Tb/(2*PI)).log()/(2*PI)
    if MODE == 'S':   # d/dT [main - theta/pi] = (1/2pi) log(T/2pi) - (1/(2pi)) Re psi(1/4+iT/2) + (1/(2pi))
     log pi
        zm = acb(arb(1)/4, Tm/2); zb = acb(arb(1)/4, Tb/2)
        P0 = psi(0, zm)+psi(1, zb)*acb(0, rad/2)
        dbase = dbase+(Tb/(2*PI)).log()/(2*PI)-(P0.real-LOGPI)/(2*PI)
    ps = [P(k, s1r) for k in range(J+1)]
    R, I = combine([acb(0, 1)*dDj(j, s1, ps) for (_, j) in B])
    return dbase+(R+I)/PI-A1/Tb, -(dbase-(R-I)/PI)-A1/Tb
ratio = Fr(1001, 1000); a = Fr(1); n = 0
while a < T0:
    b = min(a*ratio, Fr(T0)); m = (a+b)/2; dl = (b-a)/2
    fu, fl = F_at(A(m)); gu, gl = dF_ball(A(a).union(A(b)))
    eu = fu+gu*A(dl)*arb(0, 1); el = fl+gl*A(dl)*arb(0, 1)
    supu = eu if n == 0 else supu.union(eu); supl = el if n == 0 else supl.union(el)
    a = b; n += 1
print('intervals', n, ' sup F_u on [1,T0] <=', supu.upper().str(10, radius=False), '  sup F_l <=', supl.upper
    ().str(10, radius=False))
# T >= T0: F_u <= 7/8 - C1 log(2 pi) + K/T, F_l <= -7/8 - C1 log(2 pi) + K/T  (A1 >= C1)
T = A(Fr(T0)); s1 = s1r; s2 = s2r
K = (2*s2-1)/PI+(s1*s1/2+(s2-1)*s2/2+1)/PI+(2*s2*s2/4+(s1-1)*s1)/PI      # arg and Gamma parts (both signs)
K += abs(cR1)/PI*((2*s1-1)+s1*s1/4+arb(8)/3)/T                          # j = 1 corrections (O(1/T^2))
for (k, j), cc in zip(B, c):
    if j == 1: continue
    # |D_j| <= 2/T^j + (1/((j-1)! 2^j)) |psi^(j-1)(z)|,  |psi^(k)(z)| <= (k-1)!(2/T)^k + k!(2/T)^(k+1)/2 + (k
    +1)!(4/(3T))(4/T)^(k+1)
    kk = j-1
    ps = arb(factorial(kk-1))*(2/T)**kk+arb(factorial(kk))*(2/T)**(kk+1)/2+arb(factorial(kk+1))*(4/(3*T))*(4/
    T)**(kk+1)
    Dbound = 2/T**j+ps/(factorial(j-1)*2**j)
    K += abs(cc)/PI*Dbound*T      # Dbound <= const/T, so Dbound*T is the coefficient of 1/T
Finf_u = arb(7)/8-C1*(2*PI).log(); Finf_l = -arb(7)/8-C1*(2*PI).log()
if MODE == 'S':   # theta/pi + 1 - main = 7/8 + varpi/pi, |varpi(T)| <= (1/16+1/8+1/3)/T (Stirling, Olver's
    bound)
    Finf_u = -C1*(2*PI).log(); Finf_l = -C1*(2*PI).log(); K = K+(arb(1)/16+arb(1)/8+arb(1)/3)/PI
tu = Finf_u+K/T; tl = Finf_l+K/T
S = max(supu.upper(), supl.upper(), tu.upper(), tl.upper())
print('T>=T0: F_u <=', tu.upper().str(10, radius=False), ' F_l <=', tl.upper().str(10, radius=False), ' (K=',
     K.str(6), ')')
print('SUP over T>=1 of max(F_u,F_l) <=', S.str(12, radius=False))
```

```
json.dump({'A1': sys.argv[2], 'mode': MODE, 'Fsup_upper': S.str(15, radius=False)}, open(sys.argv[1].replace
    ('.json', '_elem%s.json' % ('' if MODE == 'N' else 'S')), 'w'))
```

certify_line_pp.py: driver for one explicit line

```
# Certify one line with the per-prime phase maximum in C_zeta: kernel inequality, C_zeta (cz_gen_pp),
    elementary part (N and S).
import json, sys, os, subprocess, math, glob
tag=sys.argv[1]; src=sys.argv[2]; pat=sys.argv[3] if len(sys.argv)>3 else 'sweep*.jsonl'
if src.endswith('.json'): r=json.load(open(src))
else:
    L=float(src); rs=[json.loads(l) for f in glob.glob(pat) for l in open(f)]
    r=min((x for x in rs if x['Lstar']==L), key=lambda x:x['val'])
D='line_'+tag; os.makedirs(D,exist_ok=True); sol=D+'/sol.json'; json.dump(r,open(sol,'w'))
def run(cmd,log):
    with open(D+'/'+log,'w') as fh: p=subprocess.run(cmd,stdout=fh,stderr=subprocess.STDOUT)
    out=open(D+'/'+log).read(); lines=out.strip().splitlines(); print(lines[-1] if lines else '(no output)',
    flush=True); return p.returncode,out
from decimal import Decimal, ROUND_CEILING
A1s=str(Decimal(repr(r['C1'])).quantize(Decimal('0.00001'),rounding=ROUND_CEILING)); A1=float(A1s)
print('candidate', tag, 'd=',r['d'], 'C1=',r['C1'], 'A3_est=',r['A3_est'], flush=True)
rc1,o1=run(['python3','../hybrid_asymptotic/certify_hybrid.py',sol,'100'],'cert_kernel.txt')
if not (rc1==0 and 'all certified' in o1): rc1,o1=run(['python3','../hybrid_asymptotic/certify_hybrid.py',sol
    ,'2000'],'cert_kernel.txt')
rc2,o2=run(['python3','cz_gen_pp.py',sol,'1e8','1000'],'cert_czeta_pp.txt')
rc3,o3=run(['python3','elem_gen.py',sol,A1s,'10000'],'cert_elem.txt')
rc4,o4=run(['python3','elem_gen.py',sol,A1s,'10000','S'],'cert_elem_S.txt')
def finish(D, sol, r, A1s, tag):
    # The certified upper endpoints are decimal strings; keep them as decimals and round up to three decimals
    .
    from decimal import Decimal, ROUND_CEILING, localcontext
    with localcontext() as dc:
        dc.prec = 60
        cz=Decimal(json.load(open(sol.replace('.json','_czpp.json')))['Czeta_upper'])
        fN=Decimal(json.load(open(sol.replace('.json','_elem.json')))['Fsup_upper'])
        fS=Decimal(json.load(open(sol.replace('.json','_elemS.json')))['Fsup_upper'])
        AN=(cz+fN).quantize(Decimal('0.001'),rounding=ROUND_CEILING); AS=(cz+fS).quantize(Decimal('0.001'),
    rounding=ROUND_CEILING)
    json.dump({'A1':A1s,'A3_N':str(AN),'A3_S':str(AS),'Czeta_pp_upper':str(cz),'Fsup_N':str(fN),'Fsup_S':str(
    fS),'d':r['d'],'B':r['B'],'Lstar':r['Lstar']},open(D+'/summary_pp.json','w'),indent=1)
    print('LINE %s CERTIFIED: |N(T)-main| <= %s log T + %s ;  |S(T)| <= %s log T + %s  (T>=1)'%(tag,A1s,AN,
    A1s,AS),flush=True)
if rc1==0 and 'all certified' in o1 and rc2==0 and rc3==0 and rc4==0: finish(D, sol, r, A1s, tag)
else: print('LINE %s FAILED'%tag,flush=True)
```

Search programs

lpm.py

```
# LP with margin: f <= sum c_j G_j + sum kappa_k W_k - eta/(1+t^2) on (b,t) in [0,1/2] x (R\{0}), cutting
    planes.
import numpy as np, sys, json
from fractions import Fraction as Fr
from scipy.optimize import linprog
def f(b,t,d): return 2*np.arctan((b+d)/t)+2*np.arctan((d-b)/t)-np.arctan((b+2*d)/t)-np.arctan((2*d-b)/t)
def F(x,t,k,j):
    z=(x+1j*t)**(-j); return z.real if k=='R' else -z.imag
def bourgain_nodes():
    k,l=Fr(13,84),Fr(55,84); nodes=[]
    for _ in range(8):
        nodes.append((l-k,k)); k,l=k/(2*k+2),Fr(1,2)+l/(2*k+2)
    nodes.append((Fr(1),Fr(0))); return nodes
def explicit_nodes():
    return [(Fr(1,2),Fr(27,164))]+[(1-Fr(m,2**m-2),Fr(1,2**m-2)) for m in range(4,10)]+[(Fr(1),Fr(0))]
```

```
def J_float(nodes,c,R,n=200001):
    xs=np.array([float(a) for a,_ in nodes]); es=np.array([float(e) for _,e in nodes])
    th=np.linspace(0,np.pi,n); s=c+R*np.cos(th); v=np.where(s>=1,0,np.interp(s,xs,es))
    return np.trapezoid(v,th)/np.pi
def rows(d,B,discs,bb,tt,eta):
    G=np.array([F(d+bb,tt,k,j)+F(d-bb,tt,k,j) for (k,j) in B]).T
    W=np.array([np.maximum(0,np.log(R/np.sqrt((c-0.5-bb)**2+(tt+v)**2))) for (c,R,v) in discs]).T
    return np.hstack([G,W]), f(bb,tt,d)+eta/(1+tt**2)
def tgrid(tmax):
    p=np.unique(np.concatenate([[1e-9],np.geomspace(1e-4,tmax,700),np.linspace(0,4,400)[1:]])); return np.
     concatenate([-p[::-1],p])
def solve(d,m,discs,nodes,eta,maxit=40,tol=0.0):
    B=[('R',j) for j in range(1,m+1)]+[('I',j) for j in range(2,m+1)]
    sc=np.array([(d-0.5)**j for (_,j) in B])
    J=[J_float(nodes,c,R) for (c,R,v) in discs]
    obj=np.concatenate([[1/(2*np.pi)],np.zeros(len(B)-1),np.array(J)/np.pi])
    bnds=[(None,None)]*len(B)+[(0,None)]*len(discs)
    bb,tt=[a.ravel() for a in np.meshgrid(np.linspace(0,0.5,26),tgrid(300.),indexing='ij')]
    A,Fv=rows(d,B,discs,bb,tt,eta)
    S=np.concatenate([sc,np.ones(len(discs))])
    bfine=np.linspace(0,0.5,201); tp=np.geomspace(1e-8,3000.,15000); tfine=np.concatenate([-tp[::-1],tp])
    # asymptotic constraint 2d c_R1 - 2 c_R2 >= eta
    arow=np.zeros(len(B)+len(discs)); arow[0]=2*d; arow[1]=-2
    for it in range(maxit):
        r=linprog(obj*S,A_ub=-np.vstack([A,arow])*S,b_ub=-np.concatenate([Fv,[eta]]),bounds=bnds,method='
     highs'); x=r.x*S
        nb=[];nt=[];worst=-1
        used=[k for k in range(len(discs)) if x[len(B)+k]>0]
        dU=[discs[k] for k in used]; xU=np.concatenate([x[:len(B)],x[len(B)+np.array(used,dtype=int)]]) if
     used else x[:len(B)]
        for b in bfine:
            Ar,Fr_=rows(d,B,dU,np.full_like(tfine,b),tfine,eta) if used else (np.array([F(d+b,tfine,k,j)+F(d-
    b,tfine,k,j) for (k,j) in B]).T, f(b,tfine,d)+eta/(1+tfine**2))
            v=Fr_-Ar@xU; worst=max(worst,v.max())
            idx=np.where((v>tol)&(v>=np.roll(v,1))&(v>=np.roll(v,-1)))[0]; nb+=[b]*len(idx); nt+=list(tfine[
    idx])
        if not nb: break
        A2,F2=rows(d,B,discs,np.array(nb),np.array(nt),eta); A=np.vstack([A,A2]); Fv=np.concatenate([Fv,F2])
    return obj@x, x, B, J, worst, it
if __name__=='__main__':
    m=int(sys.argv[1]); d=float(sys.argv[2]); prof=sys.argv[3]; eta=float(sys.argv[4])
    nodes=bourgain_nodes() if prof=='B' else explicit_nodes()
    discs=[(c,R,v) for c in [1.03,1.06,1.1,1.15,1.2,1.25,1.3,1.4] for R in
     [0.1,0.15,0.2,0.25,0.3,0.35,0.4,0.5,0.6] if c-R>=0.5 and c-R<1 for v in
     [-0.5,-0.3,-0.15,0,0.15,0.3,0.5]]
    val,x,B,J,w,it=solve(d,m,discs,nodes,eta)
    keep=[k for k in range(len(discs)) if x[len(B)+k]>1e-10]
    print('m=%d d=%.2f prof=%s eta=%g C=%.6f kernel=%.6f viol=%.2e it=%d ndiscs_used=%d'%(m,d,prof,eta,val,x
     [0]/(2*np.pi),w,it,len(keep)),flush=True)
    json.dump({'m':m,'d':d,'prof':prof,'eta':eta,'B':B,'c':list(x[:len(B)]),'discs':[discs[k] for k in keep
    ],'kappa':[x[len(B)+k] for k in keep],'J':[J[k] for k in keep],'C':val},open('sol_m%d_%s_eta%g.json'%(m,
     prof,eta),'w'))
```

explore_discs.py

```
# Exploration (not a certificate): kernel + Jensen discs, wide families of centres c > 1 (incl. log 3, log 5,
      2, e, 3, ...),
# sparse LP with cutting planes. Weight per zero: W(1/2+|b|) (+ W(1/2-|b|) if the disc crosses Re s = 1/2),
     cost 2J log T,
# i.e. objective c_R1/(2pi) + sum kappa_k J_k/pi as in certify_hybrid.py. Profile: Bourgain nodes, reflected
     for sigma<1/2.
import numpy as np, sys, json, time
from scipy.optimize import linprog
from scipy import sparse
```

```
from lpm import f, F, bourgain_nodes
import os
PROF = os.environ.get('PROF', 'B')
if PROF == 'T':
    from profile_tty import nodes_tty; nodes = nodes_tty()
elif PROF == 'E':   # explicit-capable nodes: Patel 27/164 at 1/2, Yang sigma_k (k=4..9), (1,0)  (lpm.
     explicit_nodes)
    from lpm import explicit_nodes; nodes = explicit_nodes()
elif PROF == 'C':   # convexity (1-sigma)/2 (explicit forms: Rademacher 1959)
    nodes = [(0.5, 0.25), (1.0, 0.0)]
elif PROF == 'W':   # Weyl exponent 1/6 at 1/2, linear to (1,0) (explicit: Hiary 2016 + explicit Phragmen-
     Lindelof)
    nodes = [(0.5, 1/6), (1.0, 0.0)]
else: nodes = bourgain_nodes()
XS = np.array([float(a) for a, _ in nodes]); ES = np.array([float(e) for _, e in nodes])
def prof(s):
    s = np.asarray(s, float); out = np.zeros_like(s)
    m = (s >= 0.5) & (s < 1); out[m] = np.interp(s[m], XS, ES)
    l = s < 0.5; out[l] = 0.5 - s[l] + np.interp(1 - s[l], XS, ES)
    return out
def Jf(c, R, n=40001):
    th = np.linspace(0, np.pi, n); return np.trapezoid(prof(c + R*np.cos(th)), th)/np.pi
def Wmat(discs, bb, tt):
    rows = []; cols = []; vals = []
    for k, (c, R, v) in enumerate(discs):
        for sgn in (1, -1):
            if sgn == -1 and c - R >= 0.5: continue
            dist = np.sqrt((c - 0.5 - sgn*bb)**2 + (tt + v)**2)
            w = np.log(R/np.maximum(dist, 1e-300)); i = np.nonzero(w > 0)[0]
            rows.append(i); cols.append(np.full(len(i), k)); vals.append(w[i])
    r = np.concatenate(rows) if rows else np.zeros(0, int)
    return sparse.csr_matrix((np.concatenate(vals) if vals else [], (r, np.concatenate(cols) if cols else [])
     ), shape=(len(bb), len(discs)))
def Gmat(d, B, bb, tt): return np.array([F(d+bb, tt, k, j)+F(d-bb, tt, k, j) for (k, j) in B]).T
def tgrid(tmax):
    p = np.unique(np.concatenate([[1e-9], np.geomspace(1e-4, tmax, 500), np.linspace(0, 4, 300)[1:]]));
     return np.concatenate([-p[::-1], p])
def solve(d, m, discs, eta=0.0, maxit=30, nb=26):
    B = [('R', j) for j in range(1, m+1)]+[('I', j) for j in range(2, m+1)]; nB = len(B)
    sc = np.array([(d-0.5)**j for (_, j) in B]); J = np.array([Jf(c, R) for c, R, v in discs])
    obj = np.concatenate([[sc[0]/(2*np.pi)], np.zeros(nB-1), J/np.pi])
    bb, tt = [a.ravel() for a in np.meshgrid(np.linspace(0, 0.5, nb), tgrid(300.), indexing='ij')]
    def block(bb, tt): return sparse.hstack([sparse.csr_matrix(Gmat(d, B, bb, tt)*sc), Wmat(discs, bb, tt)]).
     tocsr(), f(bb, tt, d)+eta/(1+tt**2)
    A, Fv = block(bb, tt)
    arow = np.zeros(nB+len(discs)); arow[0] = 2*d*sc[0]; arow[1] = -2*sc[1]
    bf = np.linspace(0, 0.5, 101); tp = np.geomspace(1e-7, 3000., 6000); tf = np.concatenate([-tp[::-1], tp])
    for it in range(maxit):
        r = linprog(obj, A_ub=-sparse.vstack([A, sparse.csr_matrix(arow)]), b_ub=-np.concatenate([Fv, [eta]])
     ,
                    bounds=[(None, None)]*nB+[(0, None)]*len(discs), method='highs')
        if r.status != 0: return None
        x = r.x; used = np.nonzero(x[nB:] > 1e-12)[0]; du = [discs[k] for k in used]
        nbl = []; ntl = []; worst = -1
        for b in bf:
            v = f(b, tf, d)+eta/(1+tf**2)-Gmat(d, B, np.full_like(tf, b), tf)@(x[:nB]*sc)
            if len(used): v = v-Wmat(du, np.full_like(tf, b), tf)@x[nB+used]
            worst = max(worst, v.max()); idx = np.nonzero((v > 0) & (v >= np.roll(v, 1)) & (v >= np.roll(v,
    -1)))[0]
            nbl += [b]*len(idx); ntl += list(tf[idx])
        if not nbl: break
        A2, F2 = block(np.array(nbl), np.array(ntl)); A = sparse.vstack([A, A2]).tocsr(); Fv = np.concatenate
     ([Fv, F2])
    xs = x.copy(); xs[:nB] *= sc
    return r.fun, xs, B, J, worst, it
```

```
def family(name):
    import math
    if name == 'old':   # the m=10 certified family
        return [(c, R, v) for c in [1.03, 1.06, 1.1, 1.15, 1.2, 1.25, 1.3, 1.4] for R in [0.1, 0.15, 0.2,
     0.25, 0.3, 0.35, 0.4, 0.5, 0.6] if c-R >= 0.5 and c-R < 1 for v in [-0.5, -0.3, -0.15, 0, 0.15, 0.3,
     0.5]]
    if name == 'vwide':   # centres incl. log 3, log 5, 2, e, 3; vertical offsets up to |v| = 2
        C = [1.03, 1.06, 1.1, math.log(3), 1.15, 1.2, 1.25, 1.3, 1.4, 1.5, math.log(5), 2.0, math.e, 3.0]
        out = []
        for c in C:
            for R in [c-1+x for x in [0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5]]:
                if R <= 0 or R > c-0.5+1e-12: continue
                for v in [-2, -1.6, -1.3, -1.0, -0.8, -0.65, -0.5, -0.35, -0.2, 0, 0.2, 0.35, 0.5, 0.65, 0.8,
      1.0, 1.3, 1.6, 2]:
                    if abs(v) <= R+2.5: out.append((c, R, float(v)))
        return out
    C = [1.01, 1.02, 1.04, 1.06, 1.08, 1.1, 1.13, 1.16, 1.2, 1.25, 1.3, 1.35, 1.4, 1.5, math.log(3), math.log
     (5), 1.75, 2.0, math.e/1.25, 2.5, math.e, 3.0, math.pi, 4.0, 5.0]
    out = []
    for c in sorted(set(C)):
        rmax = c-0.5 if name != 'cross' else c-0.35
        Rs = sorted(set([c-1+x for x in [0.02, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5]] + ([c
     -0.4, c-0.45] if name == 'cross' else [])))
        for R in Rs:
            if R <= 0 or R > rmax+1e-12: continue
            h = R if R < 1 else min(R, 2.0)
            for v in np.linspace(-0.8*h, 0.8*h, 9 if R < 0.5 else 7): out.append((c, R, float(v)))
    return out
if __name__ == '__main__':
    m = int(sys.argv[1]); d = float(sys.argv[2]); fam = sys.argv[3]; eta = float(sys.argv[4]) if len(sys.argv
     ) > 4 else 0.0
    discs = family(fam); t0 = time.time()
    res = solve(d, m, discs, eta)
    val, x, B, J, w, it = res; nB = len(B); keep = [k for k in range(len(discs)) if x[nB+k] > 1e-10]
    used_c = sorted(set(round(discs[k][0], 4) for k in keep))
    print('m=%d d=%.3f fam=%s eta=%g ndiscs=%d C=%.6f kernel=%.6f viol=%.2e it=%d used=%d centres=%s  %.0fs'
     % (m, d, fam, eta, len(discs), val, x[0]/(2*np.pi), w, it, len(keep), used_c, time.time()-t0), flush=
     True)
    json.dump({'m': m, 'd': d, 'prof': PROF, 'eta': eta, 'B': B, 'c': list(x[:nB]), 'discs': [discs[k] for k
     in keep], 'kappa': [x[nB+k] for k in keep], 'J': [J[k] for k in keep], 'C': val},
              open('expl_m%d_d%.2f_%s_%s_eta%g.json' % (m, d, fam, PROF, eta), 'w'))
```

opt_line.py

```
# Optimize one explicit line |N(T)-main(T)| <= A1 log T + A3 for a target height L* = log T*.
# Kernel U = sum c_i G_i (basis R1..R_mR, I2..I_mI), d fixed. Convex program (SOCP, Clarabel):
#   minimize  (c_R1/2pi) L* + C_zeta(c) + z
#   s.t.      f(b,t) + eta/(1+t^2) <= U(b,t)      on a (b,t) grid, refined by cutting planes
#             2d c_R1 - 2 c_R2 >= eta              (behaviour at |t| -> oo)
#             z >= F_u(T;c), z >= F_l(T;c)         on a T grid in [1,1e6], and z >= limits at T = oo
# C_zeta(c) = sum_n Lambda(n) n^{-sigma1} || (A0(log n), B0(log n)) ||_2  (n <= n0) + quadrature of the tail.
# Output: JSON with exact binary coefficients, used by the certifiers.
import sys, json, math, time, numpy as np, cvxpy as cp, mpmath as mp
sys.path.insert(0, '../hybrid_asymptotic')
from lpm import f, F, tgrid

def basis(mR, mI): return [('R', j) for j in range(1, mR+1)] + [('I', j) for j in range(2, mI+1)]

def kernel_rows(d, B, bb, tt):
    G = np.array([F(d+bb, tt, k, j) + F(d-bb, tt, k, j) for (k, j) in B]).T
    return G, f(bb, tt, d)

def primes(N):
    s = np.ones(N+1, bool); s[:2] = False
```

```
    for i in range(2, int(N**0.5)+1):
        if s[i]: s[i*i::i] = False
    return np.nonzero(s)[0]

def czeta_data(d, B, n0):
    s1 = 0.5+d; rows_a = []; rows_b = []; beta0 = []; w = []
    for p in primes(n0):
        lp = math.log(p); n = p; k = 1
        while n <= n0:
            L = k*lp
            rows_a.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'R' else 0 for (kk, j) in B])
            rows_b.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'I' else 0 for (kk, j) in B])
            beta0.append((math.exp(-d*L)-2)/(L*math.pi)); w.append(lp*math.exp(-s1*L)); n *= p; k += 1
    # tail n > n0: psi(x) ~ x, sum Lambda(n) g(n) ~ int g(x) dx = int g e^L dL ; Gauss-Legendre in L on [log
     n0, 700]
    xg, wg = np.polynomial.legendre.leggauss(400); a, b = math.log(n0), 700.0
    for x, ww in zip(xg, wg):
        L = (b-a)/2*x+(a+b)/2; wt = (b-a)/2*ww*math.exp((1-s1)*L)
        rows_a.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'R' else 0 for (kk, j) in B])
        rows_b.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'I' else 0 for (kk, j) in B])
        beta0.append((math.exp(-d*L)-2)/(L*math.pi)); w.append(wt)
    return np.array(rows_a), np.array(rows_b), np.array(beta0), np.array(w)

def elem_data(d, B, Tgrid):
    mp.mp.dps = 25
    s1r = mp.mpf(1)/2+d; s2r = mp.mpf(1)/2+2*d
    base = []; Phi = []
    for T in Tgrid:
        T = mp.mpf(T); s1 = s1r+1j*T; s2 = s2r+1j*T
        Ael = lambda s: mp.im(mp.log(s)+mp.log(s-1))-T/2*mp.log(mp.pi)+mp.im(mp.loggamma(s/2))
        base.append(float((2*Ael(s1)-Ael(s2))/mp.pi-T/(2*mp.pi)*mp.log(T/(2*mp.pi*mp.e))))
        row = []
        for (k, j) in B:
            if j == 1: D = 1/s1+1/(s1-1)-mp.log(mp.pi)/2+mp.psi(0, s1/2)/2
            else: D = s1**(-j)+(s1-1)**(-j)+(-1)**(j-1)/mp.factorial(j-1)/2**j*mp.psi(j-1, s1/2)
            row.append(float(mp.re(D)) if k == 'R' else float(-mp.im(D)))
        Phi.append(row)
    return np.array(base), np.array(Phi)

def optimize(d, mR, mI, Lstar, eta=2e-4, n0=20000, maxit=25, verbose=True):
    B = basis(mR, mI); nB = len(B); i1 = B.index(('R', 1)); i2 = B.index(('R', 2))
    isR = np.array([k == 'R' for k, _ in B])
    bb, tt = [a.ravel() for a in np.meshgrid(np.linspace(0, 0.5, 26), tgrid(300.), indexing='ij')]
    G, Fv = kernel_rows(d, B, bb, tt); Fv = Fv+eta/(1+tt**2)
    RA, RB, b0, w = czeta_data(d, B, n0)
    Tg = np.unique(np.concatenate([np.geomspace(1, 1e6, 260), [1.0]]))
    base, Phi = elem_data(d, B, Tg); logT = np.log(Tg)
    sc = np.array([(d-0.5)**j for (_, j) in B])        # scaling for conditioning
    bfine = np.linspace(0, 0.5, 201); tp = np.geomspace(1e-8, 3000., 12000); tfine = np.concatenate([-tp
     [::-1], tp])
    for it in range(maxit):
        y = cp.Variable(nB); z = cp.Variable(); c = cp.multiply(sc, y)
        cons = [G@c >= Fv, 2*d*c[i1]-2*c[i2] >= eta]
        sign = np.where(isR, 1.0, -1.0)
        Fu = base+(Phi@c)/math.pi-c[i1]/(2*math.pi)*logT
        Fl = -base+(Phi@cp.multiply(sign, c))/math.pi-c[i1]/(2*math.pi)*logT
        cons += [z >= Fu, z >= Fl, z >= 7/8-c[i1]/(2*math.pi)*math.log(2*math.pi)]
        Cz = cp.sum(cp.multiply(w, cp.norm(cp.vstack([RA@c, RB@c+b0]), 2, axis=0)))
        obj = c[i1]/(2*math.pi)*Lstar+Cz+z
        prob = cp.Problem(cp.Minimize(obj), cons); prob.solve(solver=cp.CLARABEL)
        cv = sc*y.value
        nb = []; nt = []; worst = -1
        for b in bfine:
            Gr, Fr = kernel_rows(d, B, np.full_like(tfine, b), tfine); v = Fr+eta/(1+tfine**2)-Gr@cv; worst =
     max(worst, v.max())
```

```
            idx = np.where((v > 0) & (v >= np.roll(v, 1)) & (v >= np.roll(v, -1)))[0]; nb += [b]*len(idx); nt
     += list(tfine[idx])
        if verbose: print('  it %d obj=%.5f C1=%.5f Cz~%.4f z=%.4f worst=%.2e' % (it, prob.value, cv[i1]/(2*
    math.pi), Cz.value, z.value, worst), flush=True)
        if not nb: break
        G2, F2 = kernel_rows(d, B, np.array(nb), np.array(nt)); G = np.vstack([G, G2]); Fv = np.concatenate([
    Fv, F2+eta/(1+np.array(nt)**2)])
    return dict(d=d, B=B, c=list(map(float, cv)), C1=float(cv[i1]/(2*math.pi)), Cz_est=float(Cz.value), z_est
    =float(z.value),
                A3_est=float(Cz.value+z.value), Lstar=Lstar, eta=eta, prof='B', discs=[], kappa=[], J=[], m=
    max(j for _, j in B))

if __name__ == '__main__':
    d = float(sys.argv[1]); mR = int(sys.argv[2]); mI = int(sys.argv[3]); Ls = float(sys.argv[4]); out = sys.
    argv[5]
    t0 = time.time(); r = optimize(d, mR, mI, Ls)
    print('RESULT d=%.3f mR=%d mI=%d L*=%g  A1=%.5f A3~%.4f  value@L*~%.4f  (%.0fs)' % (d, mR, mI, Ls, r['C1
    '], r['A3_est'], r['C1']*Ls+r['A3_est'], time.time()-t0), flush=True)
    json.dump(r, open(out, 'w'))
```

opt_line_pp.py

```
# Optimize one explicit line |N(T)-main(T)| <= A1 log T + A3 for a target height L* = log T*.
# Kernel U = sum c_i G_i (basis R1..R_mR, I2..I_mI), d fixed. Convex program (SOCP, Clarabel):
#   minimize  (c_R1/2pi) L* + C_zeta(c) + z
#   s.t.      f(b,t) + eta/(1+t^2) <= U(b,t)      on a (b,t) grid, refined by cutting planes
#             2d c_R1 - 2 c_R2 >= eta              (behaviour at |t| -> oo)
#             z >= F_u(T;c), z >= F_l(T;c)         on a T grid in [1,1e6], and z >= limits at T = oo
# C_zeta(c) = sum_n Lambda(n) n^{-sigma1} || (A0(log n), B0(log n)) ||_2  (n <= n0) + quadrature of the tail.
# Output: JSON with exact binary coefficients, used by the certifiers.
import sys, json, math, time, os, numpy as np, cvxpy as cp, mpmath as mp
PPMAX = int(os.environ.get('PPMAX', '0'))
sys.path.insert(0, '../hybrid_asymptotic')
from lpm import f, F, tgrid

def basis(mR, mI): return [('R', j) for j in range(1, mR+1)] + [('I', j) for j in range(2, mI+1)]

def kernel_rows(d, B, bb, tt):
    G = np.array([F(d+bb, tt, k, j) + F(d-bb, tt, k, j) for (k, j) in B]).T
    return G, f(bb, tt, d)

def primes(N):
    s = np.ones(N+1, bool); s[:2] = False
    for i in range(2, int(N**0.5)+1):
        if s[i]: s[i*i::i] = False
    return np.nonzero(s)[0]

def czeta_data(d, B, n0):
    s1 = 0.5+d; rows_a = []; rows_b = []; beta0 = []; w = []
    for p in primes(n0):
        lp = math.log(p); n = p; k = 1
        while n <= n0:
            L = k*lp
            rows_a.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'R' else 0 for (kk, j) in B])
            rows_b.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'I' else 0 for (kk, j) in B])
            beta0.append((math.exp(-d*L)-2)/(L*math.pi)); w.append(lp*math.exp(-s1*L)); n *= p; k += 1
    # tail n > n0: psi(x) ~ x, sum Lambda(n) g(n) ~ int g(x) dx = int g e^L dL ; Gauss-Legendre in L on [log
    n0, 700]
    xg, wg = np.polynomial.legendre.leggauss(400); a, b = math.log(n0), 700.0
    for x, ww in zip(xg, wg):
        L = (b-a)/2*x+(a+b)/2; wt = (b-a)/2*ww*math.exp((1-s1)*L)
        rows_a.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'R' else 0 for (kk, j) in B])
        rows_b.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'I' else 0 for (kk, j) in B])
        beta0.append((math.exp(-d*L)-2)/(L*math.pi)); w.append(wt)
```

```
    return np.array(rows_a), np.array(rows_b), np.array(beta0), np.array(w)


def czeta_pp_data(d, B, PP=300, n0=20000):
    """for primes p <= PP: list of (kvec, wvec, RAk, RBk, b0k) to build max_x f_p(x) on a grid; other n via
     hypot rows."""
    s1 = 0.5+d; groups = []
    for p in primes(PP):
        lp = math.log(p); n = p; k = 1; ks = []; ra = []; rb = []; bz = []
        while n <= n0:
            L = k*lp; w = lp*math.exp(-s1*L)
            ra.append([-w*(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'R' else 0 for (kk, j) in B])
            rb.append([-w*(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'I' else 0 for (kk, j) in B])
            bz.append(w*(math.exp(-d*L)-2)/(L*math.pi)); ks.append(k); n *= p; k += 1
        groups.append((p, np.array(ks), np.array(ra), np.array(rb), np.array(bz)))
    return groups

def elem_data(d, B, Tgrid):
    mp.mp.dps = 25
    s1r = mp.mpf(1)/2+d; s2r = mp.mpf(1)/2+2*d
    base = []; Phi = []
    for T in Tgrid:
        T = mp.mpf(T); s1 = s1r+1j*T; s2 = s2r+1j*T
        Ael = lambda s: mp.im(mp.log(s)+mp.log(s-1))-T/2*mp.log(mp.pi)+mp.im(mp.loggamma(s/2))
        base.append(float((2*Ael(s1)-Ael(s2))/mp.pi-T/(2*mp.pi)*mp.log(T/(2*mp.pi*mp.e))))
        row = []
        for (k, j) in B:
            if j == 1: D = 1/s1+1/(s1-1)-mp.log(mp.pi)/2+mp.psi(0, s1/2)/2
            else: D = s1**(-j)+(s1-1)**(-j)+(-1)**(j-1)/mp.factorial(j-1)/2**j*mp.psi(j-1, s1/2)
            row.append(float(mp.re(D)) if k == 'R' else float(-mp.im(D)))
        Phi.append(row)
    return np.array(base), np.array(Phi)

def optimize(d, mR, mI, Lstar, eta=2e-4, n0=20000, maxit=25, verbose=True):
    B = basis(mR, mI); nB = len(B); i1 = B.index(('R', 1)); i2 = B.index(('R', 2))
    isR = np.array([k == 'R' for k, _ in B])
    bb, tt = [a.ravel() for a in np.meshgrid(np.linspace(0, 0.5, 26), tgrid(300.), indexing='ij')]
    G, Fv = kernel_rows(d, B, bb, tt); Fv = Fv+eta/(1+tt**2)
    RA, RB, b0, w = czeta_data(d, B, n0)
    if PPMAX:
        GR = czeta_pp_data(d, B, PPMAX, n0)
        # rows of czeta_data are ordered prime by prime, powers consecutive: mask out the powers of p <=
     PPMAX
        cnt = sum(len(g[1]) for g in GR); maskPP = np.ones(len(w), bool); maskPP[:cnt] = False
    Tg = np.unique(np.concatenate([np.geomspace(1, 1e6, 260), [1.0]]))
    base, Phi = elem_data(d, B, Tg); logT = np.log(Tg)
    sc = np.array([(d-0.5)**j for (_, j) in B])        # scaling for conditioning
    bfine = np.linspace(0, 0.5, 201); tp = np.geomspace(1e-8, 3000., 12000); tfine = np.concatenate([-tp
     [::-1], tp])
    for it in range(maxit):
        y = cp.Variable(nB); z = cp.Variable(); c = cp.multiply(sc, y)
        cons = [G@c >= Fv, 2*d*c[i1]-2*c[i2] >= eta]
        sign = np.where(isR, 1.0, -1.0)
        Fu = base+(Phi@c)/math.pi-c[i1]/(2*math.pi)*logT
        Fl = -base+(Phi@cp.multiply(sign, c))/math.pi-c[i1]/(2*math.pi)*logT
        cons += [z >= Fu, z >= Fl, z >= 7/8-c[i1]/(2*math.pi)*math.log(2*math.pi)]
        if PPMAX:
            xg = np.linspace(0, 2*np.pi, 96, endpoint=False); tp_ = cp.Variable(len(GR))
            for gi, (p, ks, ra, rb, bz) in enumerate(GR):
                Cm = np.cos(np.outer(xg, ks)); Sm = np.sin(np.outer(xg, ks))
                cons.append(tp_[gi] >= Cm@(ra@c)+Sm@(rb@c+bz))
            Cz = cp.sum(tp_)+cp.sum(cp.multiply(w[maskPP], cp.norm(cp.vstack([RA[maskPP]@c, RB[maskPP]@c+b0[
     maskPP]]), 2, axis=0)))
        else:
            Cz = cp.sum(cp.multiply(w, cp.norm(cp.vstack([RA@c, RB@c+b0]), 2, axis=0)))
```

```
        obj = c[i1]/(2*math.pi)*Lstar+Cz+z
        prob = cp.Problem(cp.Minimize(obj), cons); prob.solve(solver=cp.CLARABEL)
        cv = sc*y.value
        nb = []; nt = []; worst = -1
        for b in bfine:
            Gr, Fr = kernel_rows(d, B, np.full_like(tfine, b), tfine); v = Fr+eta/(1+tfine**2)-Gr@cv; worst =
     max(worst, v.max())
            idx = np.where((v > 0) & (v >= np.roll(v, 1)) & (v >= np.roll(v, -1)))[0]; nb += [b]*len(idx); nt
     += list(tfine[idx])
        if verbose: print('  it %d obj=%.5f C1=%.5f Cz~%.4f z=%.4f worst=%.2e' % (it, prob.value, cv[i1]/(2*
    math.pi), Cz.value, z.value, worst), flush=True)
        if not nb: break
        G2, F2 = kernel_rows(d, B, np.array(nb), np.array(nt)); G = np.vstack([G, G2]); Fv = np.concatenate([
    Fv, F2+eta/(1+np.array(nt)**2)])
    return dict(d=d, B=B, c=list(map(float, cv)), C1=float(cv[i1]/(2*math.pi)), Cz_est=float(Cz.value), z_est
    =float(z.value),
                A3_est=float(Cz.value+z.value), Lstar=Lstar, eta=eta, prof='B', discs=[], kappa=[], J=[], m=
    max(j for _, j in B))

if __name__ == '__main__':
    d = float(sys.argv[1]); mR = int(sys.argv[2]); mI = int(sys.argv[3]); Ls = float(sys.argv[4]); out = sys.
     argv[5]
    t0 = time.time(); r = optimize(d, mR, mI, Ls)
    print('RESULT d=%.3f mR=%d mI=%d L*=%g  A1=%.5f A3~%.4f  value@L*~%.4f  (%.0fs)' % (d, mR, mI, Ls, r['C1
     '], r['A3_est'], r['C1']*Ls+r['A3_est'], time.time()-t0), flush=True)
    json.dump(r, open(out, 'w'))
```

Appendix C. Outputs of the certification programs

Hybrid: `certify_hybrid.py hyb_m10_P.json 40`

```
d = 9/10  m = 10  profile P  discs used: 85
C = c_R1/(2pi) + sum kappa_k J_k/pi = 0.074888389970260602744  (ball [0.074888 +/- 3.91e-7] )
kernel part c_R1/(2pi) = 0.0656118899398
limit t^2 U at |t|->inf = [0.001761888763 +/- 9.16e-14] True
t in (0,T1]: certified, boxes 1648951
t in [-T1,0): certified, boxes 2054381
|t|>=T1: certified, boxes 155116
all certified;  C < 0.0748883899703   time 716s
```

Line ℓ_5: kernel inequality

```
d = 1  m = 5  profile B  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = 0.20175655042691417704  (ball [0.20176 +/- 3.45e-6] )
kernel part c_R1/(2pi) = 0.201756550427
limit t^2 U at |t|->inf = [1.133488684 +/- 3.74e-10] True
t in (0,T1]: certified, boxes 10255
t in [-T1,0): certified, boxes 4207
|t|>=T1: certified, boxes 2
all certified;  C < 0.201756550427   time 1s
```

Line ℓ_5: C_ζ

```
per-prime phase maximum for p <= 1000 : gain [0.13567339 +/- 3.39e-9]
primes 5761455  partial sum (n<=100000000) = [0.700266525818397 +/- 2.62e-16]  psi(N0) = [99998242.7966268
     +/- 1.77e-8]  102s
g monotone on [2000, oo): crude bound [+/- 4.40e-3] < sigma1
tail <= [0.003333488646 +/- 2.02e-13]  (integral [0.003174276158 +/- 3.32e-14] , far tail [2.6758e-167 +/-
     3.15e-172] )
C_zeta < 0.703600014464    time 103s
```

Line ℓ_5: elementary part, mode N

```
intervals 9215  sup F_u on [1,T0] <= 0.6653662947   sup F_l <= -0.7495389120
T>=T0: F_u <= 0.5045914643  F_l <= -1.245408536  (K= [3.95201 +/- 3.63e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.665366294652
```

Line ℓ_5: elementary part, mode S

```
intervals 9215  sup F_u on [1,T0] <= -0.2236678259   sup F_l <= 0.1395021165
T>=T0: F_u <= -0.3703919570  F_l <= -0.3703919570  (K= [4.11780 +/- 2.69e-8] )
SUP over T>=1 of max(F_u,F_l) <= 0.139502116497
```

Line ℓ_{20}: kernel inequality

```
d = 6/5  m = 5  profile B  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = 0.10822193548420830108  (ball [0.10822 +/- 1.94e-6] )
kernel part c_R1/(2pi) = 0.108221935484
limit t^2 U at |t|->inf = [0.02372706155 +/- 2.76e-12] True
t in (0,T1]: certified, boxes 473575
t in [-T1,0): certified, boxes 279773
|t|>=T1: certified, boxes 1022
all certified;  C < 0.108221935484   time 54s
```

Line ℓ_{20}: C_ζ

```
per-prime phase maximum for p <= 1000 : gain [0.21171880 +/- 3.25e-9]
primes 5761455  partial sum (n<=100000000) = [1.46785649757579 +/- 4.11e-15]  psi(N0) = [99998242.7966268 +/-
      1.77e-8]  100s
g monotone on [2000, oo): crude bound [+/- 0.0145] < sigma1
tail <= [0.0002188390453 +/- 1.36e-15]  (integral [0.0002060161219 +/- 1.10e-14] , far tail [7.2489e-238 +/-
     1.12e-244] )
C_zeta < 1.46807533662    time 101s
```

Line ℓ_{20}: elementary part, mode N

```
intervals 9215  sup F_u on [1,T0] <= 0.7020709510   sup F_l <= -1.072389150
T>=T0: F_u <= 0.6766231015  F_l <= -1.073376898  (K= [5.21715 +/- 1.54e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.702070950967
```

Line ℓ_{20}: elementary part, mode S

```
intervals 9215  sup F_u on [1,T0] <= -0.1827863047   sup F_l <= -0.1965944845
T>=T0: F_u <= -0.1983603198  F_l <= -0.1983603198  (K= [5.38293 +/- 4.87e-6] )
SUP over T>=1 of max(F_u,F_l) <= -0.182786304679
```

Line ℓ_{60}: kernel inequality

```
d = 1  m = 5  profile B  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = 0.094721574236353990059  (ball [0.094722 +/- 4.26e-7] )
kernel part c_R1/(2pi) = 0.0947215742364
limit t^2 U at |t|->inf = [0.008043907939 +/- 4.87e-13] True
t in (0,T1]: certified, boxes 1441685
t in [-T1,0): certified, boxes 483889
|t|>=T1: certified, boxes 4434
all certified;  C < 0.0947215742364   time 115s
```

Line ℓ_{60}: C_ζ

```
per-prime phase maximum for p <= 1000 : gain [0.24105429 +/- 2.19e-10]
primes 5761455  partial sum (n<=100000000) = [2.09743955289942 +/- 3.84e-15]  psi(N0) = [99998242.7966268 +/-
      1.77e-8]  104s
g monotone on [2000, oo): crude bound [+/- 0.0149] < sigma1
tail <= [0.006829934351 +/- 6.03e-14]  (integral [0.006476069071 +/- 3.90e-13] , far tail [1.7143e-168 +/-
     4.48e-173] )
C_zeta < 2.10426948725    time 105s
```

Line ℓ_{60}: elementary part, mode N

```
intervals 9215  sup F_u on [1,T0] <= 0.7289418077   sup F_l <= -1.047579459
T>=T0: F_u <= 0.7013136726  F_l <= -1.048686327  (K= [4.00282 +/- 3.70e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.728941807706
```

Line ℓ_{60}: elementary part, mode S

```
intervals 9215  sup F_u on [1,T0] <= -0.1584755869   sup F_l <= -0.1717847945
T>=T0: F_u <= -0.1736697487  F_l <= -0.1736697487  (K= [4.16860 +/- 2.71e-6] )
SUP over T>=1 of max(F_u,F_l) <= -0.158475586900
```

Line ℓ_{150}: kernel inequality

```
d = 17/20  m = 5  profile B  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = 0.093480628436037879597  (ball [0.093481 +/- 3.72e-7] )
kernel part c_R1/(2pi) = 0.0934806284360
limit t^2 U at |t|->inf = [0.007450671016 +/- 2.69e-13] True
t in (0,T1]: certified, boxes 1658655
t in [-T1,0): certified, boxes 353921
|t|>=T1: certified, boxes 3198
all certified;  C < 0.0934806284360   time 127s
```

Line ℓ_{150}: C_ζ

```
per-prime phase maximum for p <= 1000 : gain [0.25085637 +/- 3.02e-9]
primes 5761455  partial sum (n<=100000000) = [2.21455819652827 +/- 4.70e-15]  psi(N0) = [99998242.7966268 +/-
      1.77e-8]  107s
g monotone on [2000, oo): crude bound [+/- 7.01e-3] < sigma1
tail <= [0.04122525901 +/- 2.13e-12]  (integral [0.03953310421 +/- 1.61e-12] , far tail [3.6826e-115 +/- 3.45
     e-120] )
C_zeta < 2.25578345554    time 107s
```

Line ℓ_{150}: elementary part, mode N

```
intervals 9215  sup F_u on [1,T0] <= 0.7303170966   sup F_l <= -1.045307483
T>=T0: F_u <= 0.7035135754  F_l <= -1.046486425  (K= [3.19479 +/- 4.84e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.730317096644
```

Line ℓ_{150}: elementary part, mode S

```
intervals 9215  sup F_u on [1,T0] <= -0.1575762200   sup F_l <= -0.1695128190
T>=T0: F_u <= -0.1714698460  F_l <= -0.1714698460  (K= [3.36057 +/- 1.57e-6] )
SUP over T>=1 of max(F_u,F_l) <= -0.157576220006
```

Line ℓ_{400}: kernel inequality

```
d = 17/20  m = 10  profile B  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = 0.083986018406625982801  (ball [0.083986 +/- 1.86e-8] )
kernel part c_R1/(2pi) = 0.0839860184066
limit t^2 U at |t|->inf = [0.0002002739955 +/- 2.69e-14] True
t in (0,T1]: certified, boxes 11835193
t in [-T1,0): certified, boxes 9801985
|t|>=T1: certified, boxes 4194302
all certified;  C < 0.0839860184066   time 2716s
```

Line ℓ_{400}: C_ζ

```
per-prime phase maximum for p <= 1000 : gain [0.28179774 +/- 3.28e-9]
primes 5761455  partial sum (n<=100000000) = [4.01127229118254 +/- 1.94e-15]  psi(N0) = [99998242.7966268 +/-
      1.77e-8]  163s
g monotone on [5000, oo): crude bound [+/- 0.0114] < sigma1
tail <= [0.4524693305 +/- 1.73e-11]  (integral [0.4325160730 +/- 4.01e-11] , far tail [2.8606e-111 +/- 3.11e
     -116] )
C_zeta < 4.46374162170    time 164s
```

Line ℓ_{400}: elementary part, mode N

```
intervals 9215  sup F_u on [1,T0] <= 0.7375657198   sup F_l <= -1.027807129
T>=T0: F_u <= 0.7209635502  F_l <= -1.029036450  (K= [3.19527 +/- 2.94e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.737565719751
```

Line ℓ_{400}: elementary part, mode S

```
intervals 9215  sup F_u on [1,T0] <= -0.1458266308   sup F_l <= -0.1520124642
T>=T0: F_u <= -0.1540198712  F_l <= -0.1540198712  (K= [3.36106 +/- 6.67e-7] )
SUP over T>=1 of max(F_u,F_l) <= -0.145826630820
```

Line ℓ_{6000}: kernel inequality

```
d = 17/20  m = 10  profile B  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = 0.083814170602092884236  (ball [0.083814 +/- 1.71e-7] )
kernel part c_R1/(2pi) = 0.0838141706021
limit t^2 U at |t|->inf = [0.0002027460239 +/- 3.01e-14] True
t in (0,T1]: certified, boxes 14760921
t in [-T1,0): certified, boxes 15866017
|t|>=T1: certified, boxes 4194302
all certified;  C < 0.0838141706021   time 3243s
```

Line ℓ_{6000}: C_ζ

```
per-prime phase maximum for p <= 1000 : gain [0.28466228 +/- 4.66e-10]
primes 5761455  partial sum (n<=100000000) = [4.10087443545128 +/- 2.45e-15]  psi(N0) = [99998242.7966268 +/-
      1.77e-8]  81s
g monotone on [5000, oo): crude bound [+/- 0.0108] < sigma1
tail <= [0.4886359120 +/- 3.99e-12]  (integral [0.4671146906 +/- 3.15e-11] , far tail [2.9139e-111 +/- 4.83e
     -116] )
C_zeta < 4.58951034745    time 82s
```

Line ℓ_{6000}: elementary part, mode N

```
intervals 9215  sup F_u on [1,T0] <= 0.7357376568   sup F_l <= -1.027508246
T>=T0: F_u <= 0.7212794331  F_l <= -1.028720567  (K= [3.19575 +/- 1.13e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.735737656779
```

Line ℓ_{6000}: elementary part, mode S

```
intervals 9215  sup F_u on [1,T0] <= -0.1464102938   sup F_l <= -0.1517135820
T>=T0: F_u <= -0.1537039882  F_l <= -0.1537039882  (K= [3.36154 +/- 2.48e-6] )
SUP over T>=1 of max(F_u,F_l) <= -0.146410293767
```

Appendix D. Exact data

The following files hold the coefficients as exact binary floating-point numbers (the decimal strings are the shortest that round-trip), the disc data before rounding to rationals (the certificate rounds c_k, R_k, v_k to denominators $\leq 10^4$ and then replaces R_k by $\min(R_k, c_k - \frac{1}{2})$, as in `certify_hybrid.py`), and the parameters d, m, η.

`hyb_m10_P.json` (hybrid)


```
{
 "m": 10,
 "d": 0.9,
 "prof": "P",
 "eta": 0.001,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["R", 6],
  ["R", 7],
  ["R", 8],
  ["R", 9],
  ["R", 10],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5],
  ["I", 6],
  ["I", 7],
  ["I", 8],
  ["I", 9],
  ["I", 10]
 ],
 "c": [0.4122516628461088, 0.3701455521800437, 0.2843954476533836, 0.1586739250962677,
  0.06960218322485388, -0.06144517777745324, -0.04110670197918963, 0.07575215034770166,
  -0.03254194251685058, 0.005663161506506656, 0.7334328542232512, -0.1362131821835759,
  0.15145172743931834, -0.11733043416084597, -0.03620957744808127, 0.10598527663641981,
  -0.06234506214305649, 0.01672536385409436, -0.0017975838554094834],
 "discs": [
  [1.03, 0.18000000000000002, -0.5],
  [1.03, 0.18000000000000002, 0.5],
  [1.03, 0.23000000000000004, -0.65],
  [1.03, 0.23000000000000004, 0.65],
  [1.03, 0.28, -0.65],
  [1.03, 0.28, 0.65],
  [1.03, 0.33, -0.65],
  [1.03, 0.33, 0.65],
  [1.03, 0.33, 0.8],
  [1.03, 0.38, -0.65],
  [1.03, 0.38, 0.65],
  [1.03, 0.43000000000000005, 1.0],
  [1.03, 0.48000000000000004, -1.0],
  [1.03, 0.48000000000000004, 1.0],
  [1.03, 0.53, -1.6],
  [1.03, 0.53, -1.3],
  [1.03, 0.53, -1.0],
```

```
[1.03, 0.53, 1.0],
[1.03, 0.53, 1.3],
[1.06, 0.16000000000000006, -0.5],
[1.06, 0.16000000000000006, 0.5],
[1.06, 0.21000000000000005, -0.5],
[1.06, 0.21000000000000005, 0.5],
[1.06, 0.26000000000000006, -0.5],
[1.06, 0.26000000000000006, 0.5],
[1.06, 0.41000000000000003, -0.8],
[1.06, 0.41000000000000003, -0.65],
[1.06, 0.41000000000000003, 0.65],
[1.06, 0.41000000000000003, 0.8],
[1.06, 0.4600000000000001, 0.8],
[1.06, 0.56, -1.6],
[1.06, 0.56, -1.3],
[1.06, 0.56, 1.3],
[1.1, 0.15000000000000008, -0.35],
[1.1, 0.2500000000000001, -0.5],
[1.1, 0.2500000000000001, 0.5],
[1.1, 0.4000000000000001, 0.8],
[1.1, 0.5000000000000001, -0.8],
[1.1, 0.6000000000000001, -1.0],
[1.1, 0.6000000000000001, 1.0],
[1.1, 0.6000000000000001, 1.3],
[1.0986122886681098, 0.1986122886681098, -0.5],
[1.0986122886681098, 0.1986122886681098, 0.5],
[1.0986122886681098, 0.24861228866810978, 0.5],
[1.0986122886681098, 0.2986122886681098, -0.5],
[1.0986122886681098, 0.2986122886681098, 0.5],
[1.0986122886681098, 0.44861228866810976, -0.8],
[1.0986122886681098, 0.44861228866810976, 0.8],
[1.0986122886681098, 0.4986122886681098, -0.8],
[1.0986122886681098, 0.4986122886681098, 0.8],
[1.0986122886681098, 0.5486122886681097, 0.8],
[1.0986122886681098, 0.5986122886681098, -1.3],
[1.15, 0.1999999999999999, -0.5],
[1.15, 0.1999999999999999, -0.35],
[1.15, 0.1999999999999999, 0.35],
[1.15, 0.4499999999999999, -0.8],
[1.15, 0.4999999999999999, -0.8],
[1.15, 0.5499999999999999, -0.8],
[1.15, 0.5499999999999999, 0.8],
[1.15, 0.6499999999999999, -1.3],
[1.15, 0.6499999999999999, -1.0],
[1.15, 0.6499999999999999, 1.0],
[1.15, 0.6499999999999999, 1.3],
[1.2, 0.24999999999999994, 0.5],
[1.2, 0.7, -1.3],
[1.2, 0.7, -1.0],
[1.2, 0.7, 1.0],
[1.2, 0.7, 1.3],
[1.25, 0.3, -0.35],
[1.25, 0.3, 0.35],
[1.25, 0.75, -1.3],
[1.25, 0.75, -1.0],
[1.25, 0.75, 1.0],
[1.25, 0.75, 1.3],
[1.3, 0.8, -1.3],
[1.3, 0.8, -1.0],
[1.3, 0.8, 1.0],
[1.4, 0.4499999999999999, -0.35],
[1.4, 0.4499999999999999, 0.35],
[1.4, 0.8999999999999999, -1.3],
[1.4, 0.8999999999999999, 1.3],
[1.5, 0.55, -0.35],
```

```
  [1.5, 0.55, 0.35],
  [1.6094379124341003, 0.6594379124341003, -0.35],
  [1.6094379124341003, 0.6594379124341003, 0.35]
 ],
 "kappa": [0.04794121511240191, 0.03690501429948402, 0.04834170661391959,
  0.1573997379960051, 0.025878339185481502, 0.10616848331185574, 0.0236188805019172,
  0.10239002275273693, 0.0035455108226660923, 0.01901918959502707, 0.06465743477314946,
  0.0036165210446881912, 0.003351206340933337, 0.015268105926847056,
  0.00013974452258912968, 0.00029690557506736377, 0.0009108669135881503,
  0.0044801747294262095, 0.0014866484759580832, 0.04885368436282812, 0.16104424399371173,
  0.023453424086482003, 0.22759923236993776, 0.08939597402006162, 0.15196837128808857,
  0.013732157713891885, 0.018648005594139164, 0.08526394849897515, 0.017416570930371206,
  0.01374894904083696, 4.075855253142233e-06, 4.409961547443315e-05,
  0.0001588715206077226 4, 0.05505606780147275, 0.06981663462962027, 0.17735666987784413,
  0.07198100073739731, 0.0004564504972963723, 0.0006124208150023242, 0.0027472118552408628,
  0.0002146100180381553, 0.05850326139983594, 0.25492355822075796, 0.08999758511034867,
  0.010181593964904851, 0.1909304490478848, 0.0029738069635381367, 0.016589993444860514,
  0.0022927729672325117, 0.011430625597017244, 0.0019943154789860904,
  5.446961828302678e-05, 0.08543526352424897, 0.043464950358225625, 0.22638169555984836,
  0.0027941777234533577, 0.0033936220562532783, 0.006938963470320746, 0.03251223251077918,
  5.267650802806455e-05, 0.00043316823604372834, 0.0017765965271024221,
  0.0002226772506880452, 0.3345194179772268, 4.572964532994767e-05, 0.00034452393334691165,
  0.0015466014726329327, 0.0001463592826135654, 0.08907592125371844, 0.3795839670899206,
  3.644530875721061e-05, 0.00033552900976080115, 0.0014263339874773812,
  0.0002927499897014895 7, 4.5564918091396025e-05, 0.000824171377274319 7,
  0.003939227432175864, 0.37130750564186144, 0.6314222324457542, 0.00012348097971946856,
  0.0014416384256192418, 0.09582582544943793, 0.37811125601693457, 0.17674185090571795,
  0.48384344283661246],
 "J": [0.006913876985130073, 0.006913876985130073, 0.010786963484310835,
  0.010786963484310835, 0.015047271802978894, 0.015047271802978894, 0.019561733449682445,
  0.019561733449682445, 0.019561733449682445, 0.024731917448308403, 0.024731917448308403,
  0.03030669746033152, 0.03614020656354029, 0.03614020656354029, 0.04223496977173971,
  0.04223496977173971, 0.04223496977173971, 0.04223496977173971, 0.04223496977173971,
  0.0032593310610211064, 0.0032593310610211064, 0.006361647634553729, 0.006361647634553729,
  0.010091729993481463, 0.010091729993481463, 0.023721523851824588, 0.023721523851824588,
  0.023721523851824588, 0.023721523851824588, 0.029203800126850244, 0.04097279256246833,
  0.04097279256246833, 0.04097279256246833, 0.0007855777381792925, 0.00579739181011531,
  0.00579739181011531, 0.01761480160057199, 0.02790642447295338, 0.039455280415185943,
  0.039455280415185943, 0.039455280415185943, 0.0029087213249446332, 0.0029087213249446332,
  0.005814504320743654, 0.009368782031364846, 0.009368782031364846, 0.02258821735546429,
  0.02258821735546429, 0.02794854691757632, 0.02794854691757632, 0.03358841946708462,
  0.03950510294401413, 0.0006778485850694062, 0.0006778485850694062, 0.0006778485850694062,
  0.016535359036317563, 0.02130627562403806, 0.026506130501019225, 0.026506130501019225,
  0.03777806830297255, 0.03777806830297255, 0.03777806830297255, 0.03777806830297255,
  0.0006049812018696226, 0.036299512623474355, 0.036299512623474355, 0.036299512623474355,
  0.036299512623474355, 0.0005514853690123469, 0.0005514853690123469, 0.034982998649202596,
  0.034982998649202596, 0.034982998649202596, 0.034982998649202596, 0.033800772734260194,
  0.033800772734260194, 0.033800772734260194, 0.00044923232543557626,
  0.00044923232543557626, 0.03175777779183872, 0.03175777779183872, 0.000406003160664041,
  0.000406003160664041, 0.00037055380885643406, 0.00037055380885643406],
 "C": 0.07485530842562414,
 "note": "c_R1 increased by 2e-4 (fine-grid violation -8.9e-5 repaired by eps*G_R1, G_R1>0)"
}
```


line_L5/sol.json (line ℓ_5)


```
{
 "d": 1.0,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
```

```
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5]
 ],
 "c": [1.2676737932696245, 0.7009294514561635, -0.4946330486621655, 0.09751555052498845,
  -0.0005893962931625271, -0.3784073335097401, 0.13473062893139318, -0.01708382778797021,
  0.0001003433684765934],
 "C1": 0.20175655042691418,
 "Cz_est": 0.8391011111542401,
 "z_est": 0.665362725928392,
 "A3_est": 1.504463837082632,
 "Lstar": 5.0,
 "eta": 0.0002,
 "prof": "B",
 "discs": [],
 "kappa": [],
 "J": [],
 "m": 5,
 "val": 2.5132465892172027,
 "time": 8.346621751785278
}
```


`line_L20/sol.json` (line ℓ_{20})


```
{
 "d": 1.2,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5]
 ],
 "c": [0.6799784749489147, 0.8041106391650762, 0.7774270907421565, -0.008448613496901295,
  2.955972158446658e-05, 1.1843502097112268, -0.576537489831307, 0.00610293149500534,
  -2.1076986306361527e-05],
 "C1": 0.1082219354842083,
 "Cz_est": 1.6797641910534995,
 "z_est": 0.7020688851693405,
 "A3_est": 2.38183307622284,
 "Lstar": 20.0,
 "eta": 0.0002,
 "prof": "B",
 "discs": [],
 "kappa": [],
 "J": [],
 "m": 5,
 "val": 4.546271785907006,
 "time": 7.479561805725098
}
```


`line_L60/sol.json` (line ℓ_{60})


```
{
 "d": 1.0,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
```

```
  ["R", 4],
  ["R", 5],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5]
 ],
 "c": [0.5951532035147798, 0.5911312495450366, 0.3889216428262093, -0.004325167636924481,
  1.5378902475155497e-05, 0.8079466147098463, -0.3400480079099986, 0.003649167179891613,
  -1.2750467413103112e-05],
 "C1": 0.094721574236354,
 "Cz_est": 2.344884047970574,
 "z_est": 0.7289397780667036,
 "A3_est": 3.0738238260372777,
 "Lstar": 60.0,
 "eta": 0.0002,
 "prof": "B",
 "discs": [],
 "kappa": [],
 "J": [],
 "m": 5,
 "val": 8.757118280218517,
 "time": 8.941782712936401
}
```


`line_L150/sol.json` (line ℓ_{150})


```
{
 "d": 0.85,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5]
 ],
 "c": [0.5873561110952275, 0.4955273589230774, 0.17191023951649326, -0.04515429156322006,
  0.00027174675540190917, 0.5292986070563637, -0.26974254048270235, 0.03950670966367516,
  -0.00023459073769482711],
 "C1": 0.09348062843603788,
 "Cz_est": 2.5047342757708364,
 "z_est": 0.7303153300893256,
 "A3_est": 3.235049605860162,
 "Lstar": 150.0,
 "eta": 0.0002,
 "prof": "B",
 "discs": [],
 "kappa": [],
 "J": [],
 "m": 5,
 "val": 17.257143871265846,
 "time": 13.073456287384033
}
```


`line_C400/sol.json` (line ℓ_{400})


```
{
 "d": 0.85,
 "B": [
  ["R", 1],
```

```
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["R", 6],
  ["R", 7],
  ["R", 8],
  ["R", 9],
  ["R", 10],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5],
  ["I", 6],
  ["I", 7],
  ["I", 8],
  ["I", 9],
  ["I", 10]
 ],
 "c": [0.5276997168610267, 0.44844462233410926, 0.20734915440049123, -0.05170881362706953,
  0.04277439204584691, -0.0021686787194867547, 5.225366348519004e-05,
  -7.194711787780742e-07, 5.548232880956561e-09, -1.890562024872484e-11, 0.57950165380938,
  -0.24304154954646912, 0.09598460587810165, -0.022316290439150017, 0.0010166745274645633,
  -2.326790970420105e-05, 3.096452357377663e-07, -2.3274894920069496e-09,
  7.769753796000765e-12],
 "C1": 0.08398601840662599,
 "Cz_est": 4.723835786462193,
 "z_est": 0.7375635792618767,
 "A3_est": 5.46139936572407,
 "Lstar": 400.0,
 "eta": 0.0002,
 "prof": "B",
 "discs": [],
 "kappa": [],
 "J": [],
 "m": 10,
 "val": 39.05580672837446,
 "time": 38.073201417922974
}
```


`line_D6000/sol.json` (line ℓ_{6000})


```
{
 "d": 0.85,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["R", 6],
  ["R", 7],
  ["R", 8],
  ["R", 9],
  ["R", 10],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5],
  ["I", 6],
  ["I", 7],
  ["I", 8],
  ["I", 9],
  ["I", 10]
```

```
 ],
 "c": [0.5266199652605132, 0.44752559745950127, 0.21763335933091524, -0.06632610210813339,
  0.049224751946469816, -0.002454597973796714, 5.8415182530844107e-05,
  -7.96150671490144e-07, 6.08645994127779e-09, -2.0583832928014207e-11, 0.5834246015810649,
  -0.23667212703027896, 0.09567840750457196, -0.01858966229544025, 0.0008313848476117577,
  -1.89384002058713e-05, 2.517567194510178e-07, -1.8927048877024136e-09,
  6.322548809318692e-12],
 "C1": 0.08381417060209288,
 "Cz_est": 4.566001466369842,
 "z_est": 0.7357358598268879,
 "A3_est": 5.30173732619673,
 "Lstar": 6000.0,
 "eta": 0.0002,
 "prof": "B",
 "discs": [],
 "kappa": [],
 "J": [],
 "m": 10,
 "val": 508.18676093875405,
 "time": 46.305405139923096,
 "pp": 300
}
```


Universidad Nacional de Educación a Distancia, Madrid, Spain
Email address: vcasana1@alumno.uned.es